

# CSE 4303

## Data Structures

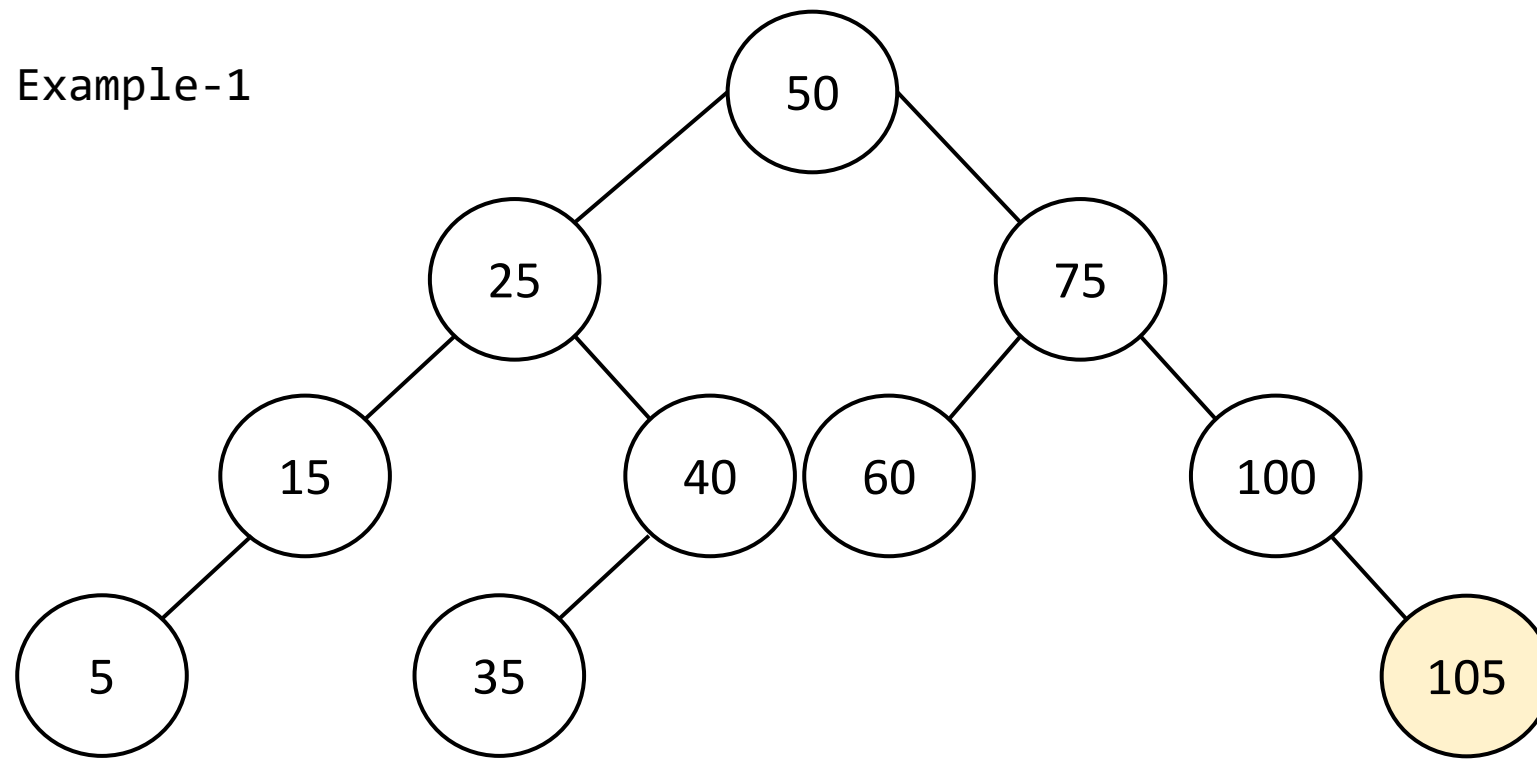
Topic: Deletion on Binary Search Trees

Sabbir Ahmed

Assistant Prof | CSE | IUT

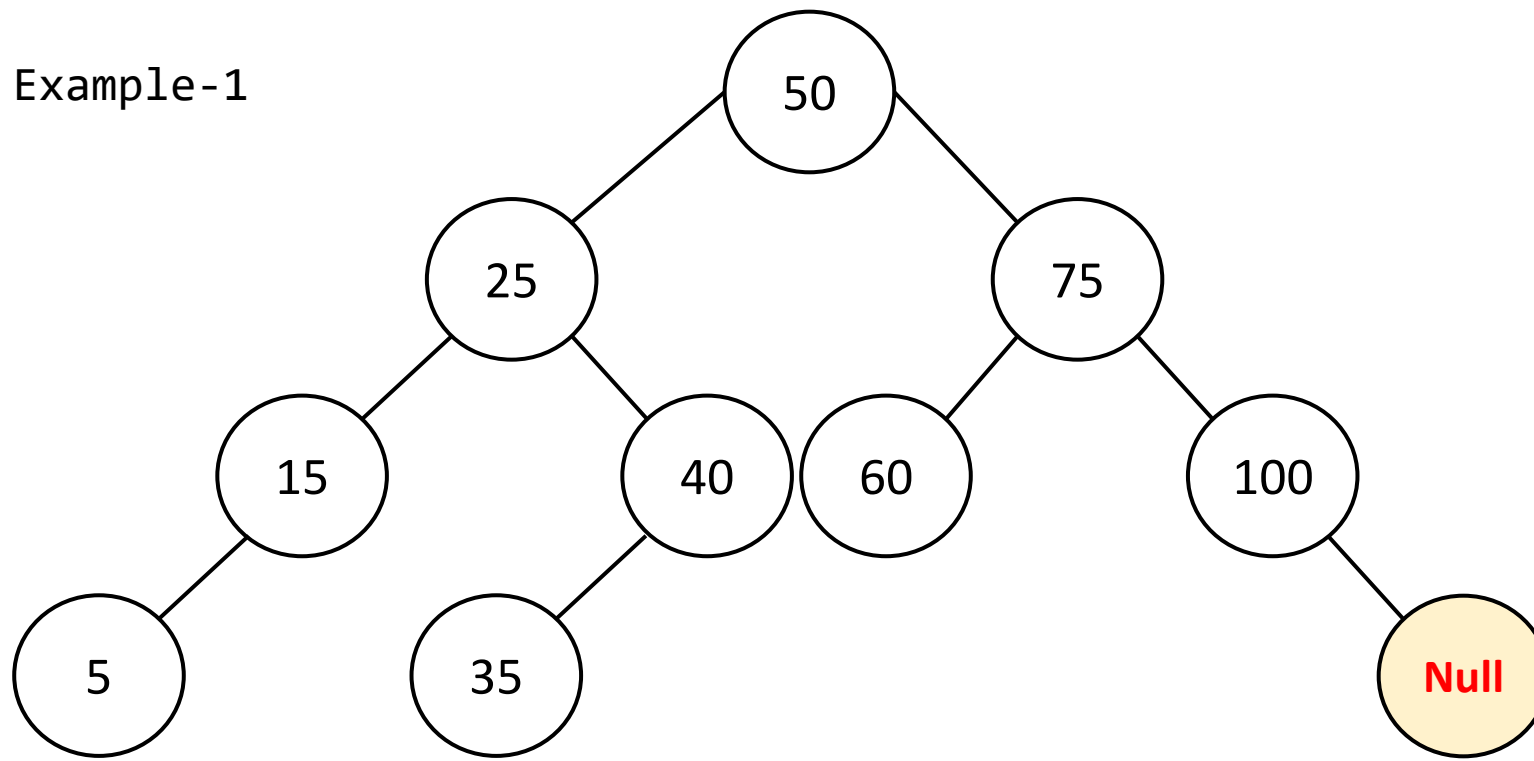
sabbirahmed@iut-dhaka.edu

Example-1



Delete Leaf  
Delete(105)

## Example-1

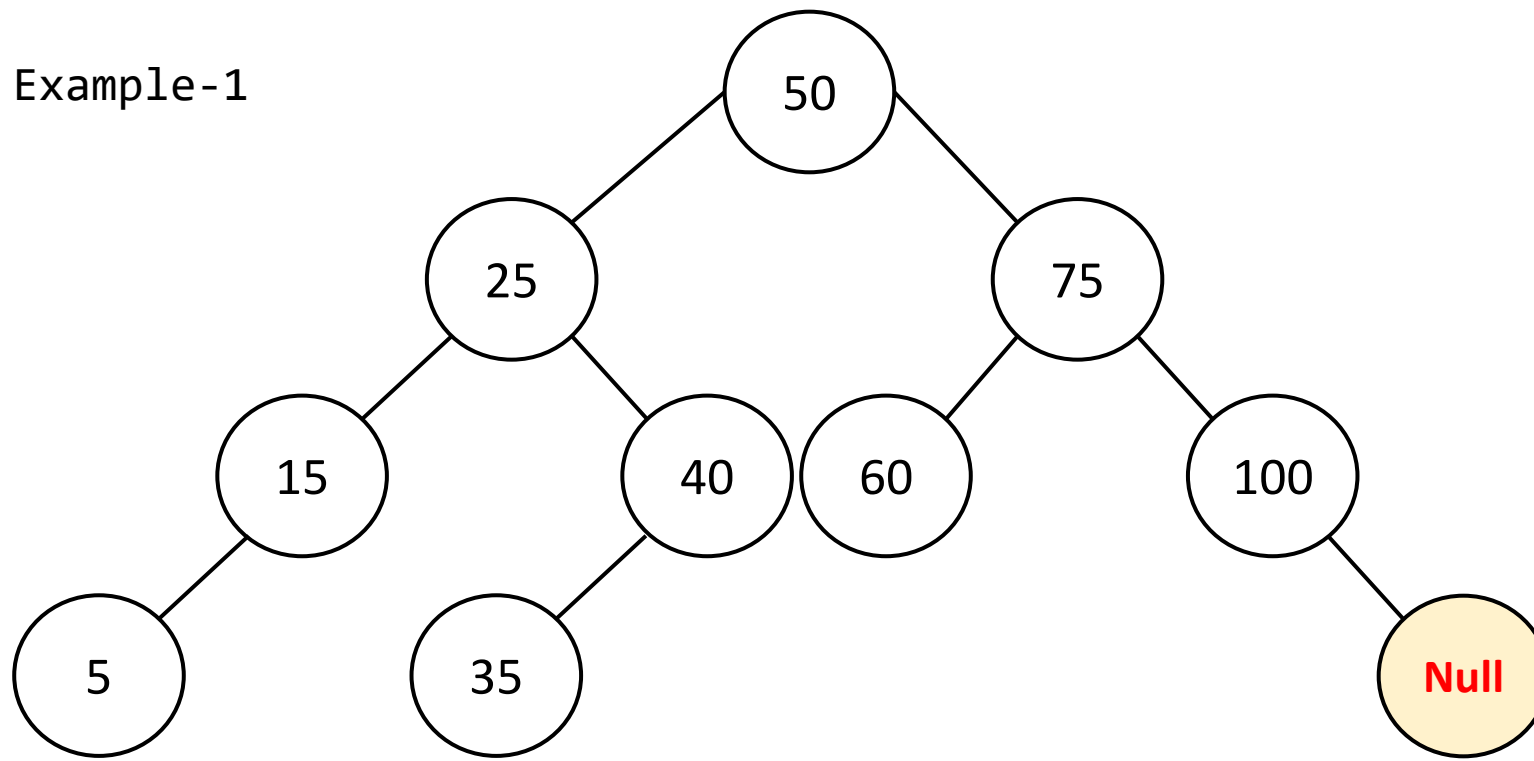


## Delete Leaf

Delete(105)

- 105 has No child (leaf)
- Can be replaced by Null!

### Example-1



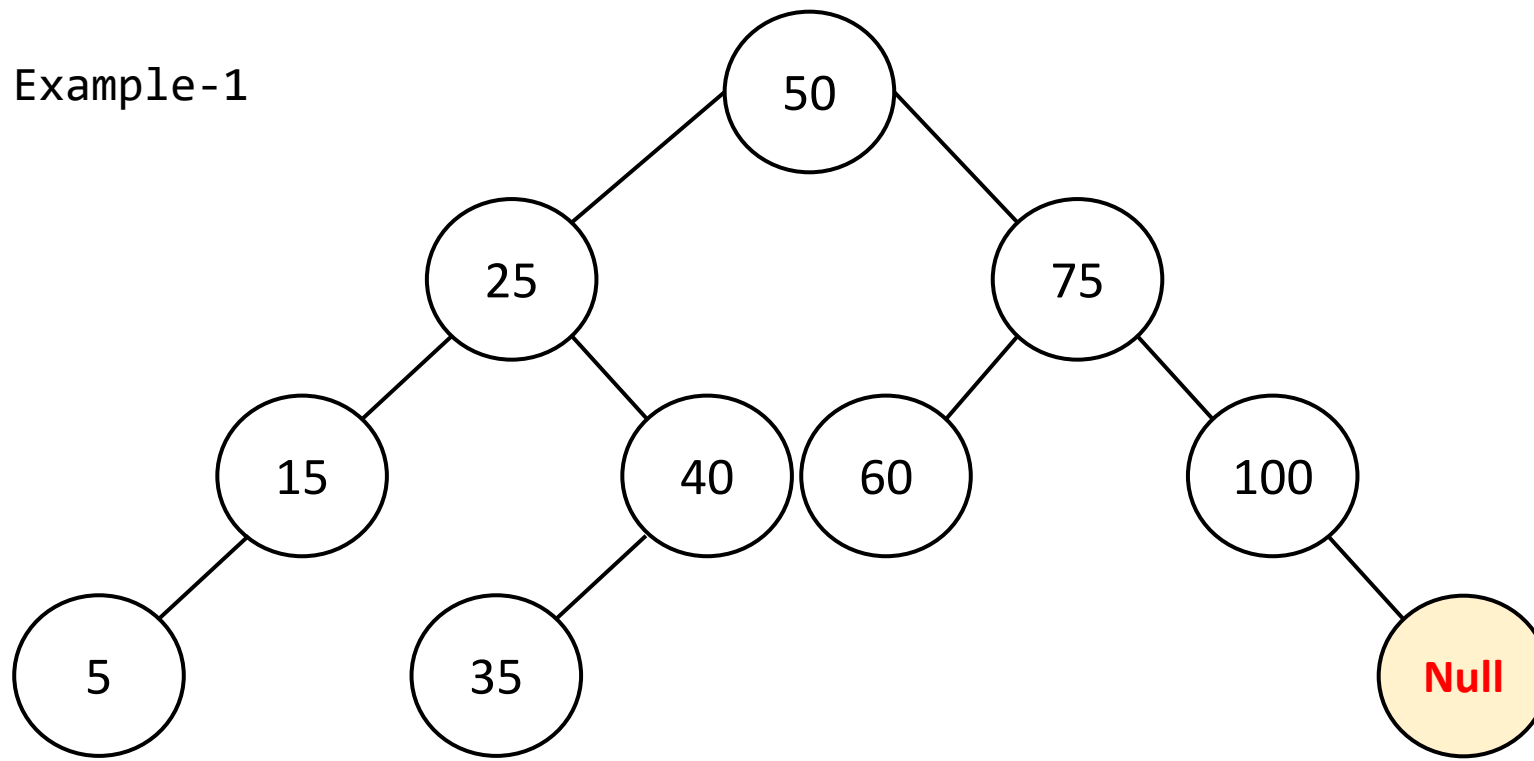
### Delete Leaf

Delete(105)

- 105 has No child (leaf)
- Can be replaced by Null!

Let,  $z = 105$  ( $z$  is the node to be deleted)

### Example-1



### Delete Leaf

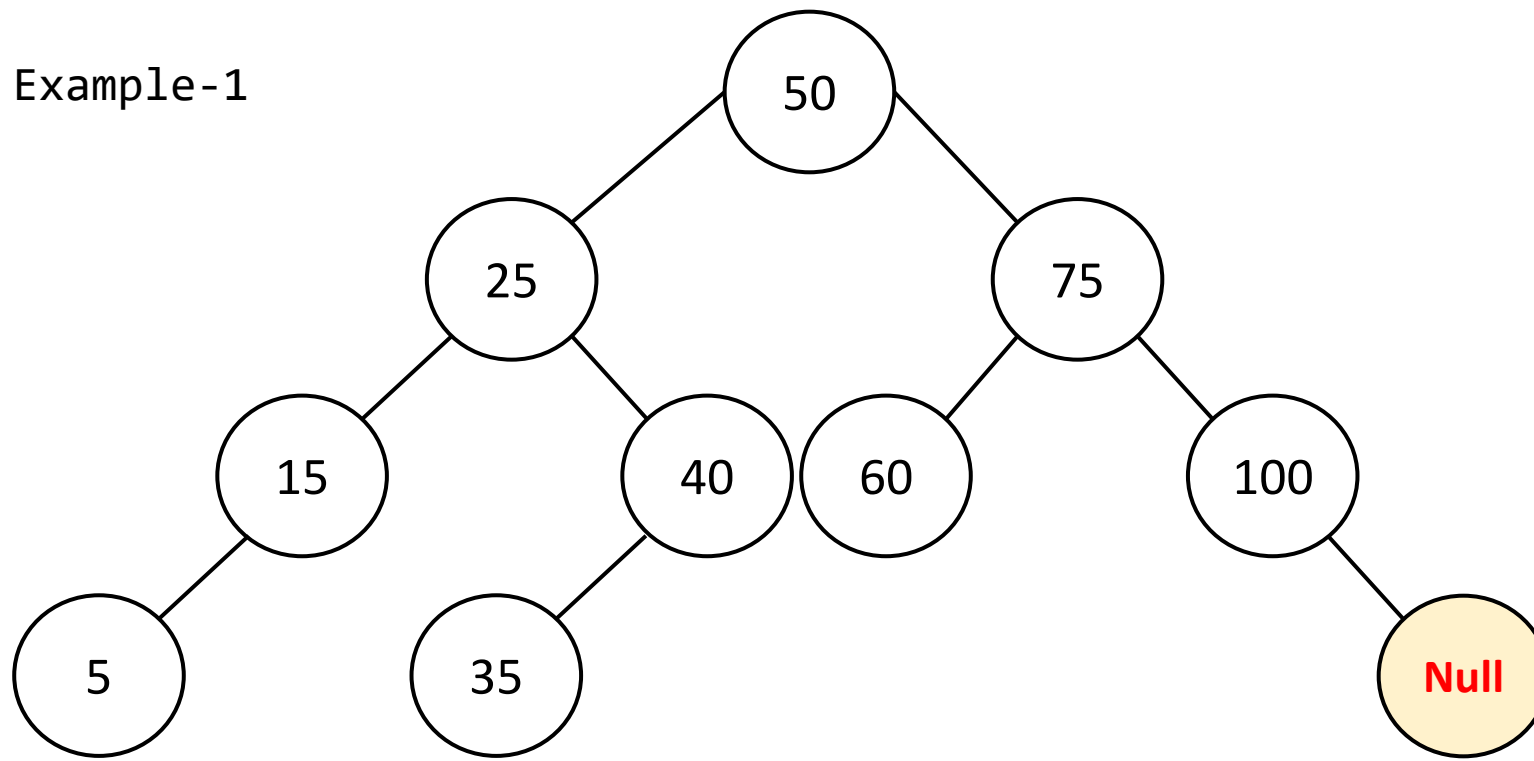
Delete(105)

- 105 has No child (leaf)
- Can be replaced by Null!

Let,  $z = 105$  ( $z$  is the node to be deleted)

If  $z$  is the right of its parent,  
 $z.p.right = Null$

### Example-1



### Delete Leaf

Delete(105)

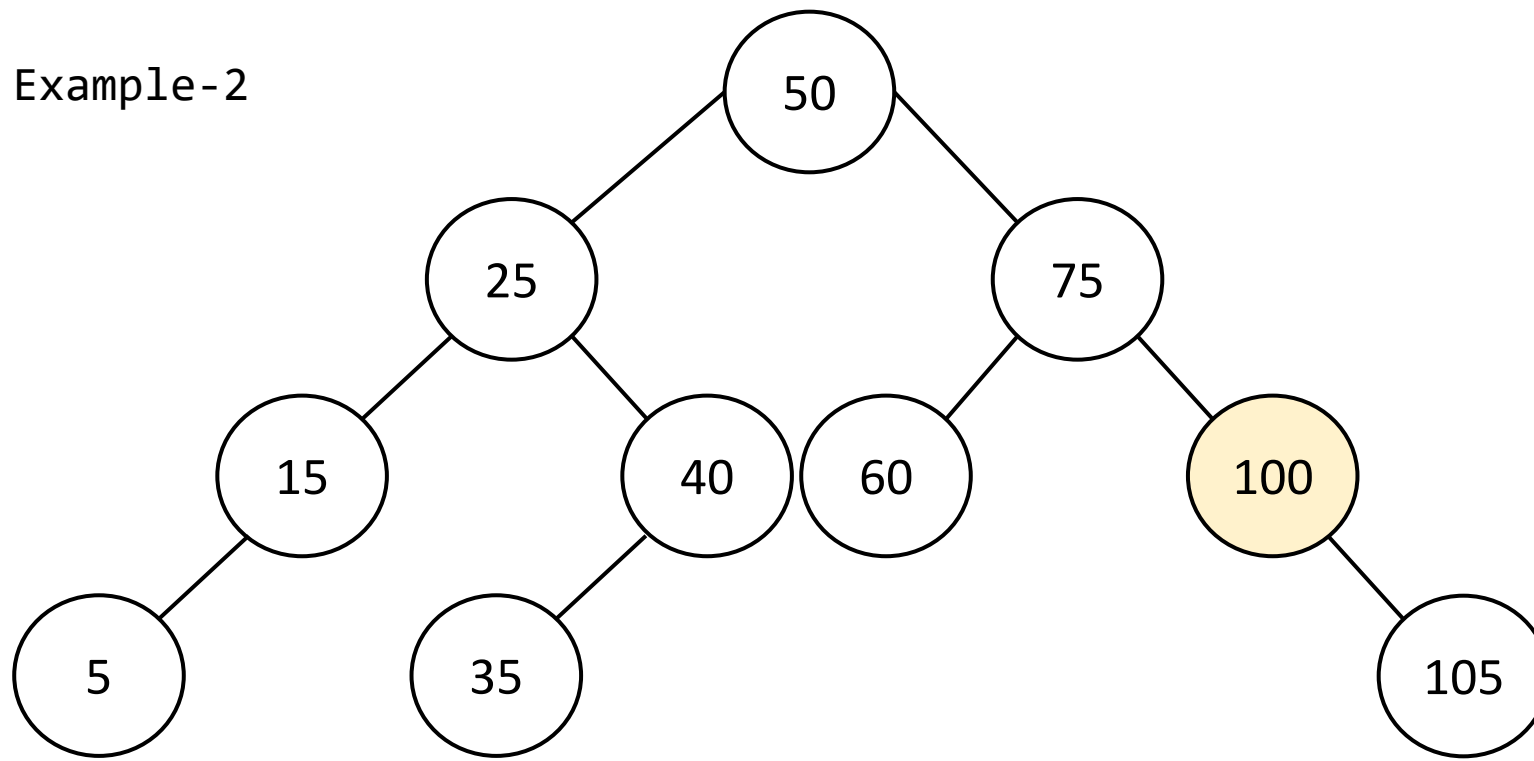
- 105 has No child (leaf)
- Can be replaced by Null!

Let,  $z = 105$  ( $z$  is the node to be deleted)

If  $z$  is the right of its parent,  
 $z.p.right = Null$

Else  $z.p.left = Null$

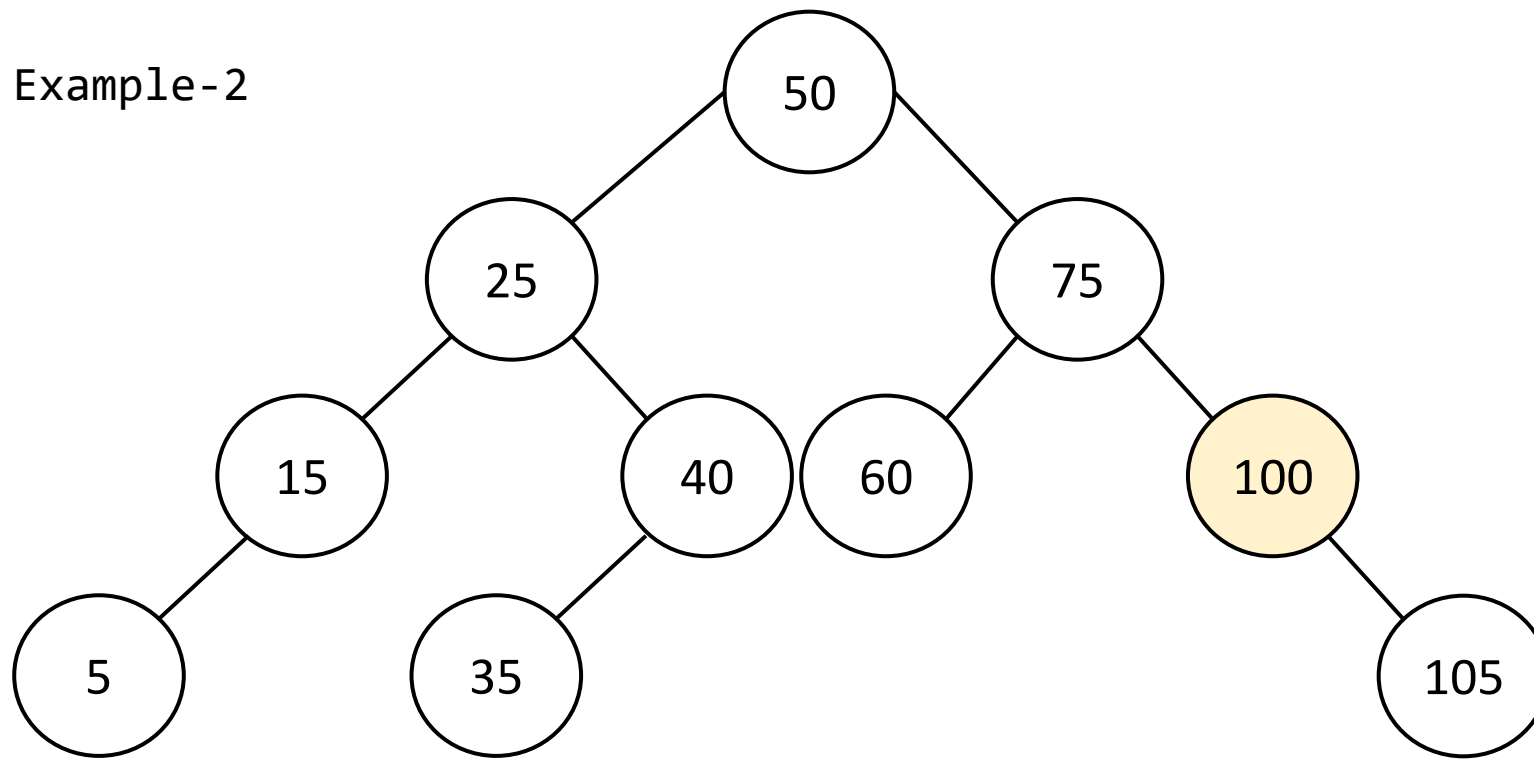
## Example-2



Delete Node with 1  
child

Delete(100)

## Example-2

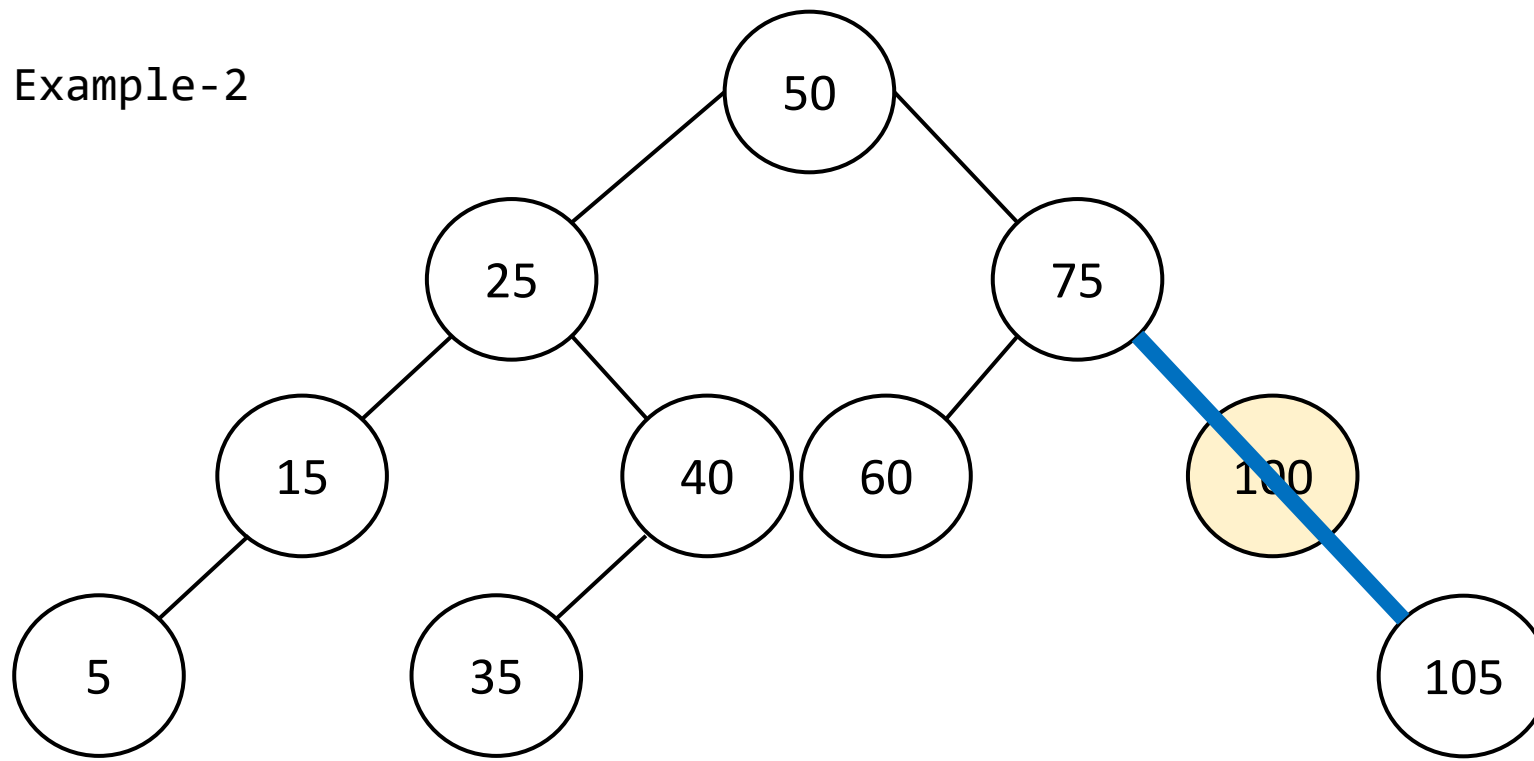


Delete Node with 1  
child

Delete(100)

- z=100, has 1 subtree
- z.right is not Null,  
but z.left is Null

## Example-2



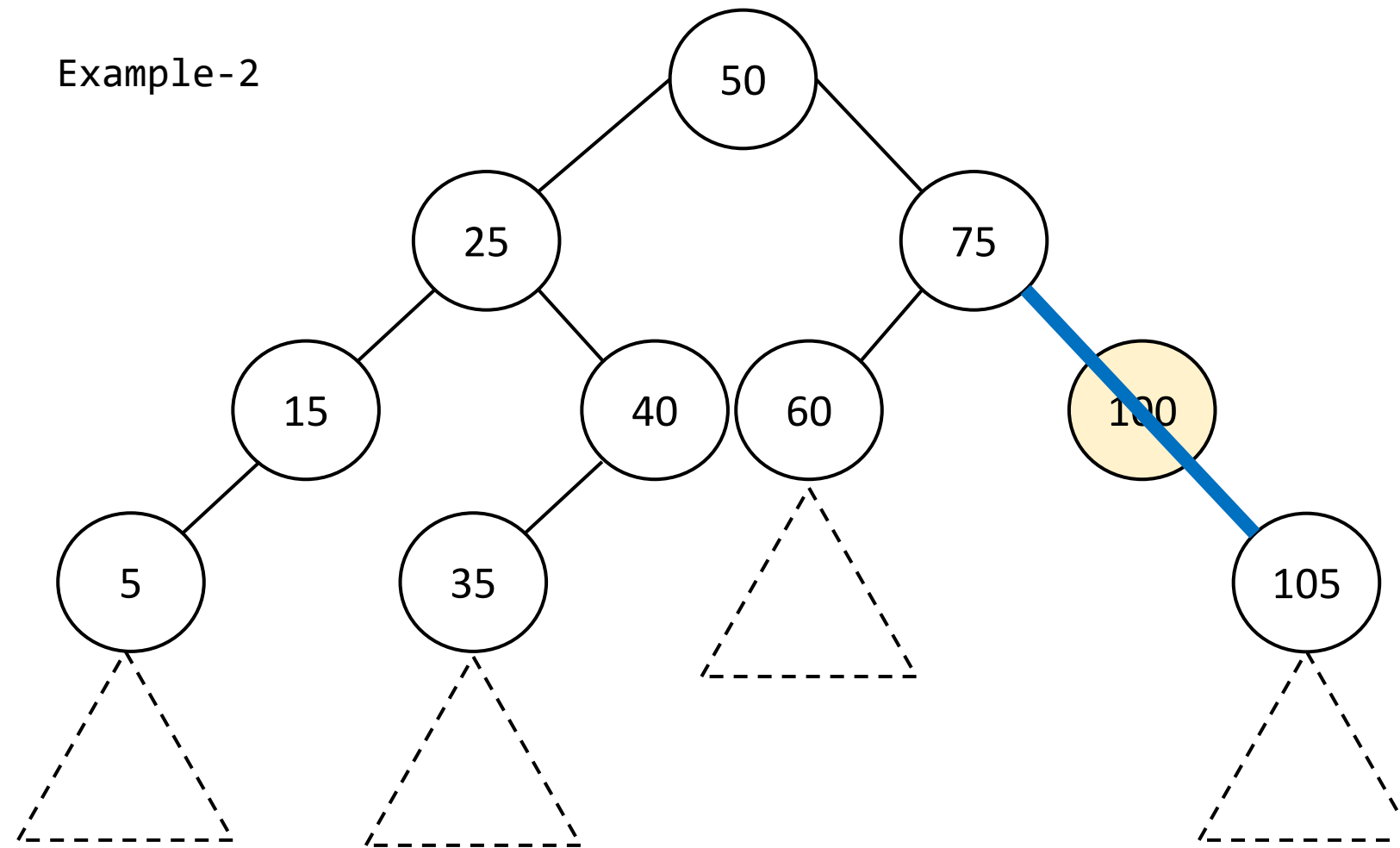
### Delete Node with 1 child

Delete(100)

- z=100 has 1 subtree
- z.right is not Null, but z.left is Null

So z.right can be connected with z.p

## Example-2



Delete Node with 1 child

Delete(100)

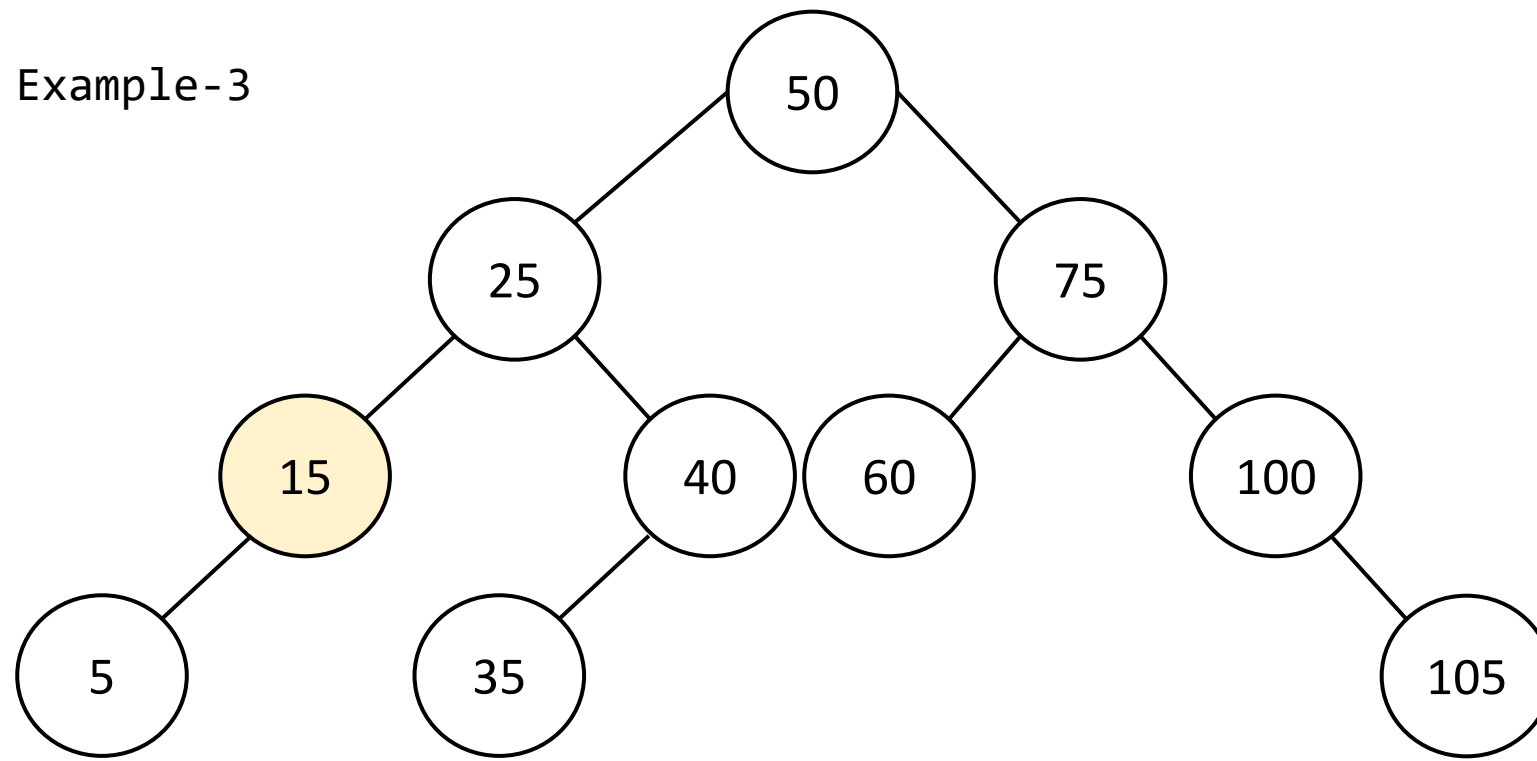
- z=100 has 1 subtree
- z.right is not Null, but z.left is Null

So z.right can be connected with z.p

*Assume, each node has a subtree under it.*

*Once a node is connected, the entire subtree gets connected*

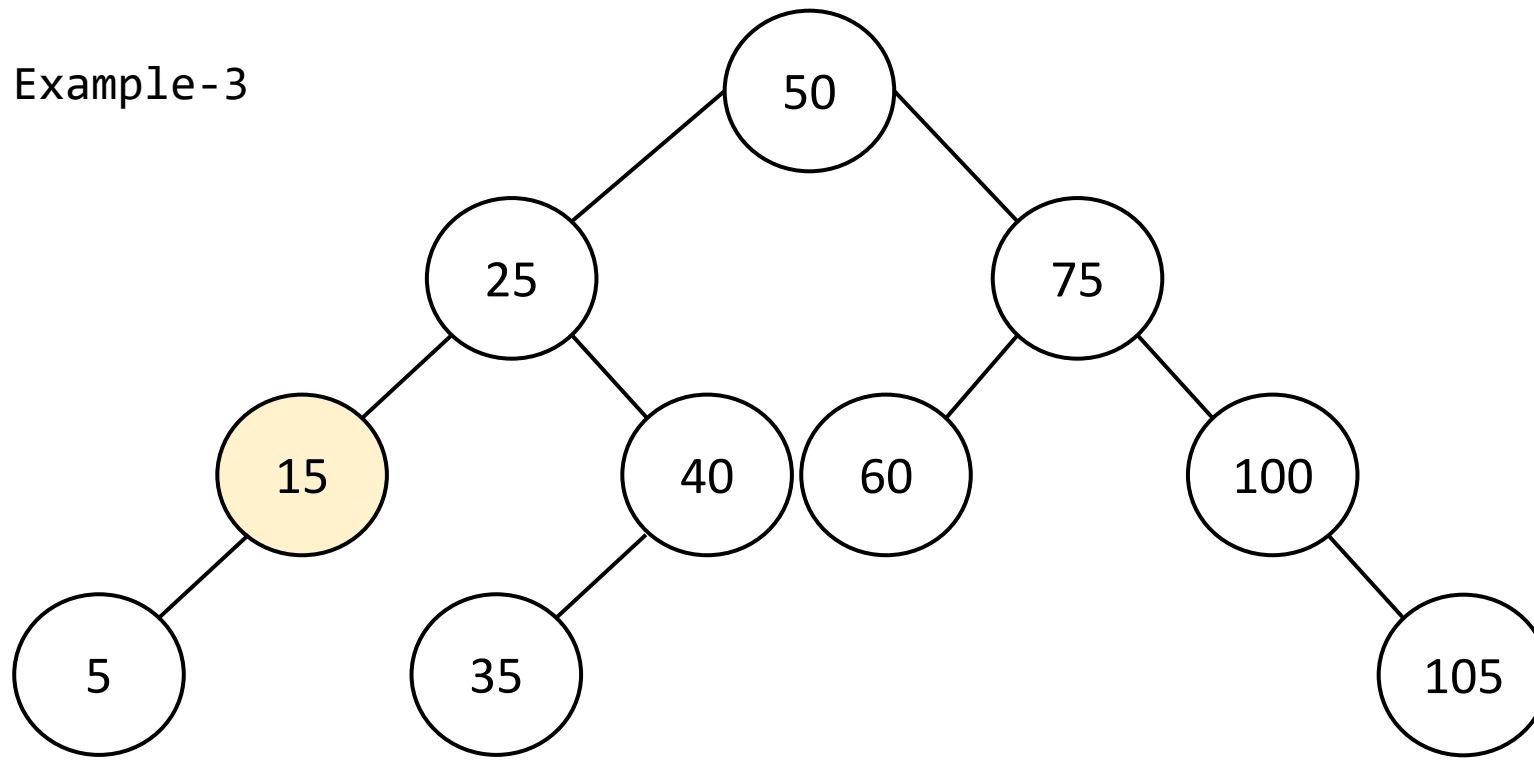
### Example-3



Delete Node with 1  
child

Delete(15)

### Example-3

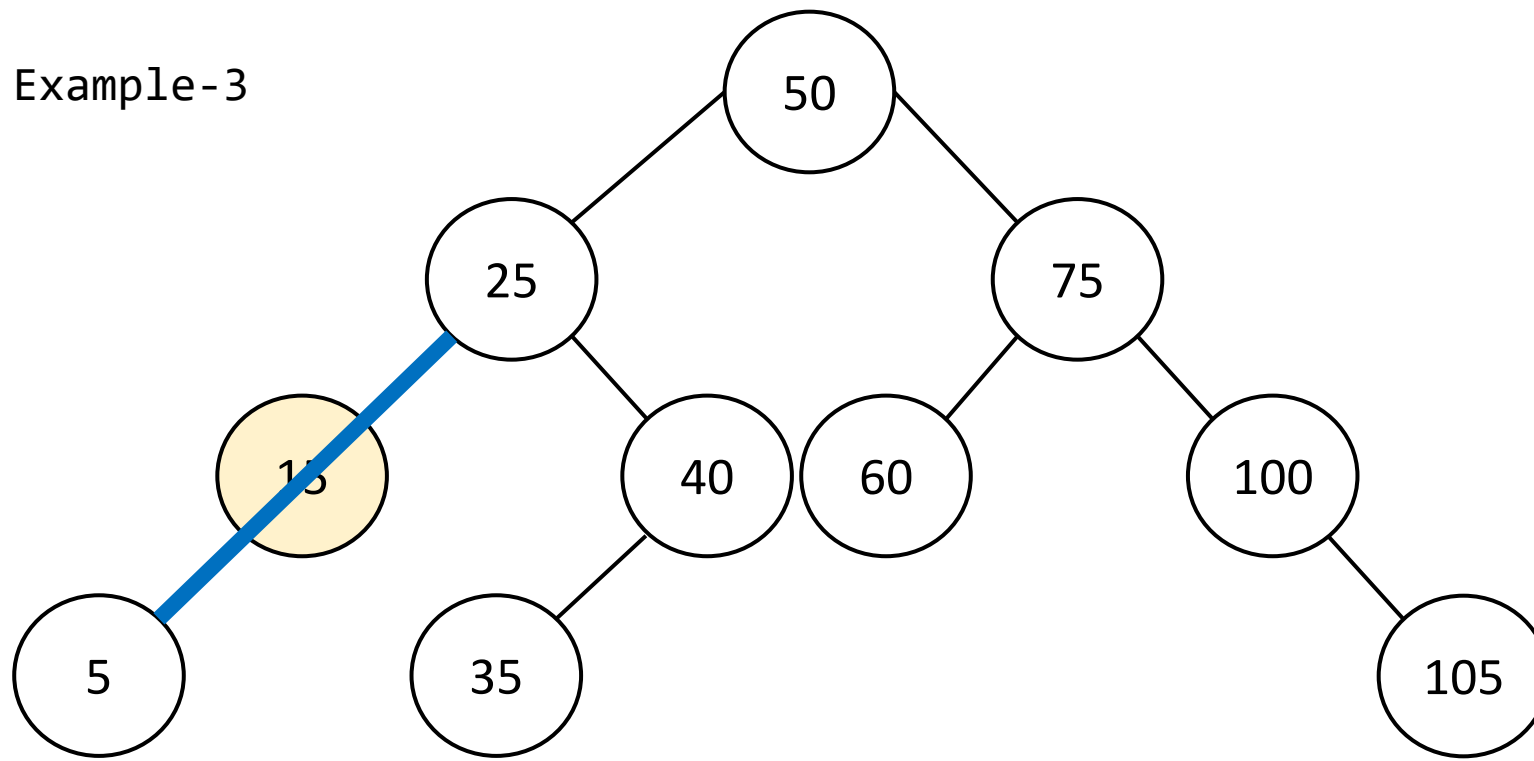


Delete Node with 1  
child

Delete(15)

- z=15 has 1 subtree
- z.left is not Null,  
but z.right is Null

### Example-3



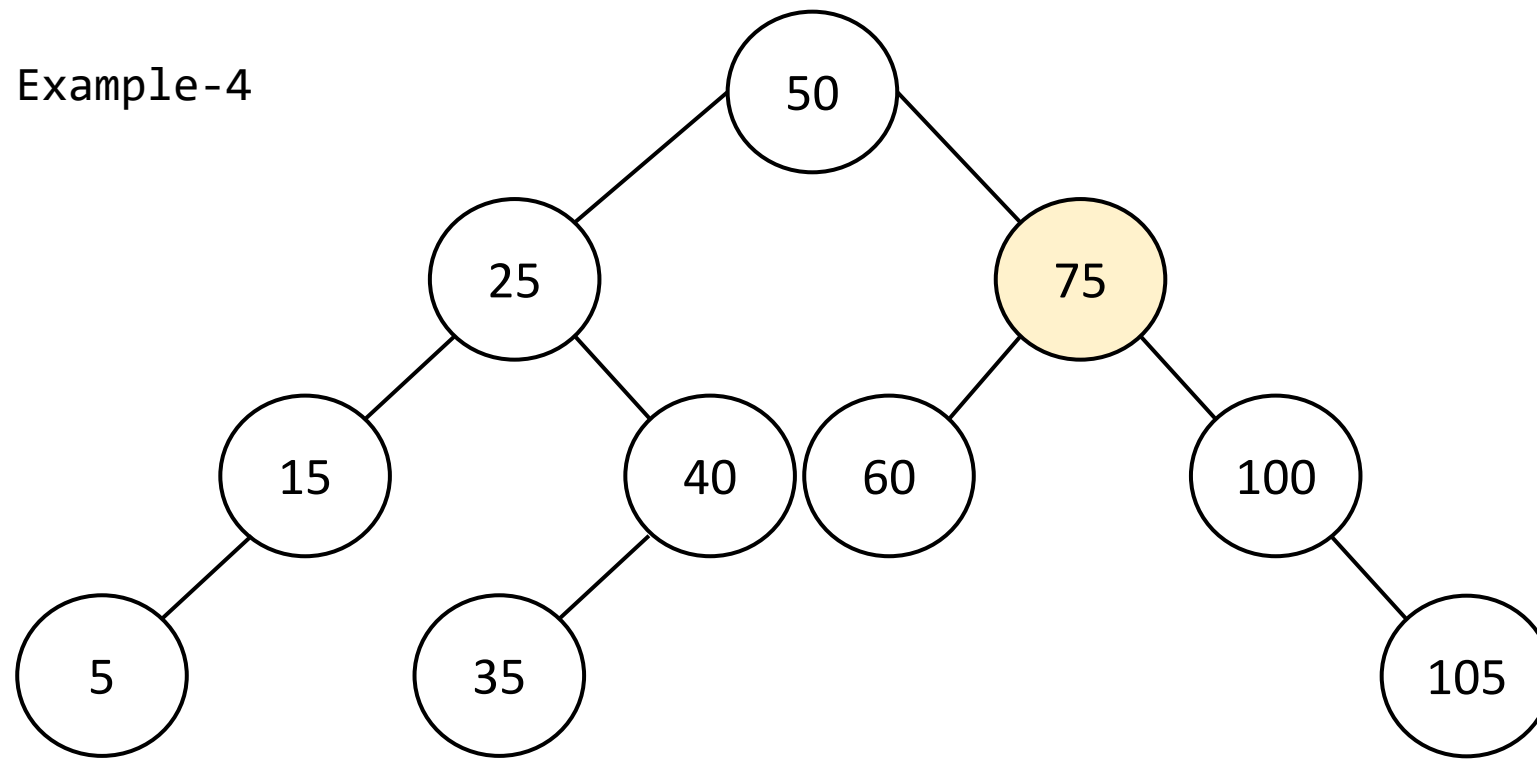
Delete Node with 1 child

Delete(15)

- z=15 has 1 subtree
- z.left is not Null, but z.right is Null

So z.left can be connected with z.p

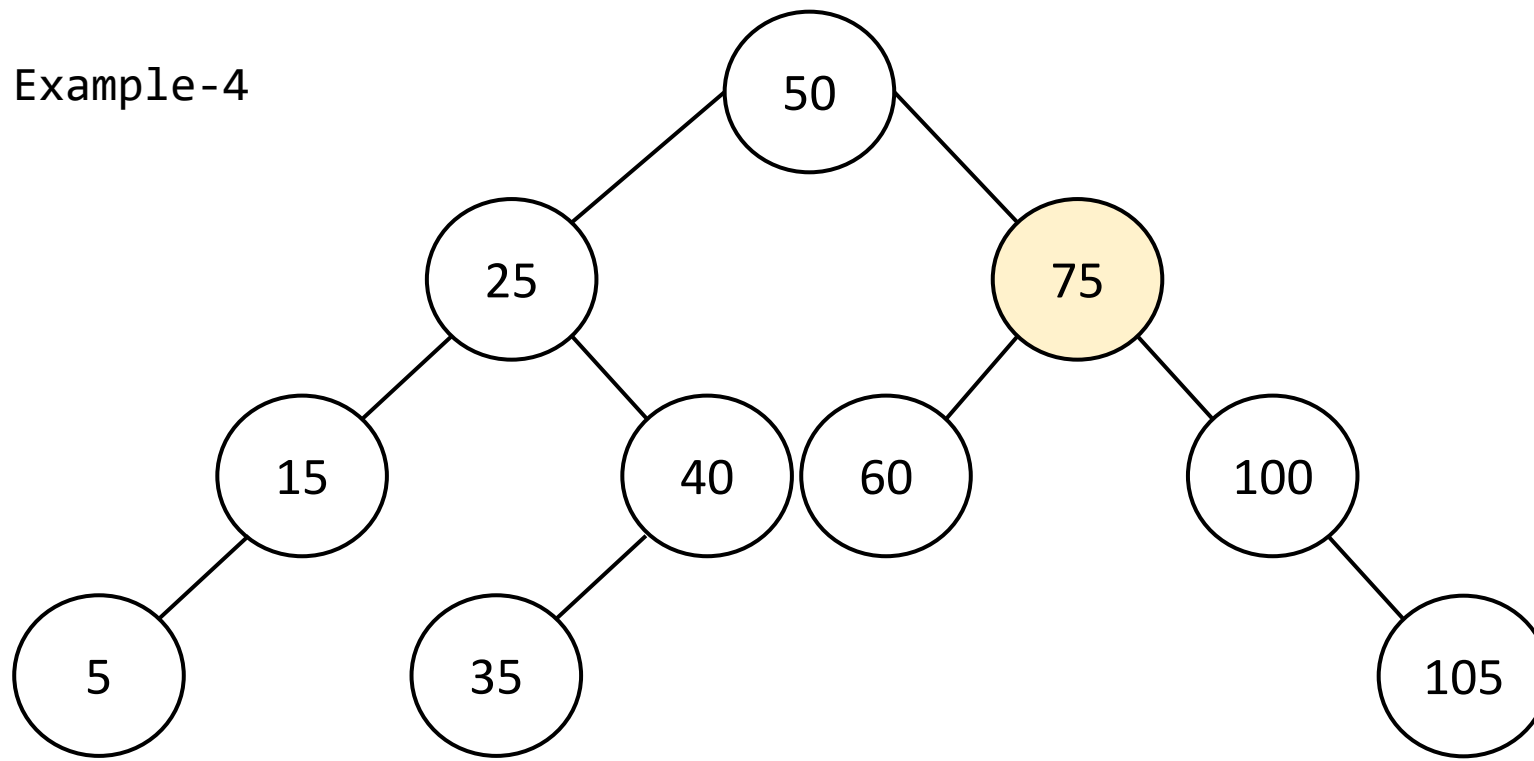
#### Example-4



Delete Node with 2  
child

Delete(75)

#### Example-4

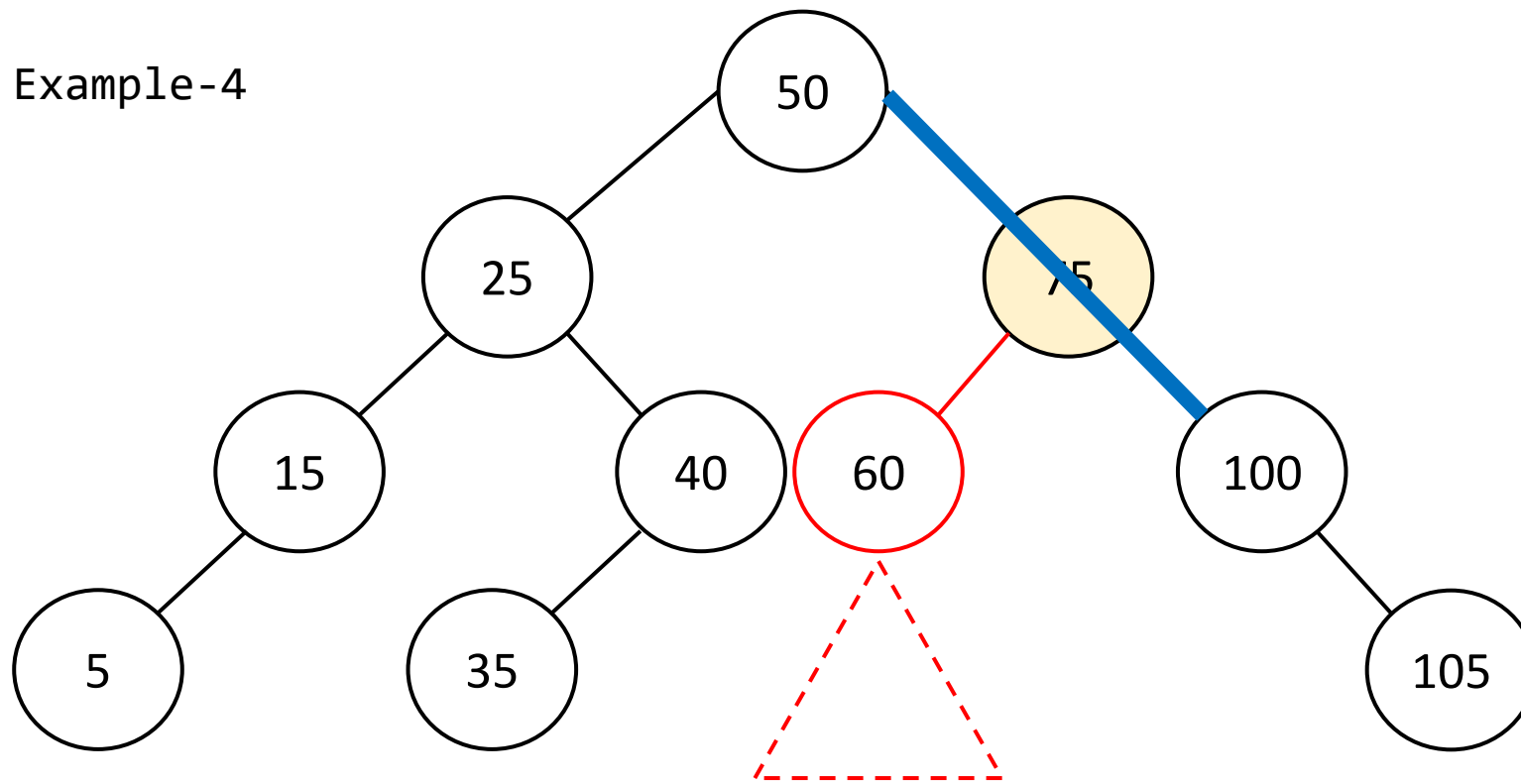


Delete Node with 2 child

Delete(75)

- z=75 has 2 subtrees
- z.left and z.right both exist

#### Example-4

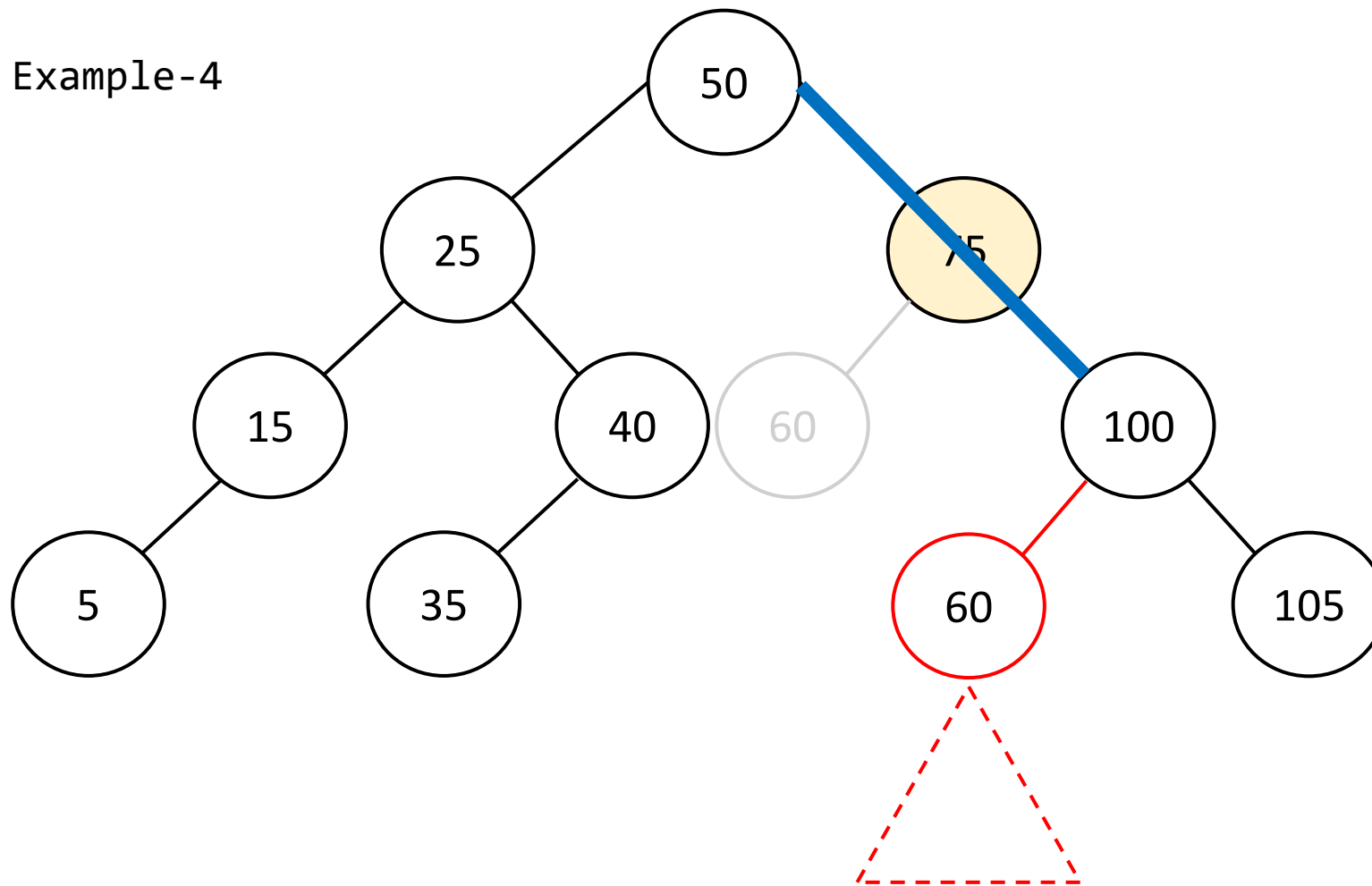


#### Delete Node with 2 child

Delete(75)

- z=75 has 2 subtrees
- z.left and z.right both exist
- z.p can be connected with z.right
- But **what about z.left subtree?**

#### Example-4

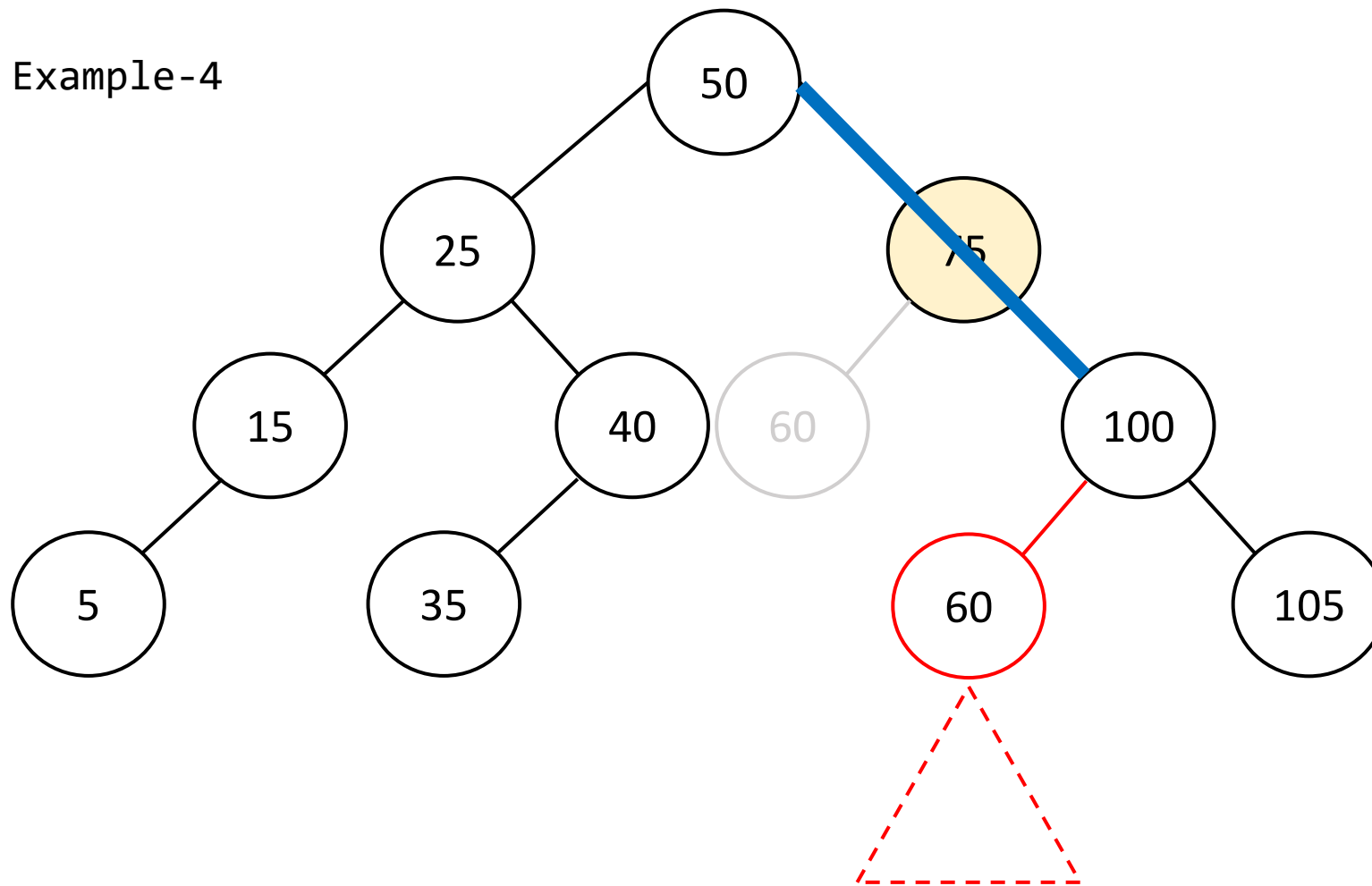


#### Delete Node with 2 child

Delete(75)

- z=75 has 2 subtrees
- z.left and z.right both exist
- z.p can be connected with z.right
- But what about z.left subtree?
- **Seems it can be connected with 100.left**

#### Example-4

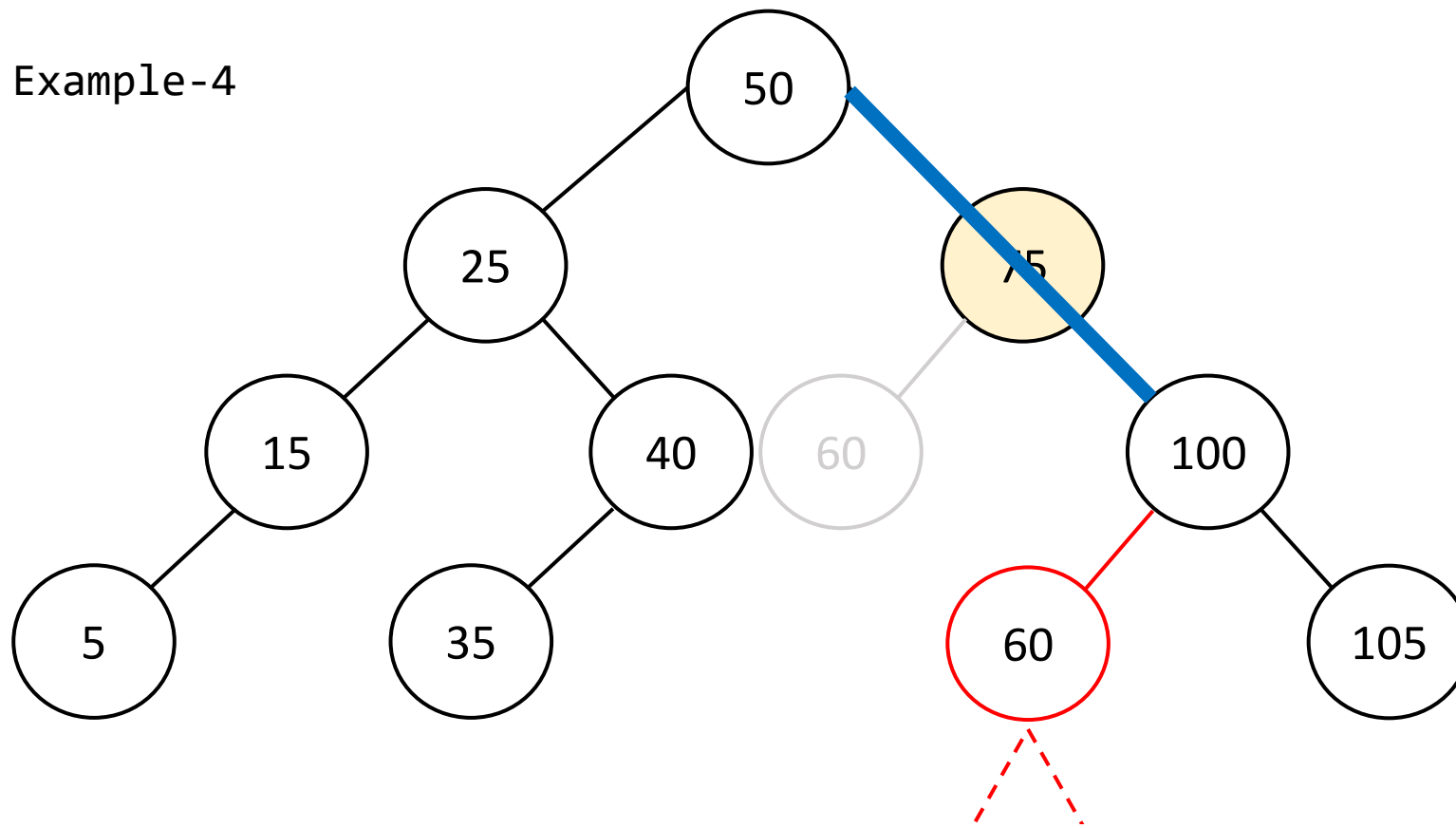


#### Delete Node with 2 child

Delete(75)

- z=75 has 2 subtrees
- z.left and z.right both exist
- z.p can be connected with z.right
- But what about z.left subtree?
- **Seems it can be connected with 100.left- but is it always possible?**

#### Example-4



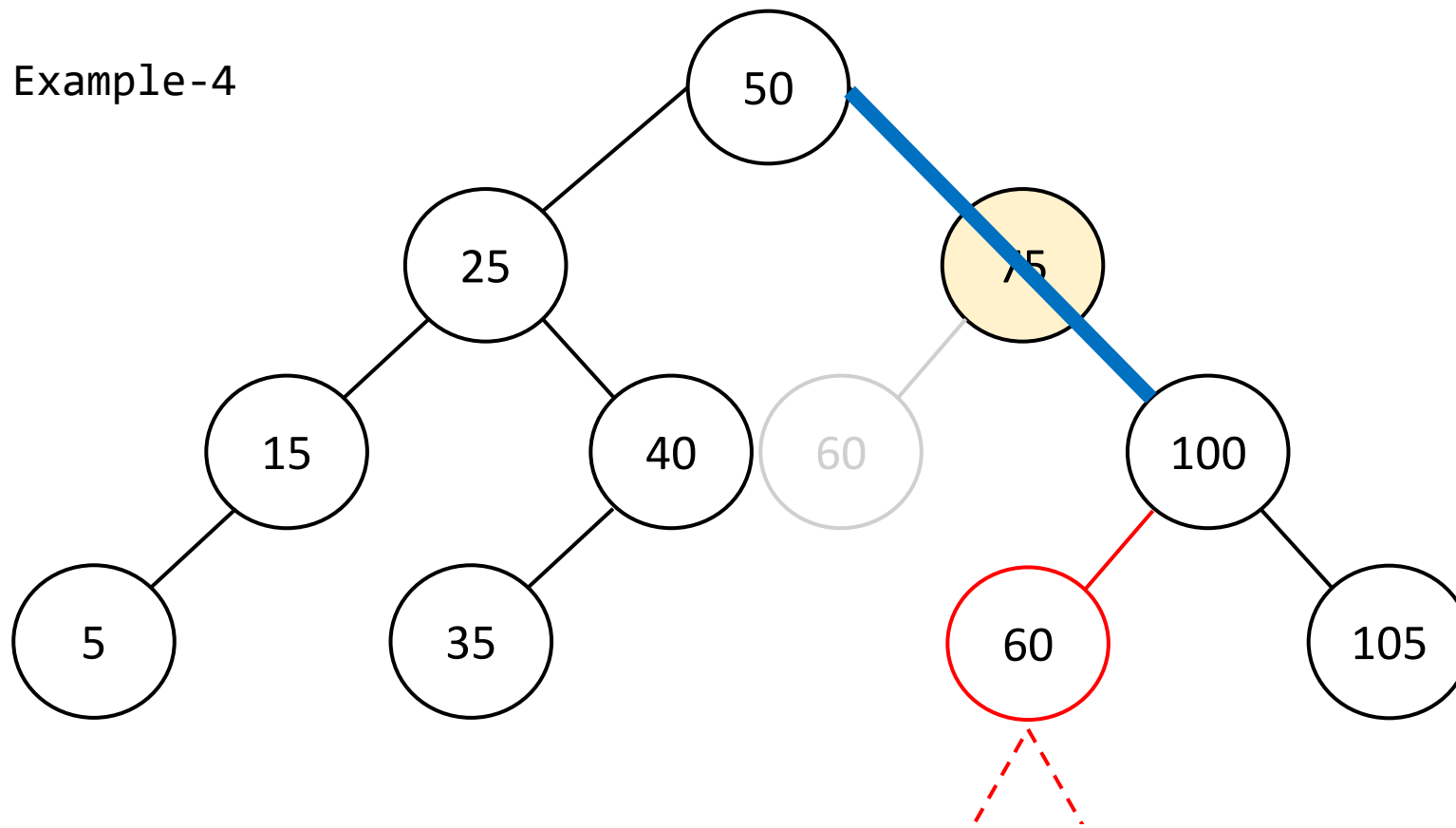
*Can't guarantee that 100's (z.right's) left subtree is always free so that z.left can go there!*

#### Delete Node with 2 child

Delete(75)

- z=75 has 2 subtrees
- z.left and z.right both exist
- z.p can be connected with z.right
- But what about z.left subtree?
- Seems it can be connected with 100.left- but is it always possible?

#### Example-4



*Can't guarantee that 100's (z.right's) left subtree is always free so that z.left can go there!*

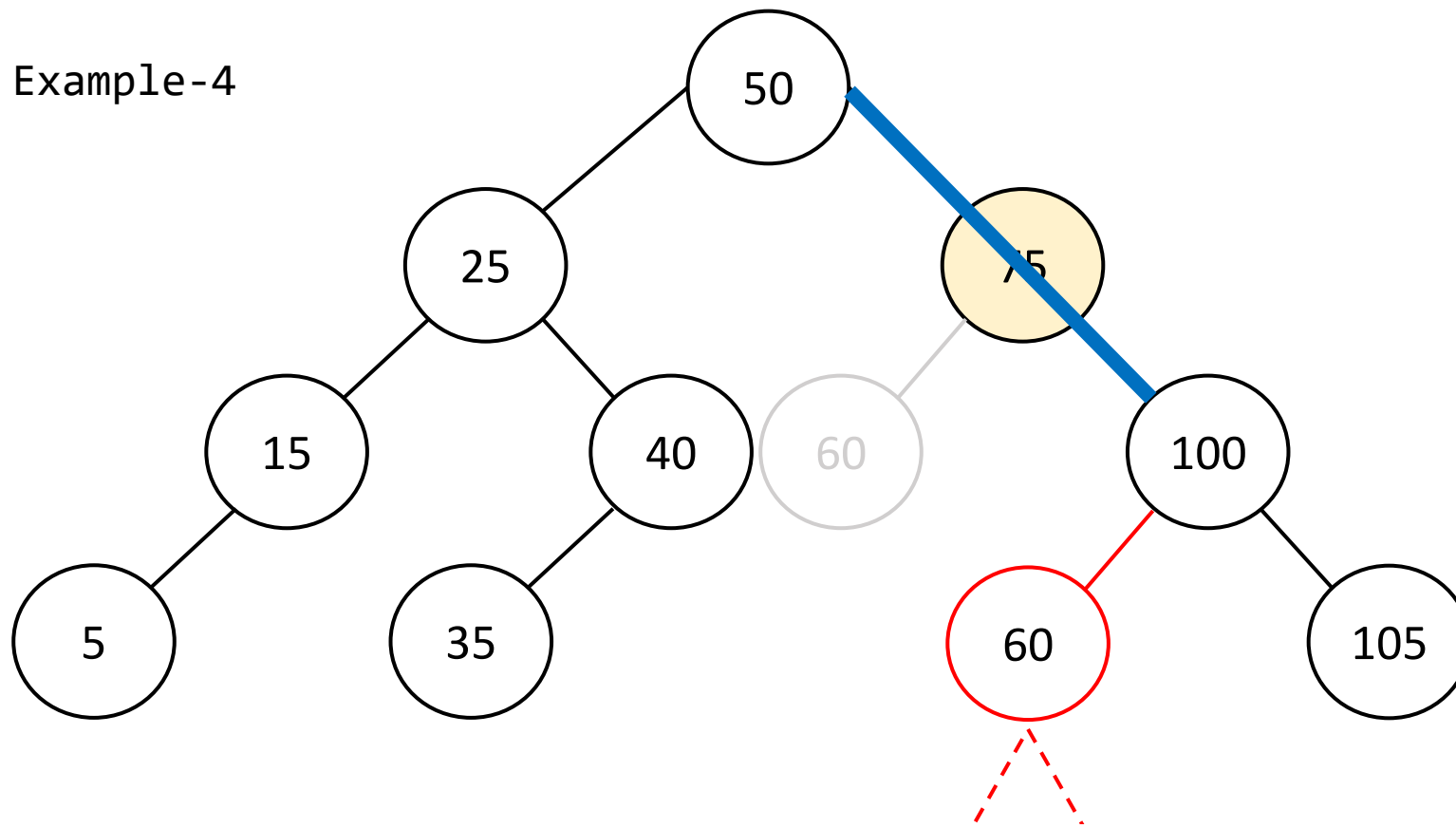
*Which node provides such a guarantee?*

#### Delete Node with 2 child

Delete(75)

- z=75 has 2 subtrees
- z.left and z.right both exist
- z.p can be connected with z.right
- But what about z.left subtree?
- **Seems it can be connected with 100.left- but is it always possible?**

#### Example-4



*Can't guarantee that 100's (z.right's) left subtree is always free so that z.left can go there!*

*Which node provides such a guarantee?*

Successor(z)

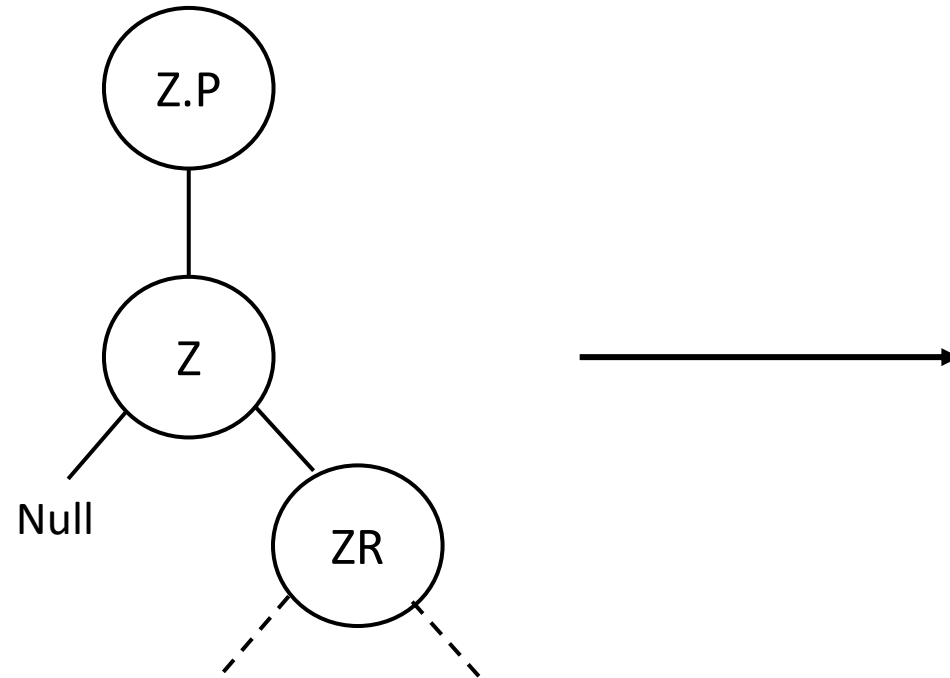
Delete Node with 2 child

Delete(75)

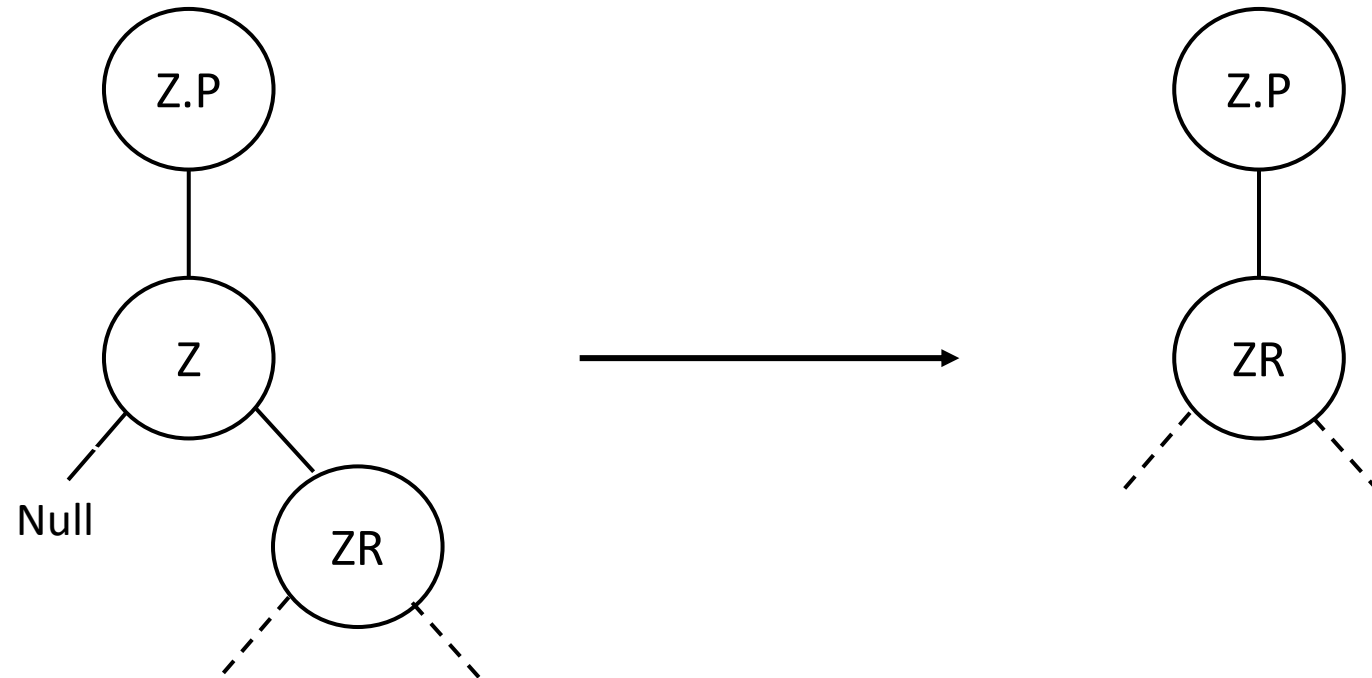
- z=75 has 2 subtrees
- z.left and z.right both exist
- z.p can be connected with z.right
- But what about z.left subtree?
- Seems it can be connected with 100.left- but is it always possible?

*Let's summarize the different scenarios...*

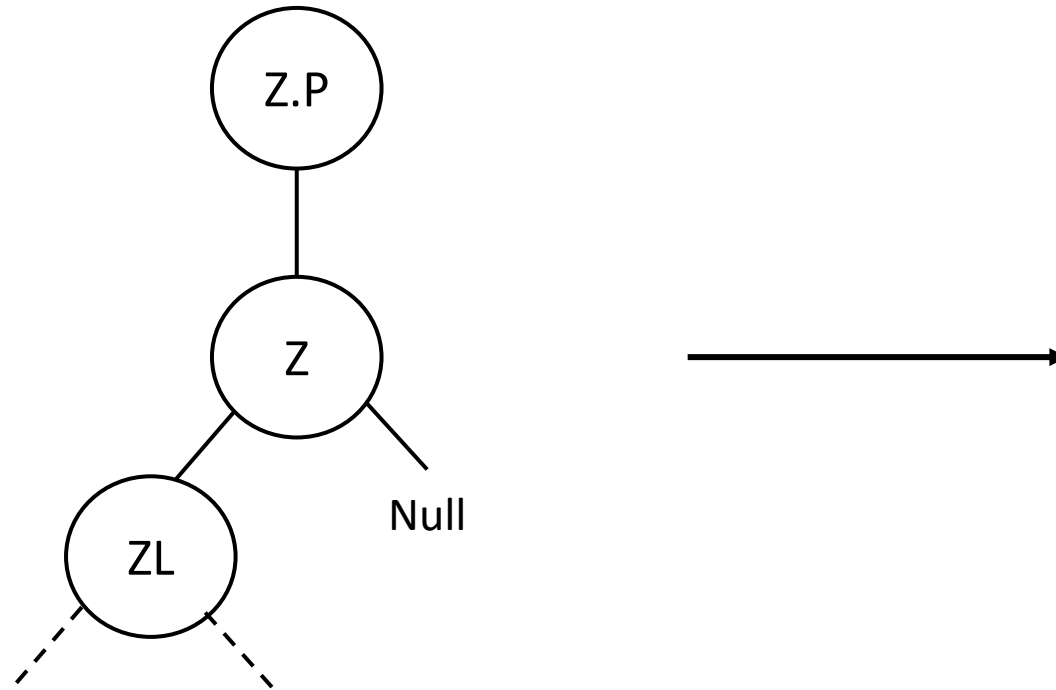
Case-A



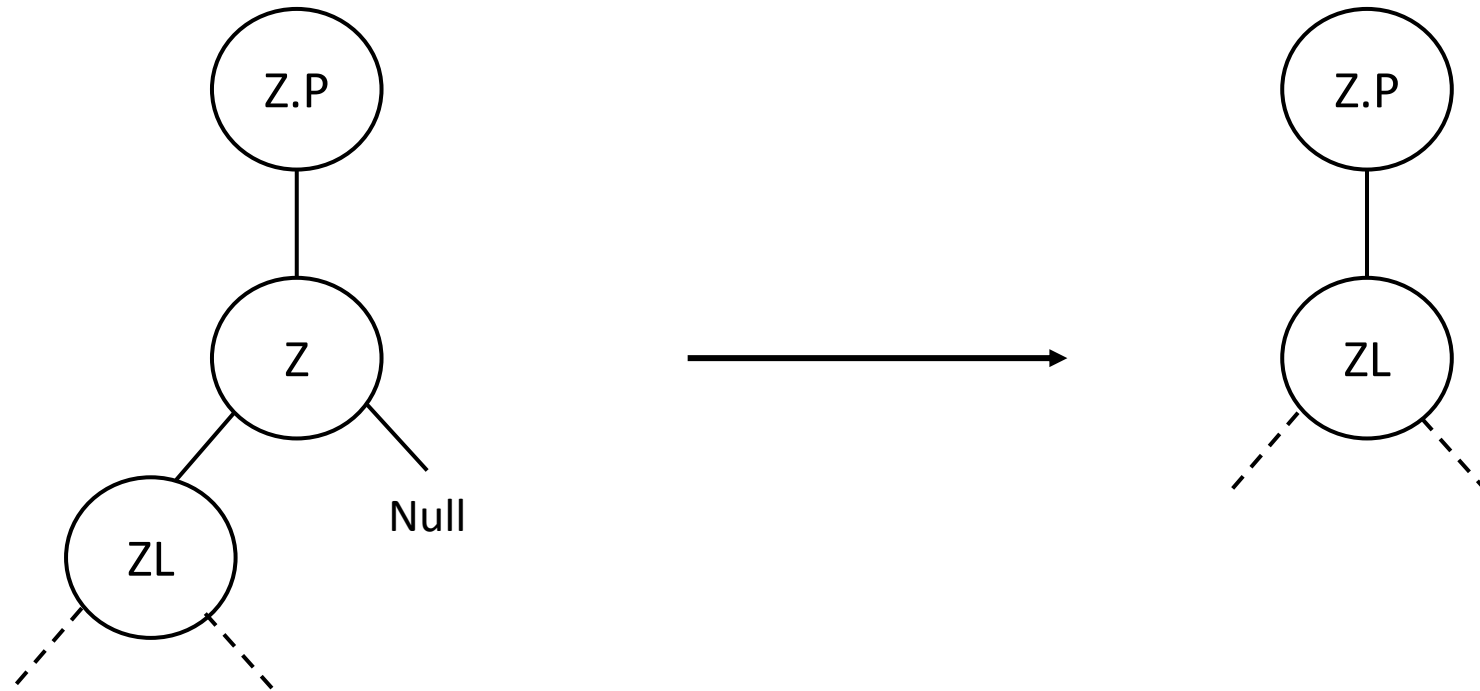
Case-A



Case-B



Case-B



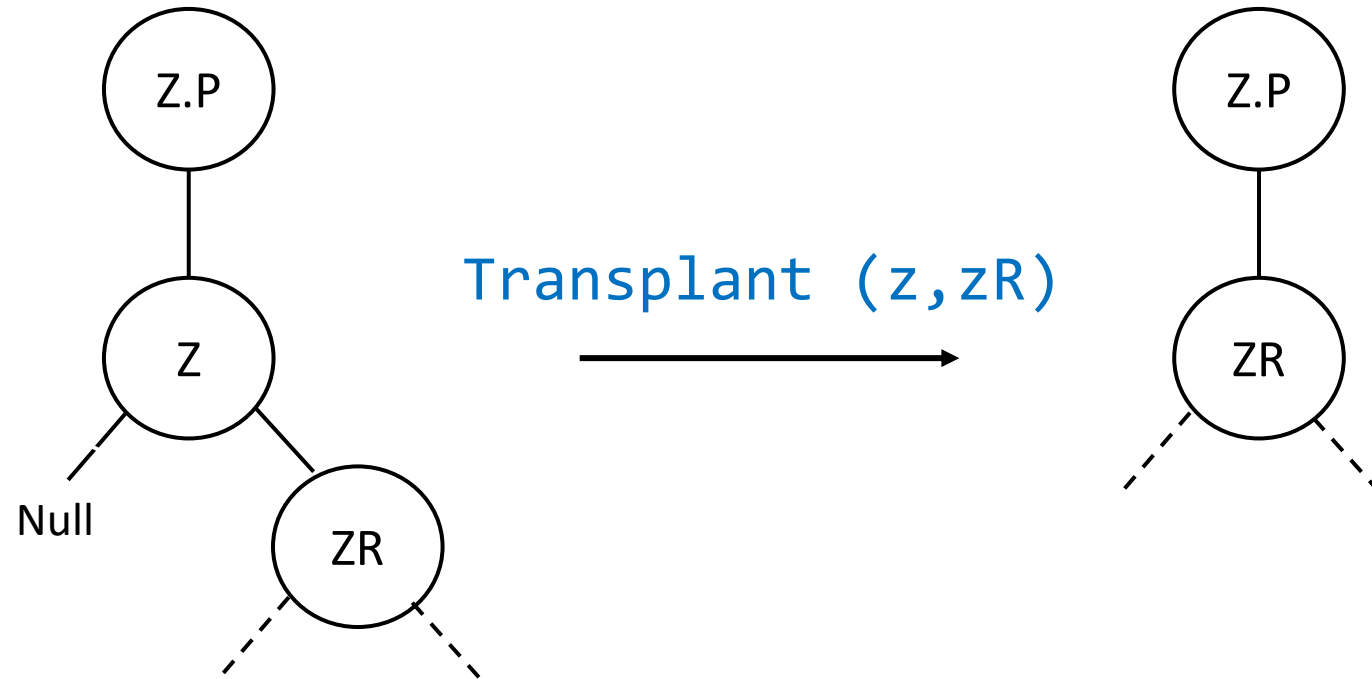
```
transplant (u,v){  
    // connects u.p with v as appropriate child
```

```
transplant (u,v){  
    // connects u.p with v as appropriate child  
  
    if (u.p == Null):  
        root = v  
  
}
```

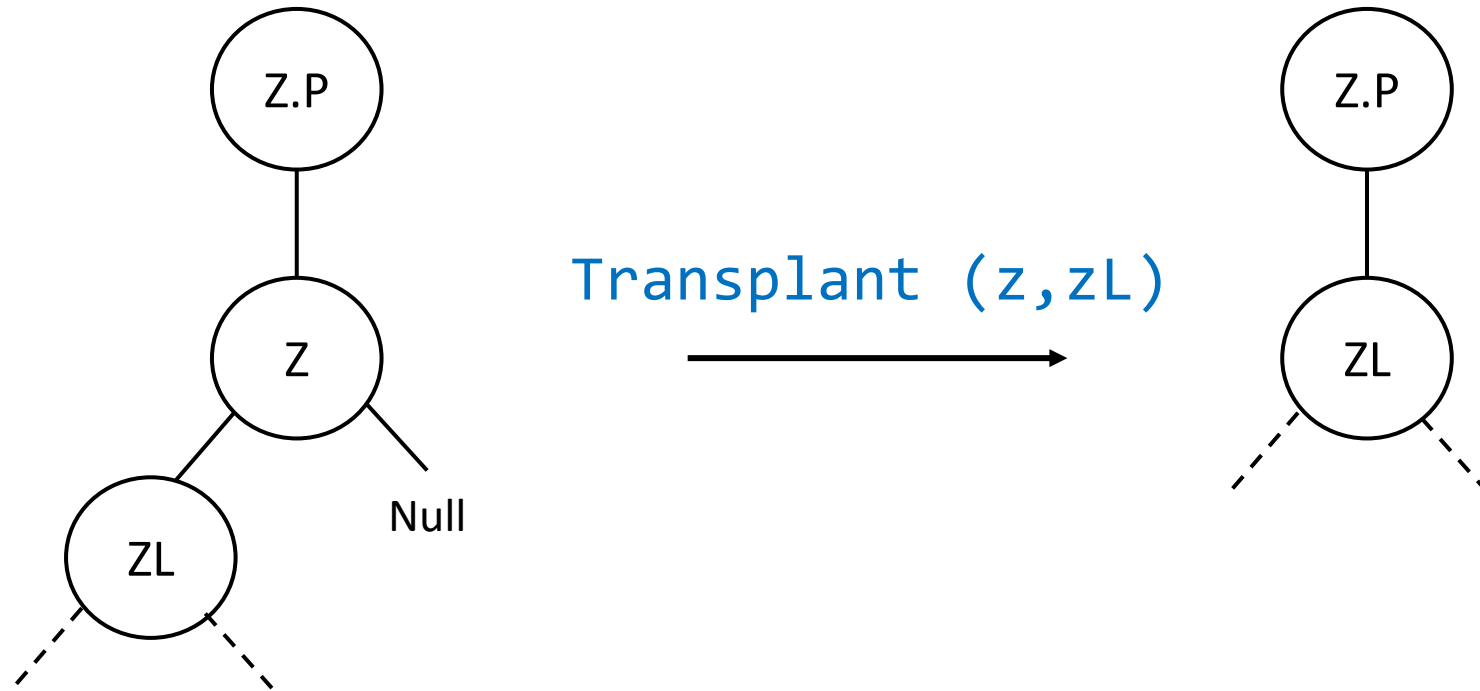
```
transplant (u,v){  
    // connects u.p with v as appropriate child  
  
    if (u.p == Null):  
        root = v  
    else if (u == u.p.left):  
        u.p.left = v  
    else  
        u.p.right = v
```

```
transplant (u,v){  
    // connects u.p with v as appropriate child  
  
    if (u.p == Null):  
        root = v  
    else if (u == u.p.left):  
        u.p.left = v  
    else  
        u.p.right = v  
    if (v!=Null):  
        v.p = u.p  
  
}
```

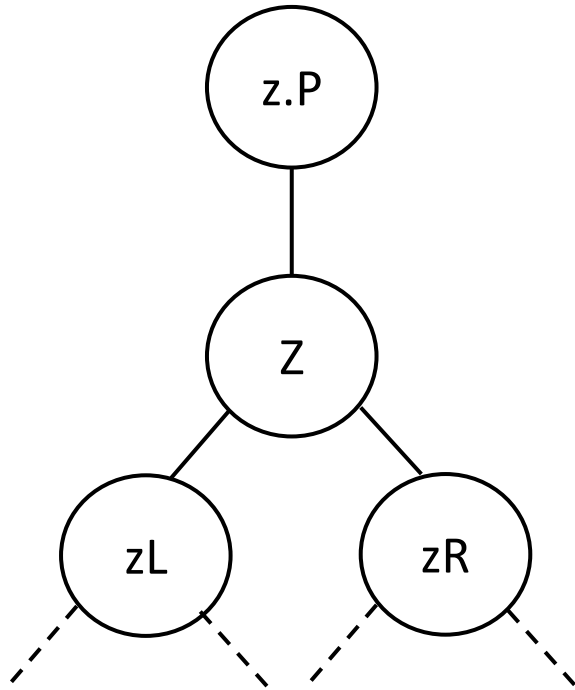
Case-A



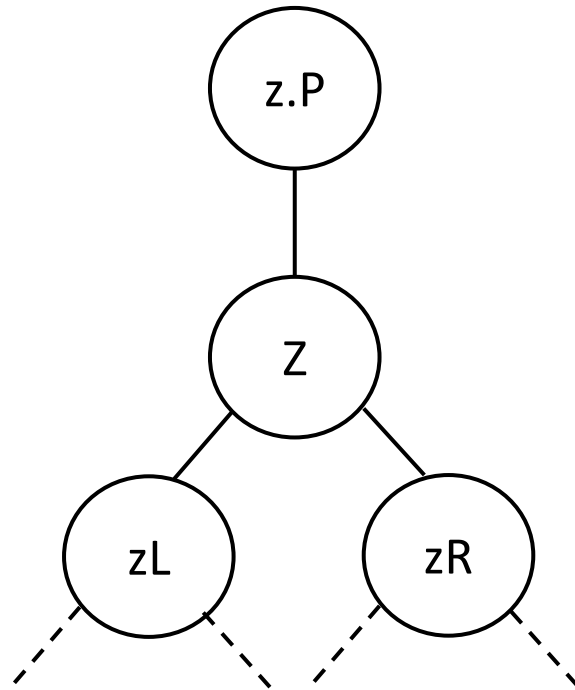
Case-B



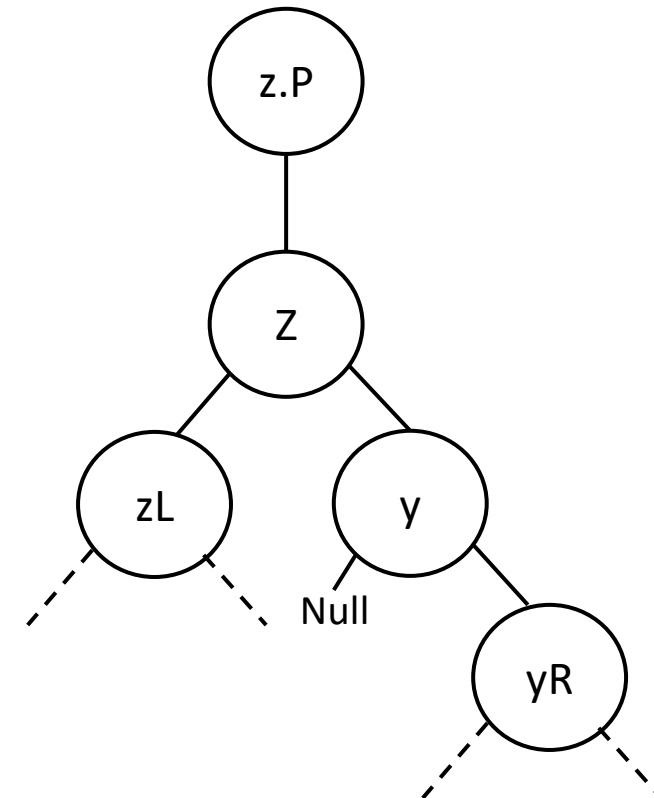
*Z has two subtrees.  
So lets find  $y = \text{successor}(z)$*



*Z has two subtrees.  
So lets find  $y = \text{successor}(z)$*

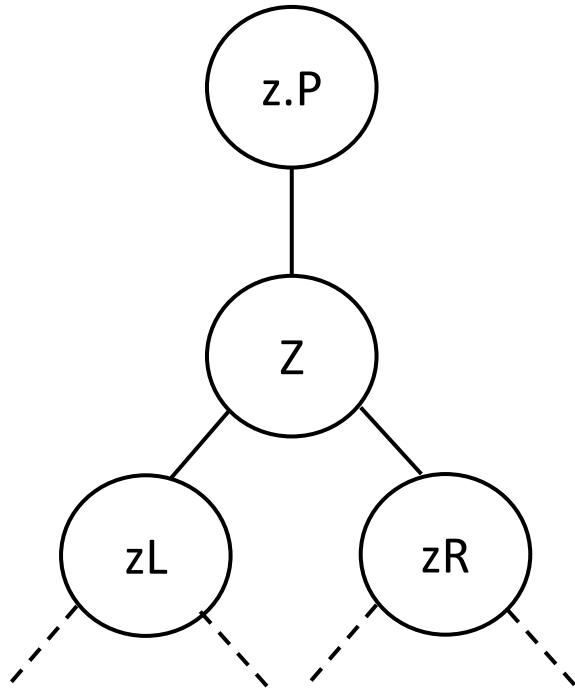


*Y can be the  
immediate right  
child of z*



Case-C

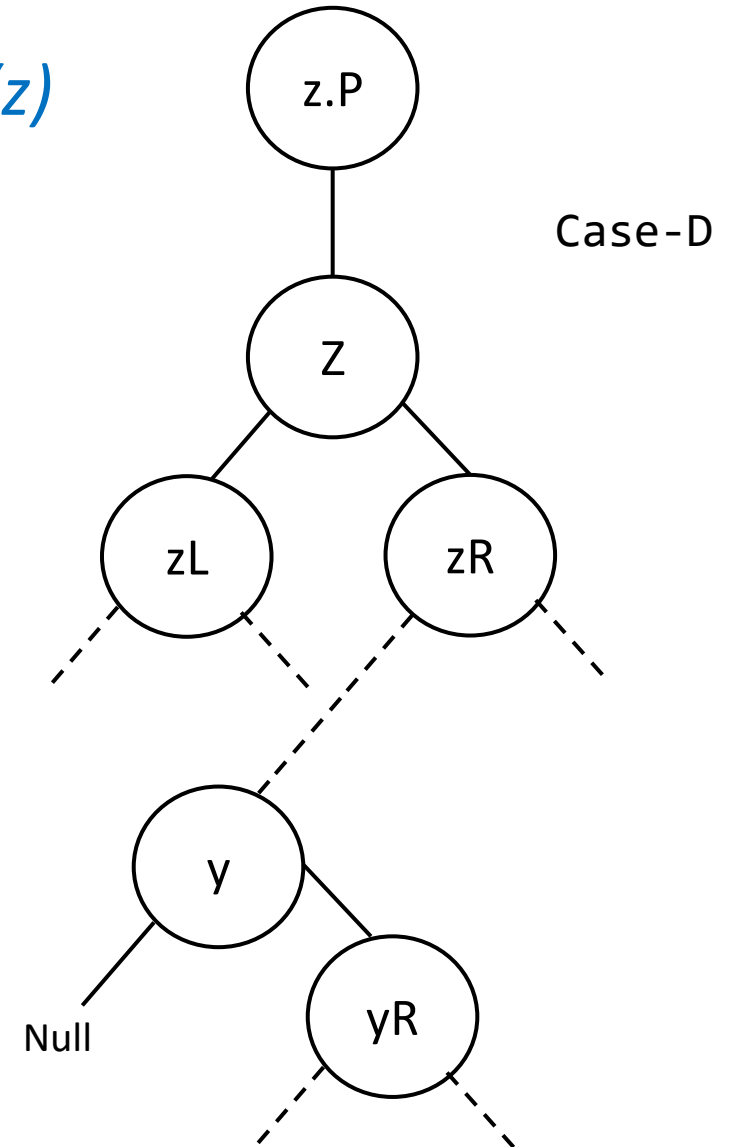
*Z has two subtrees.  
So lets find  $y = \text{successor}(z)$*



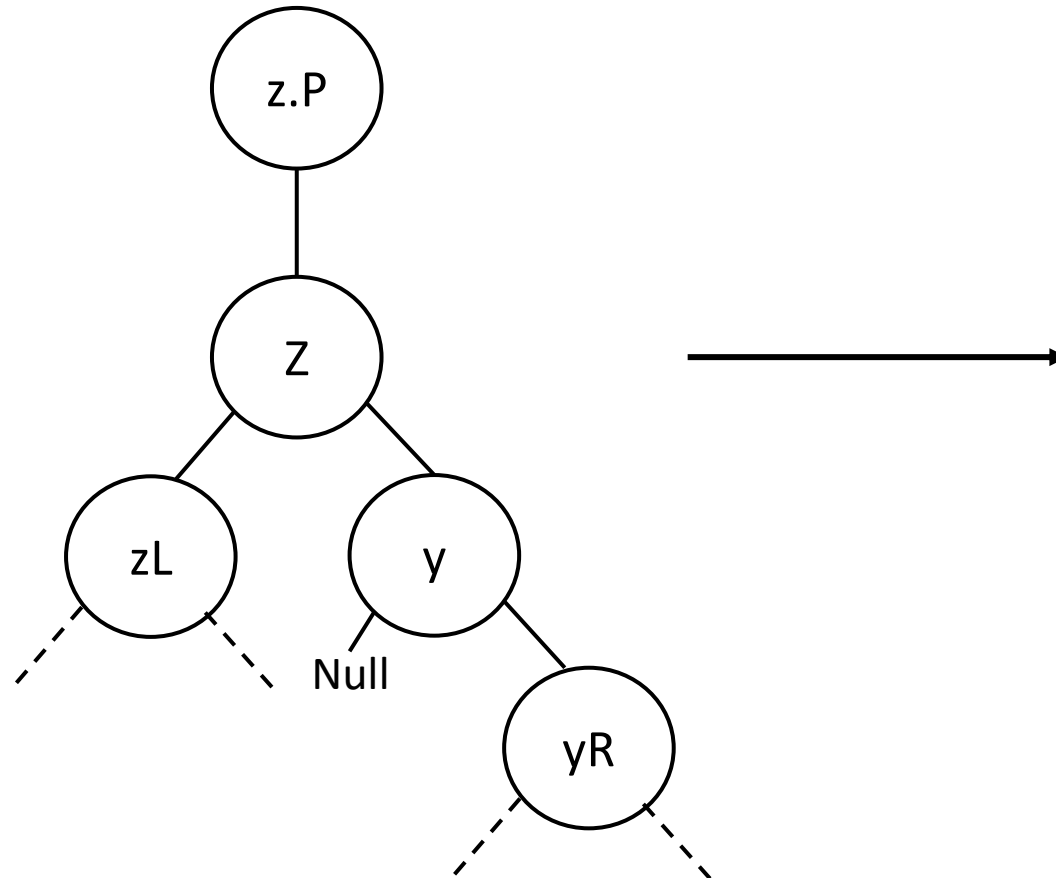
*Y can be the  
immediate right  
child of z*



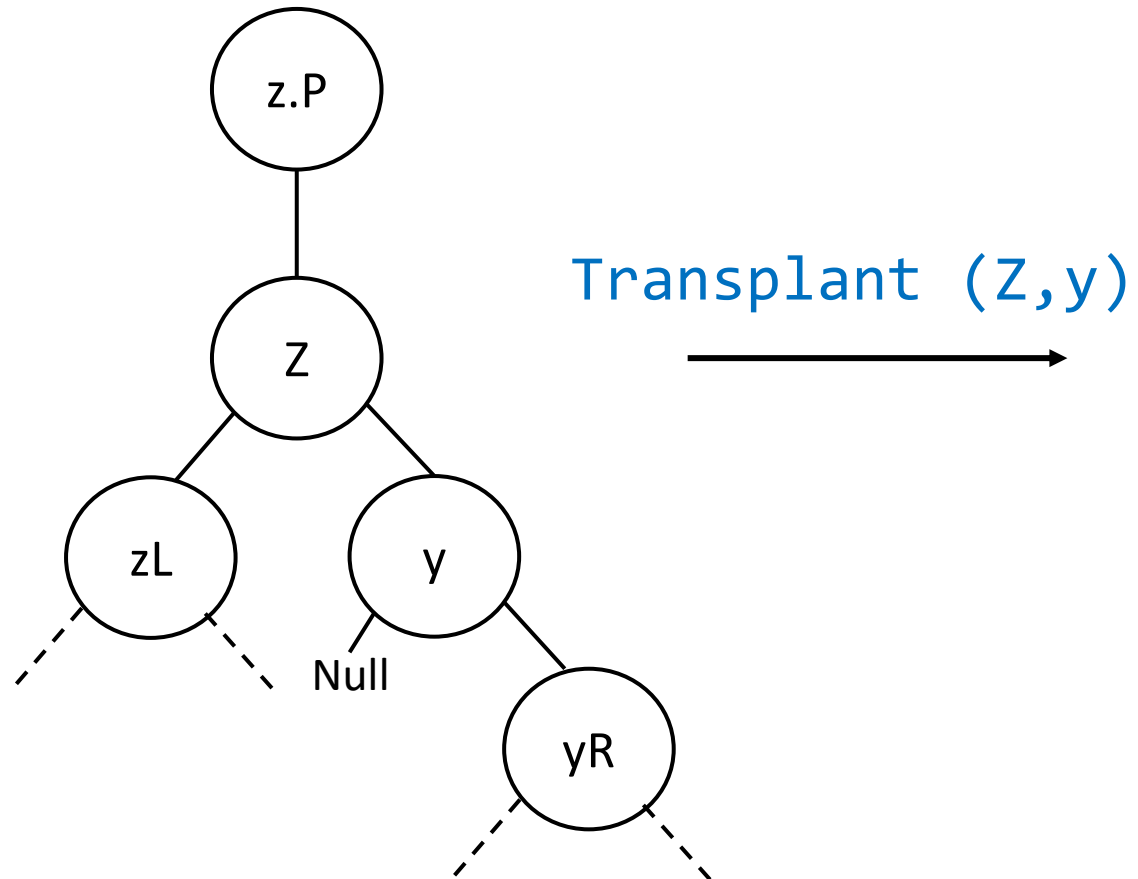
*Otherwise, y will be  
 $\text{minimum}(z.\text{right})$*



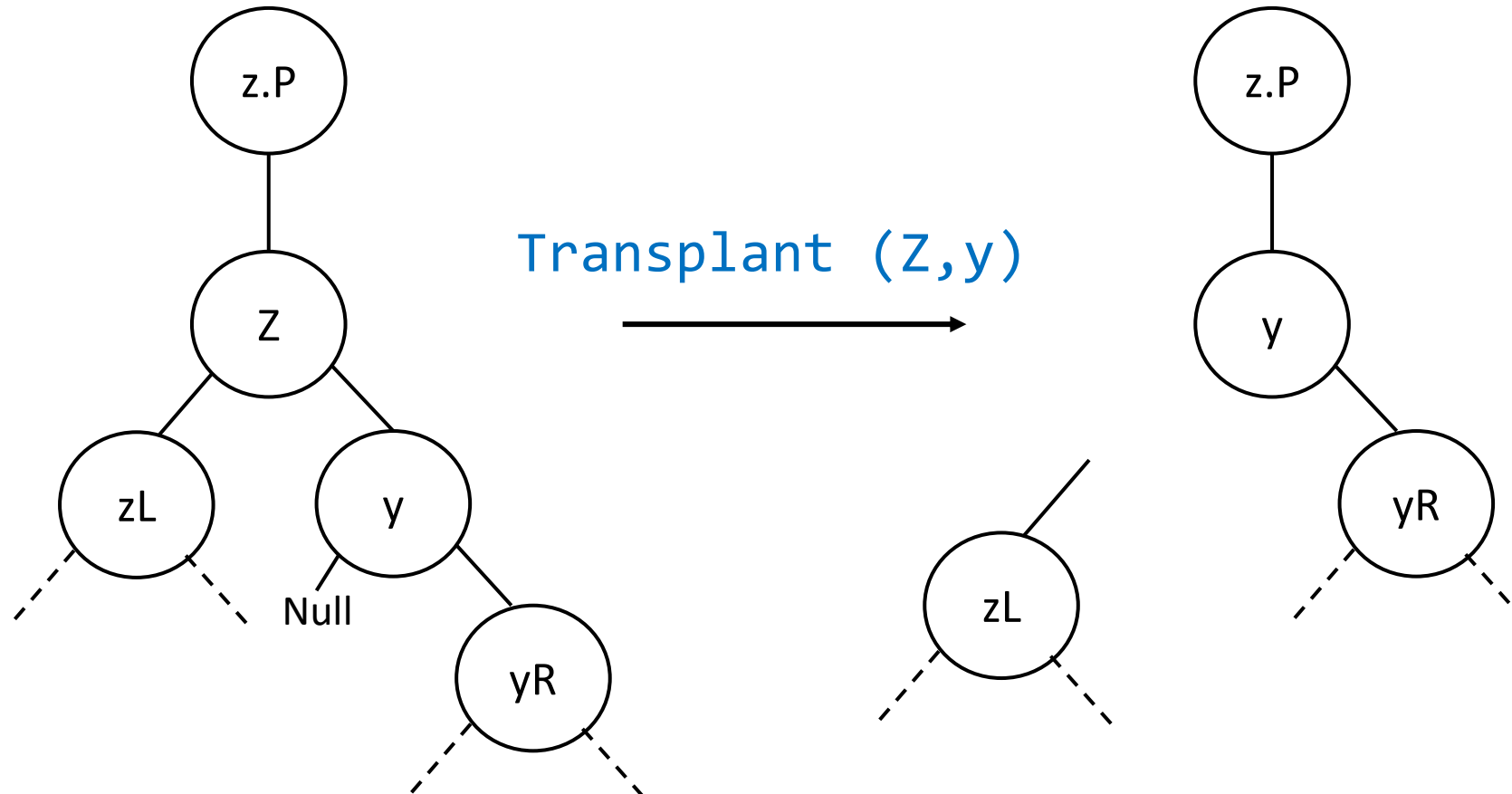
Case-C



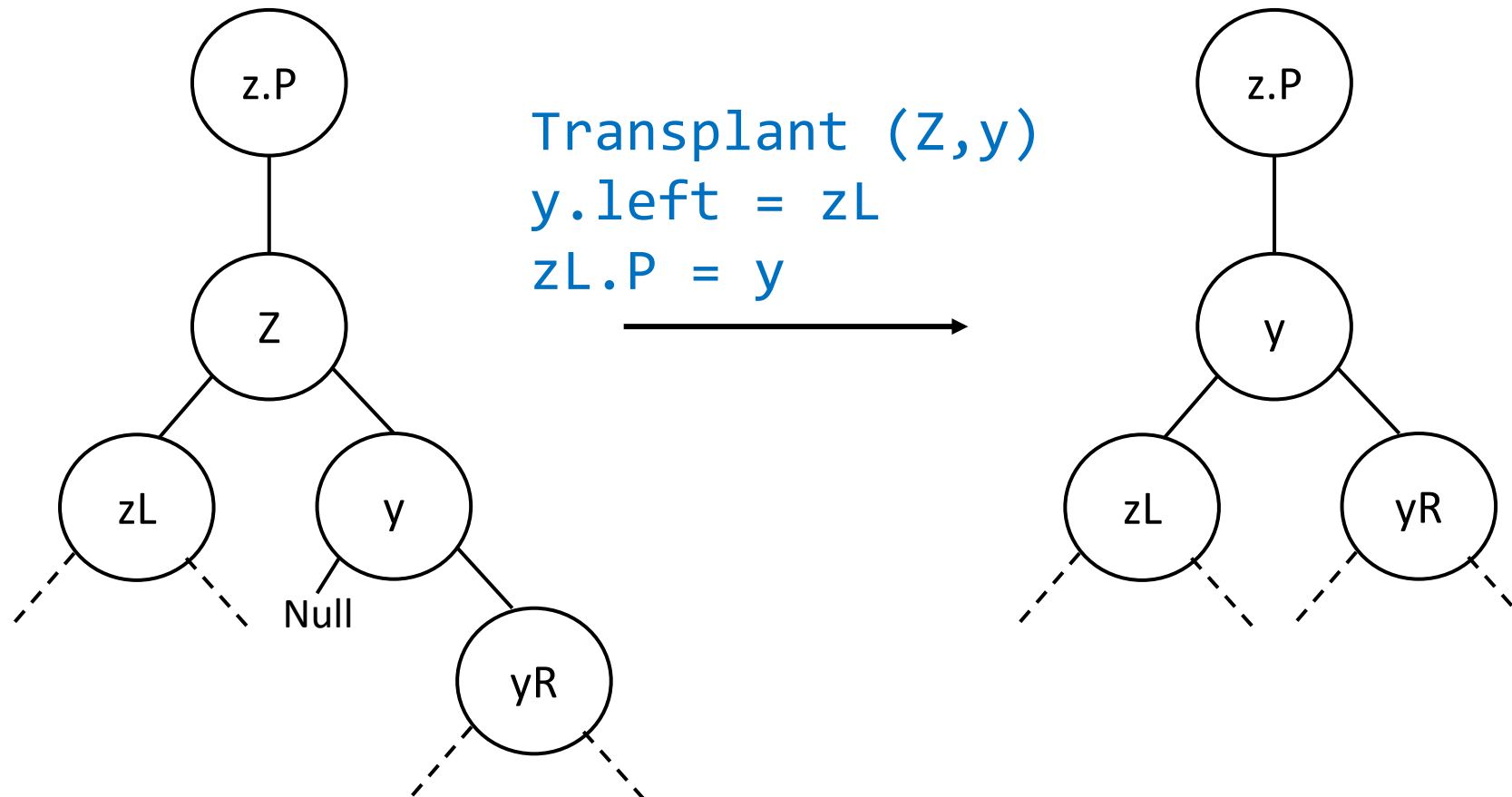
Case-C



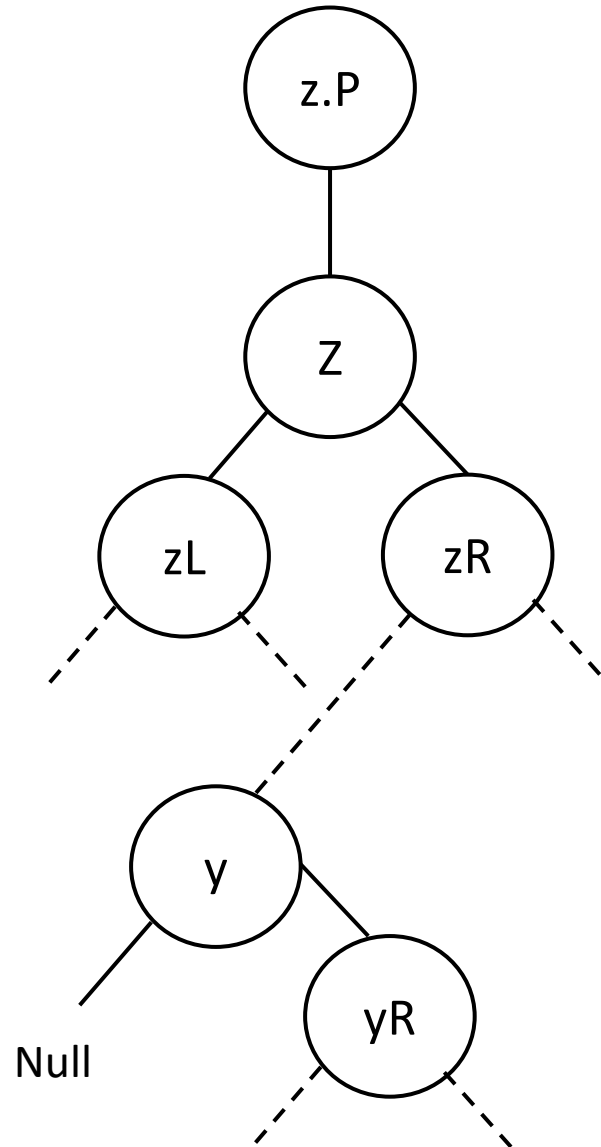
Case-C



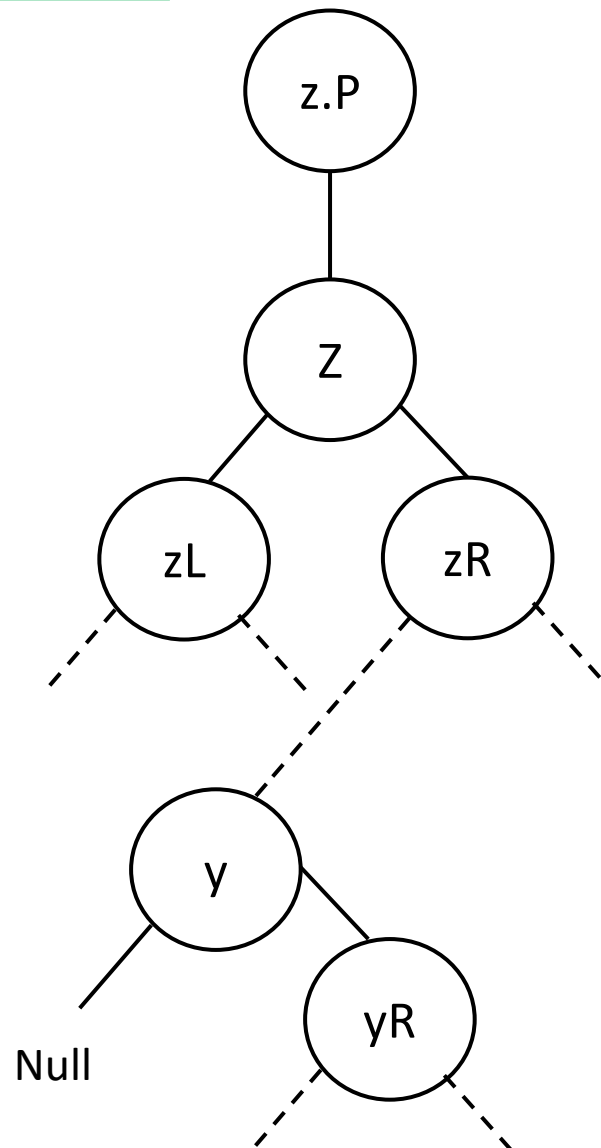
Case-C



Case-D

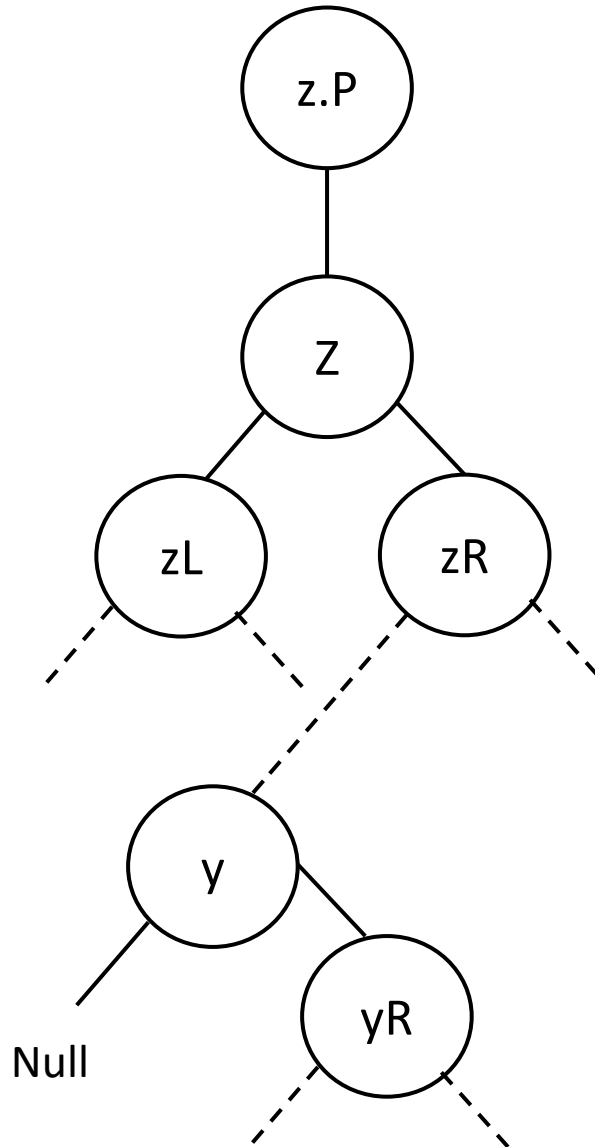


Case-D



*Plan:  
y should come above, replace Z, catch zL & zR*

## Case-D



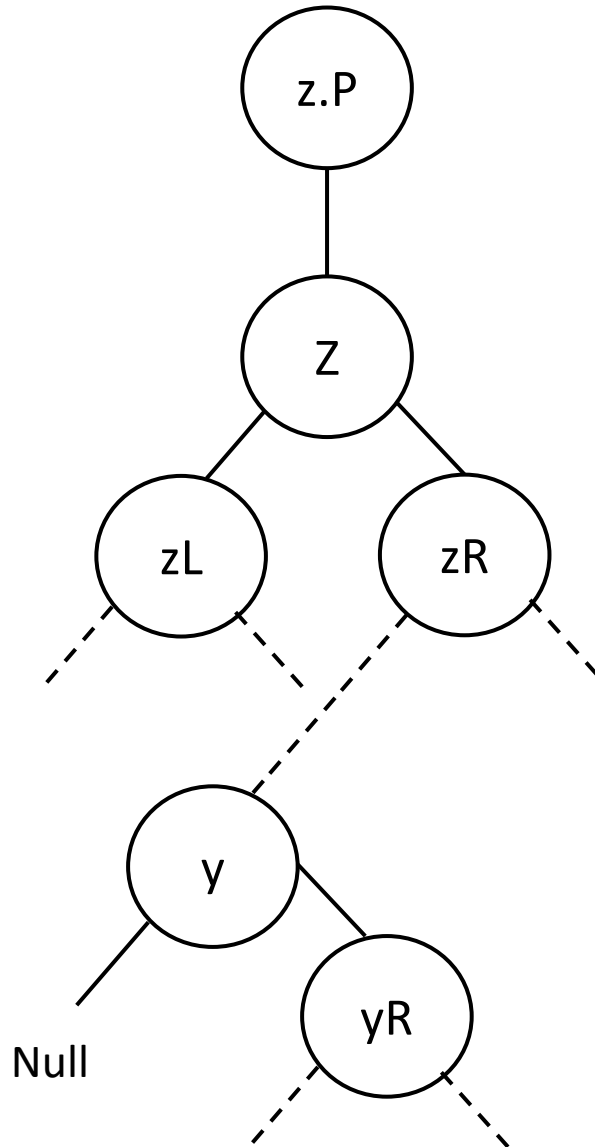
*Plan:*

*y should come above, replace Z, catch zL & zR*

*To enable y to catch zL, zR ---*

*First y should be free!!!*

## Case-D



*Plan:*

*y should come above, replace Z, catch zL & zR*

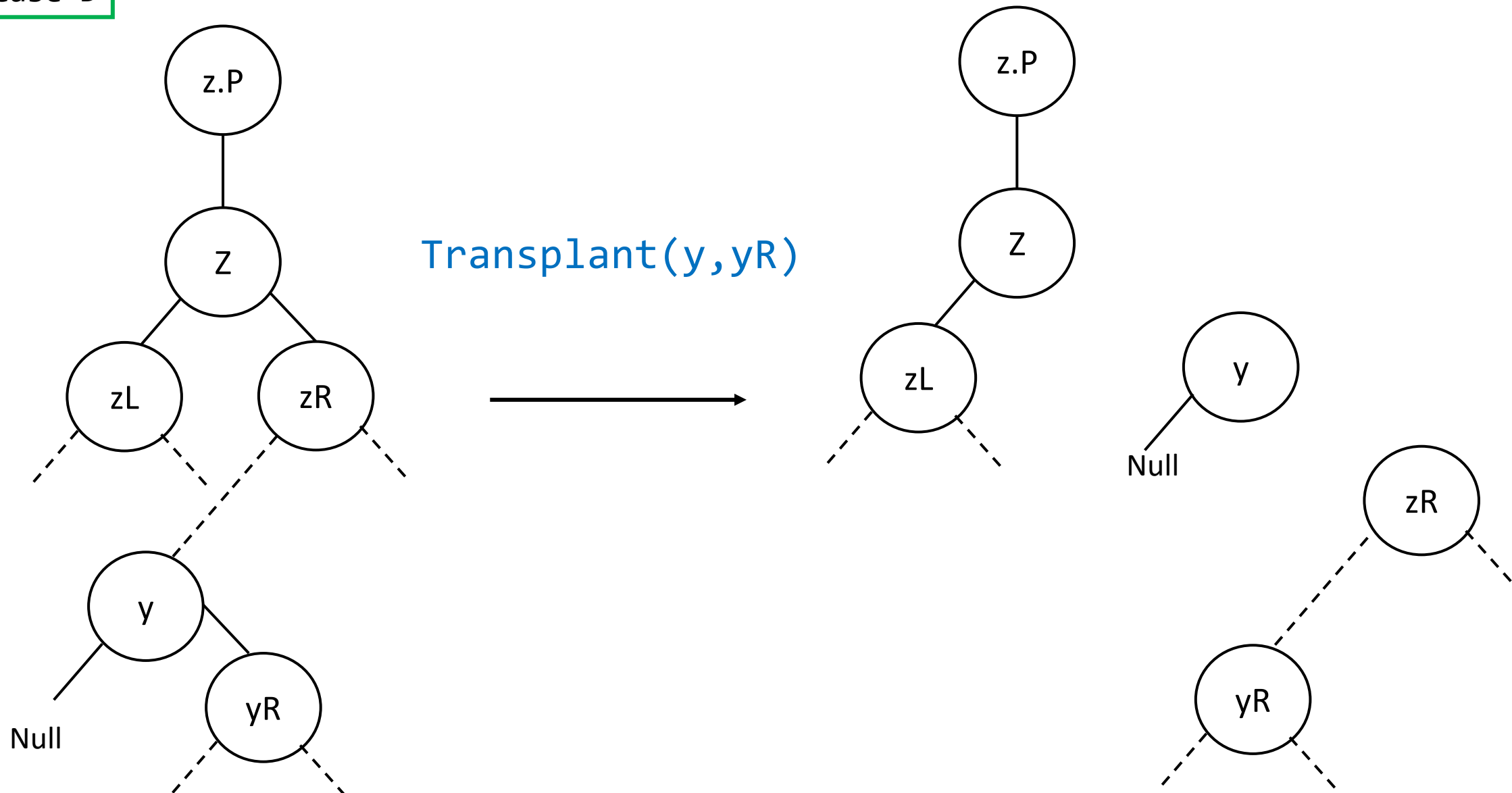
*To enable y to catch zL, zR ---*

*First y should be free!!!*

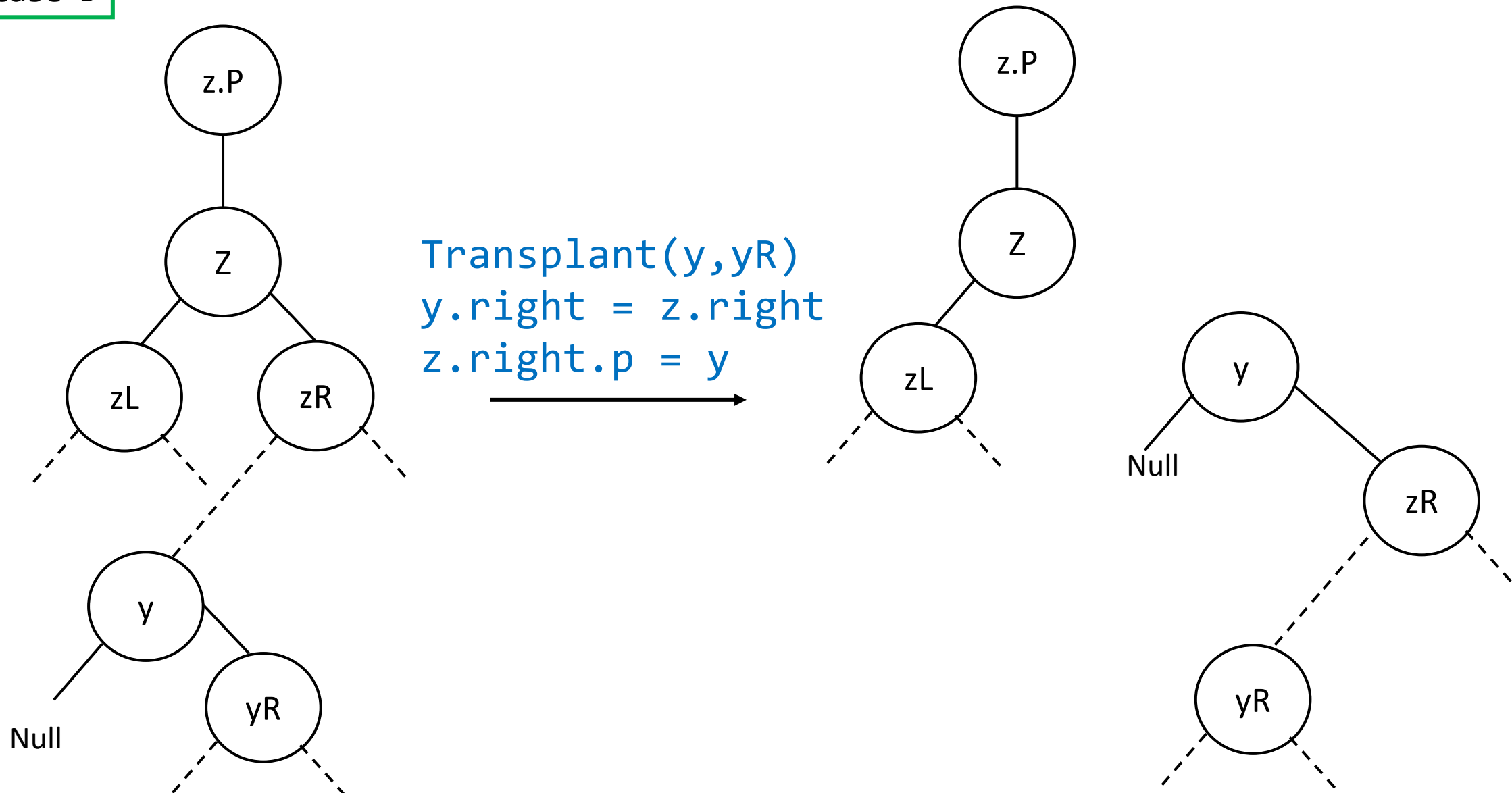
*yR becomes orphan*

*So, yR should be connected with y.p*

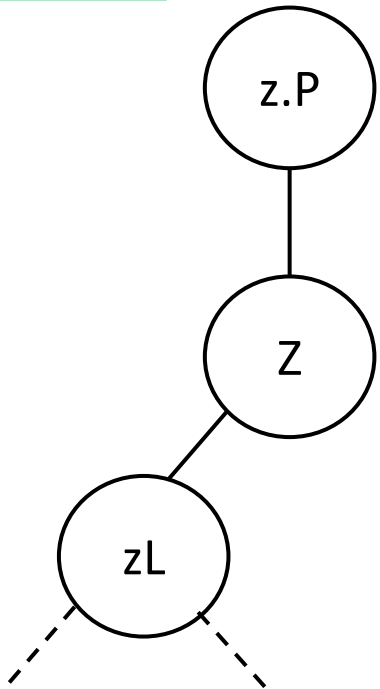
Case-D



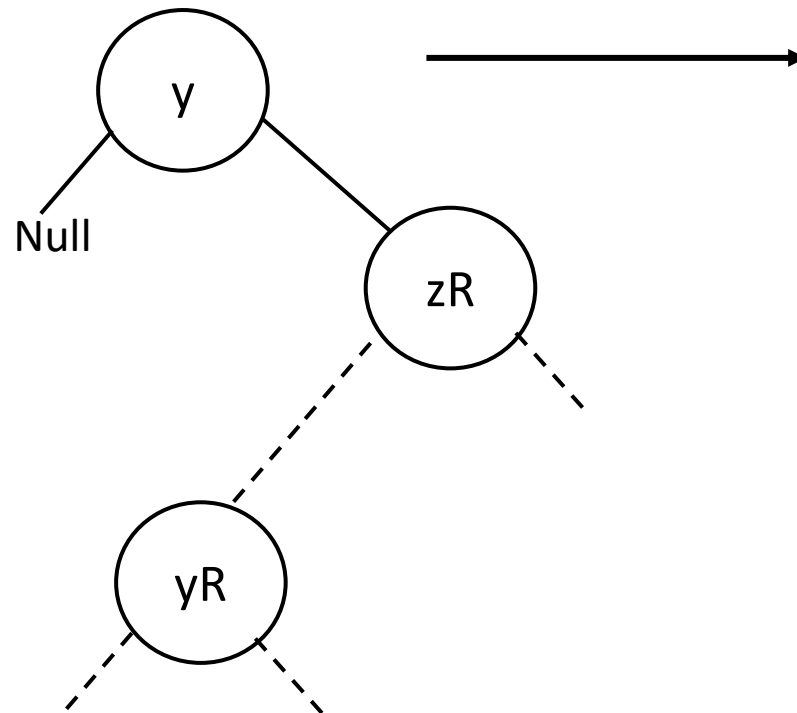
Case-D



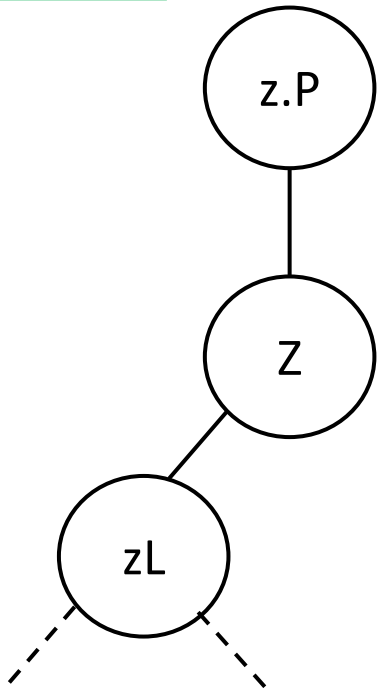
Case-D



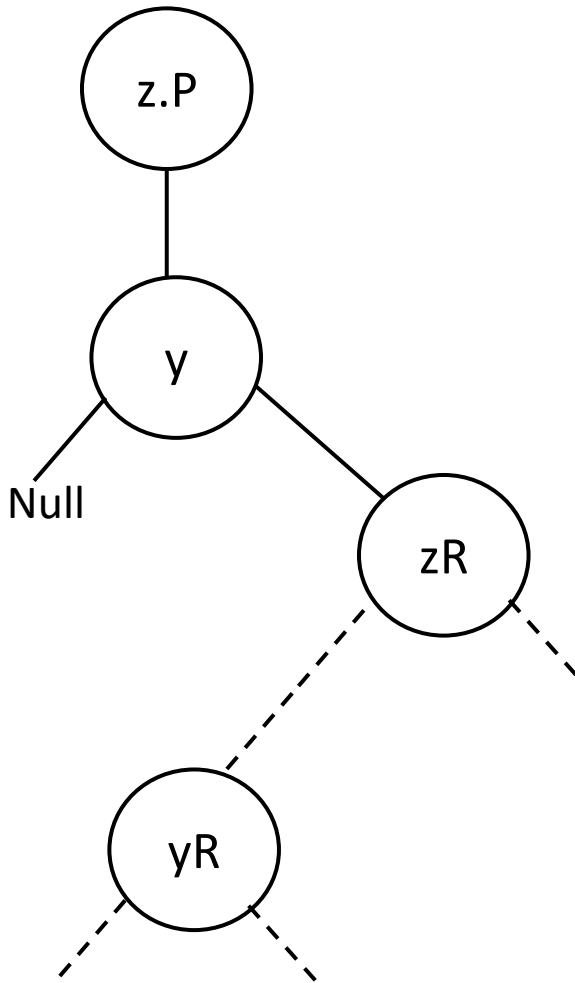
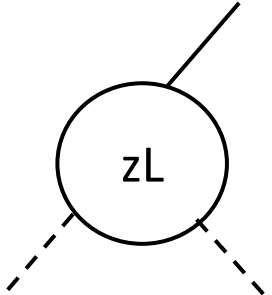
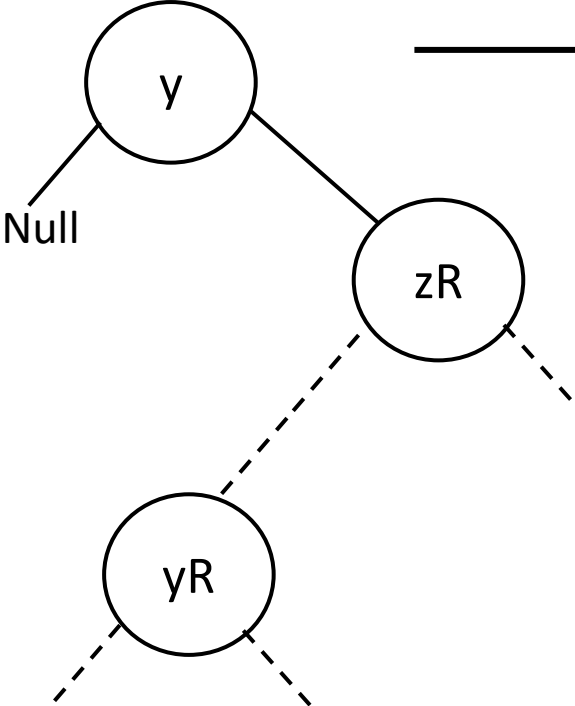
Transplant( $z, y$ )



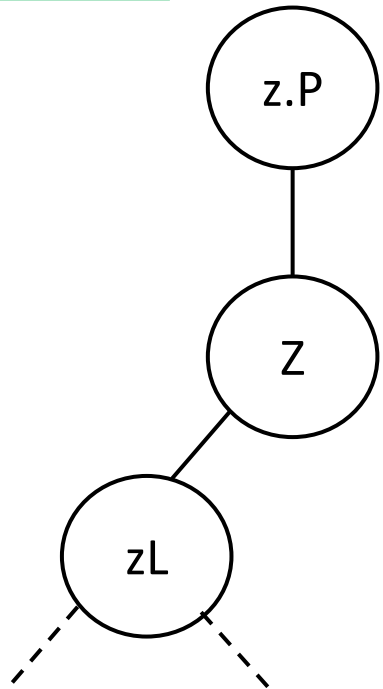
Case-D



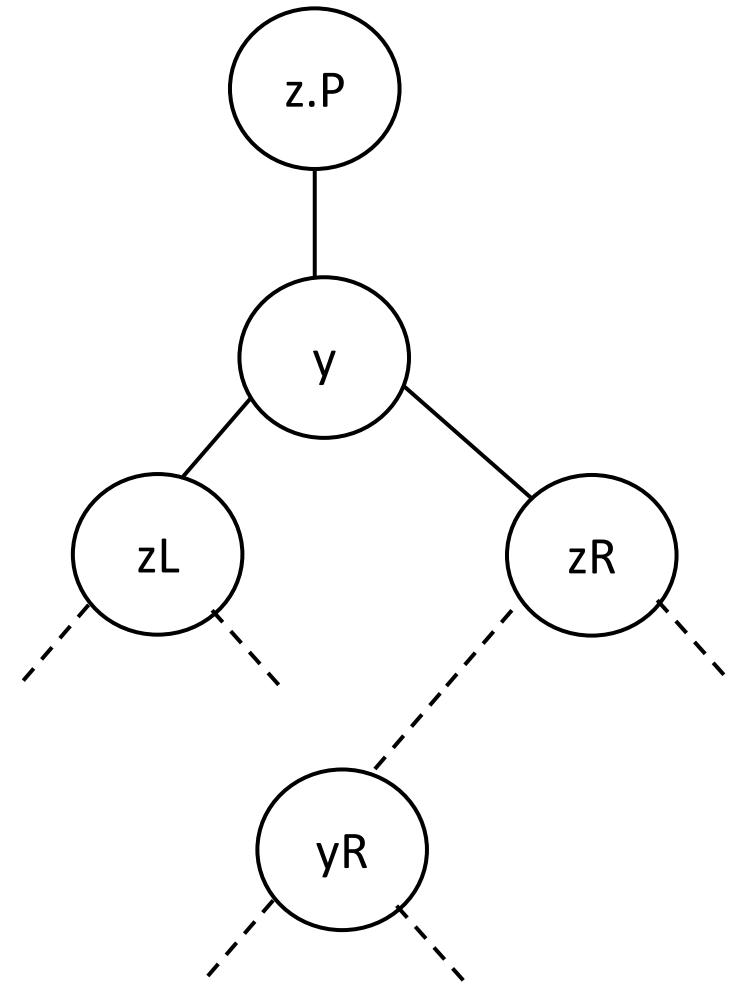
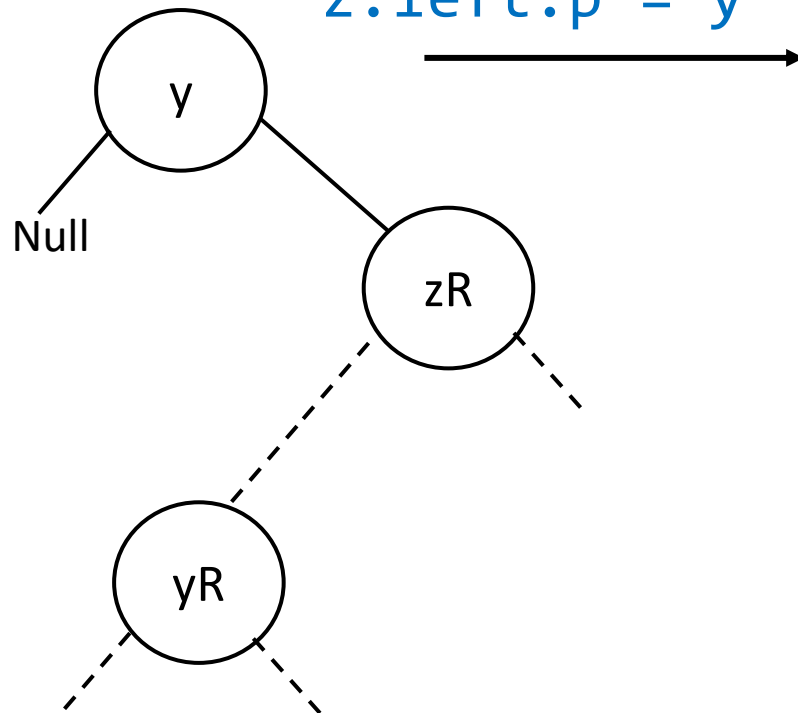
Transplant( $z, y$ )

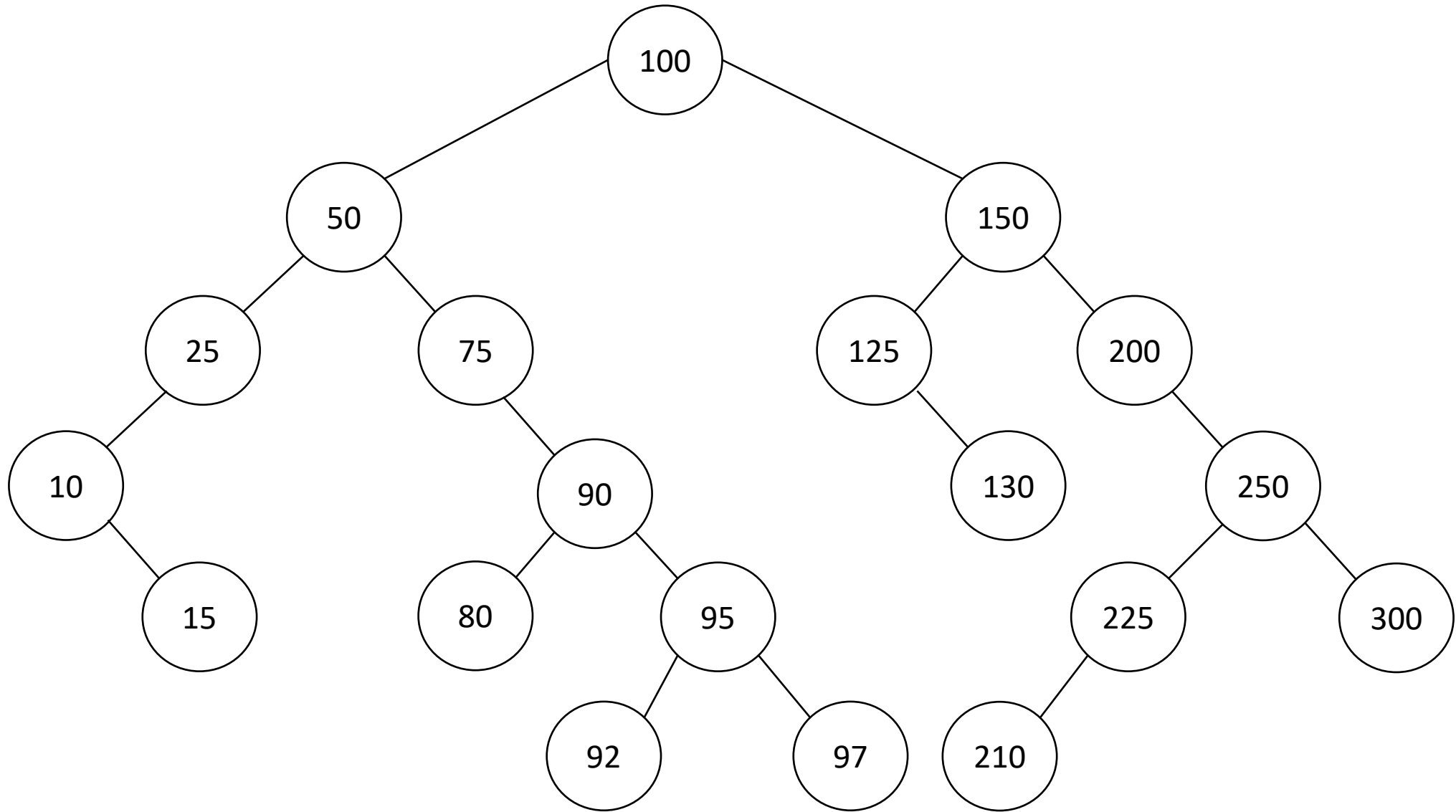


Case-D

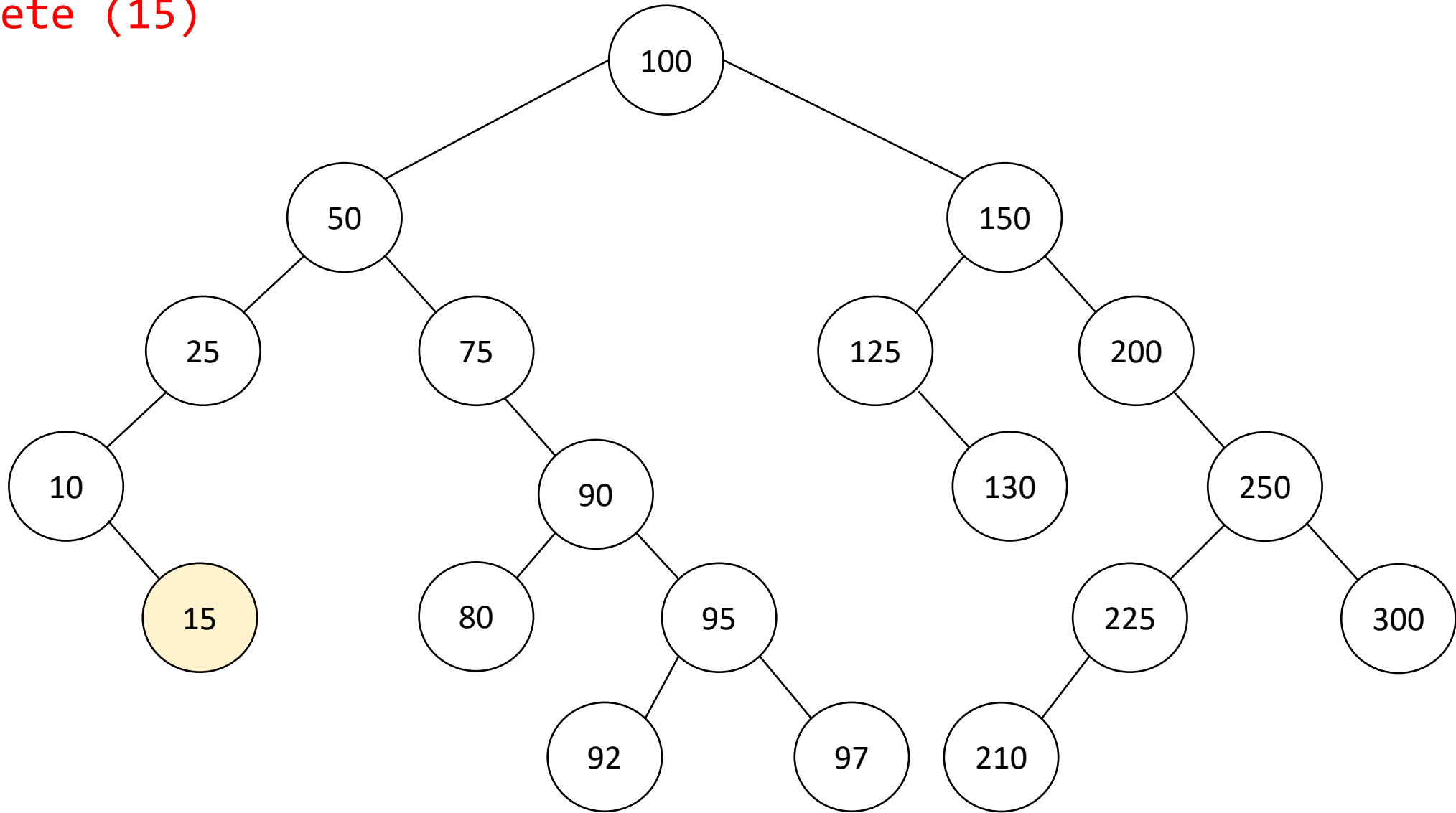


Transplant( $z, y$ )  
 $y.left = z.left$   
 $z.left.p = y$



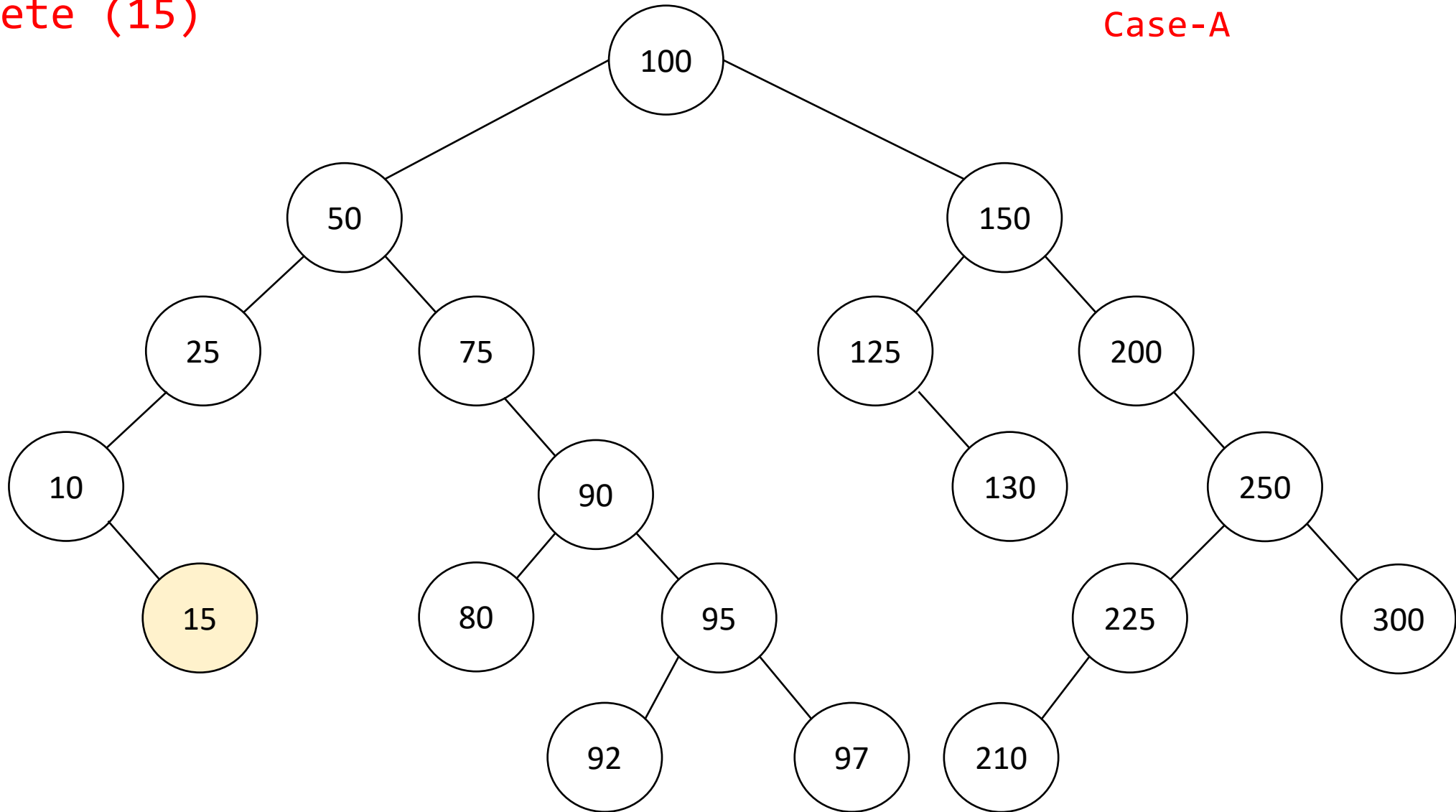


Delete (15)



Delete (15)

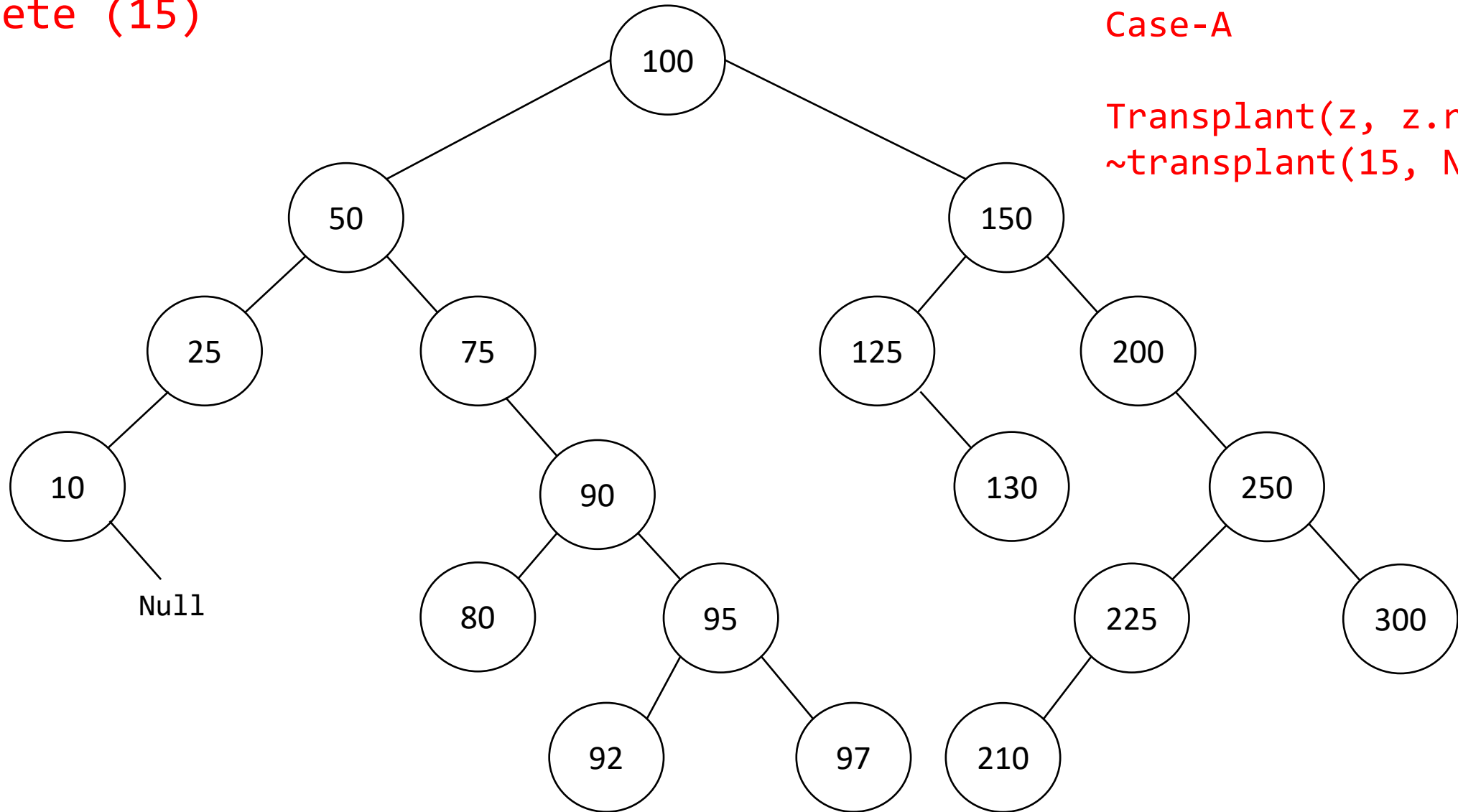
z=15, z.left=Null  
Case-A



Delete (15)

z=15, z.left=NULL  
Case-A

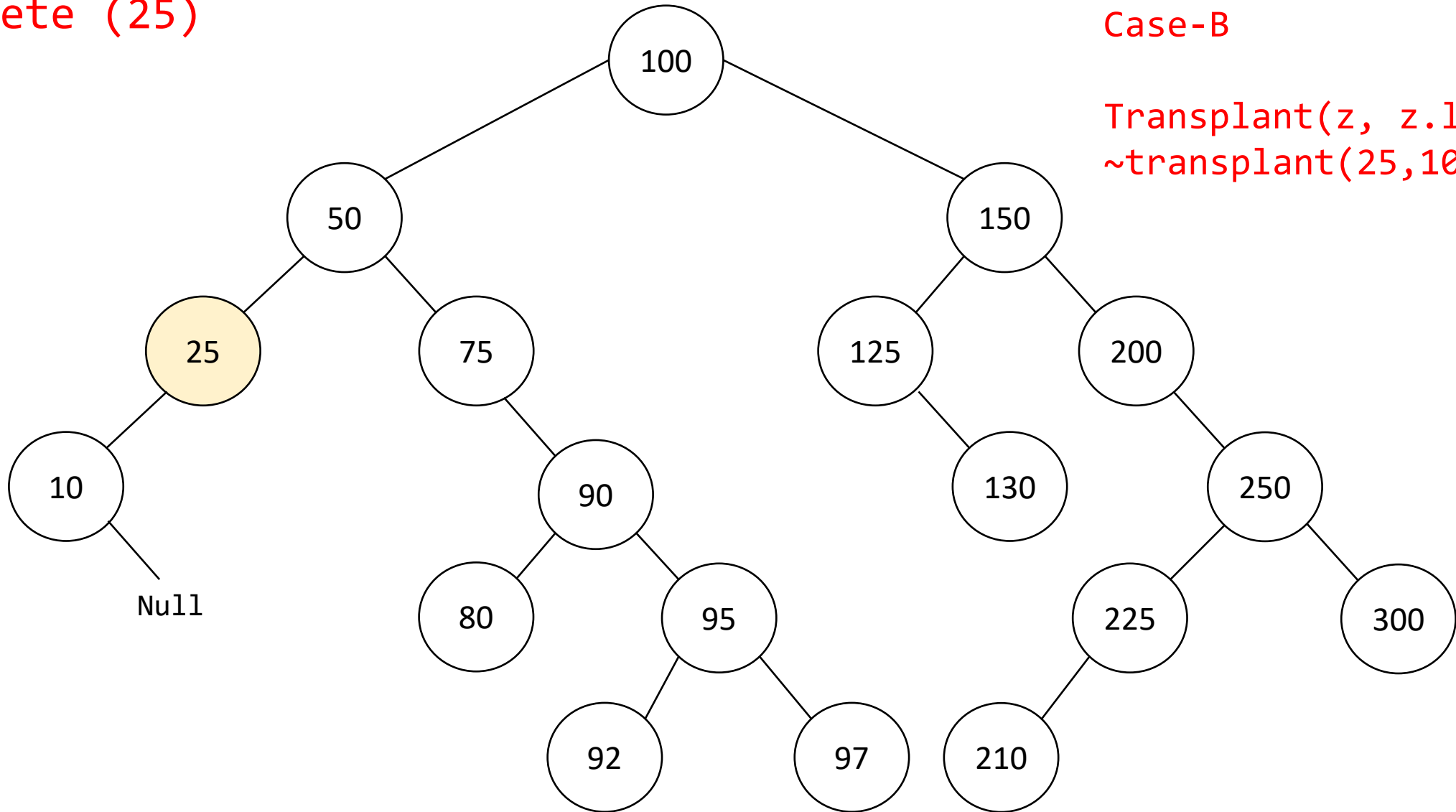
Transplant(z, z.right)  
~transplant(15, NULL)



Delete (25)

$z=25$ ,  $z.\text{right}=\text{Null}$   
Case-B

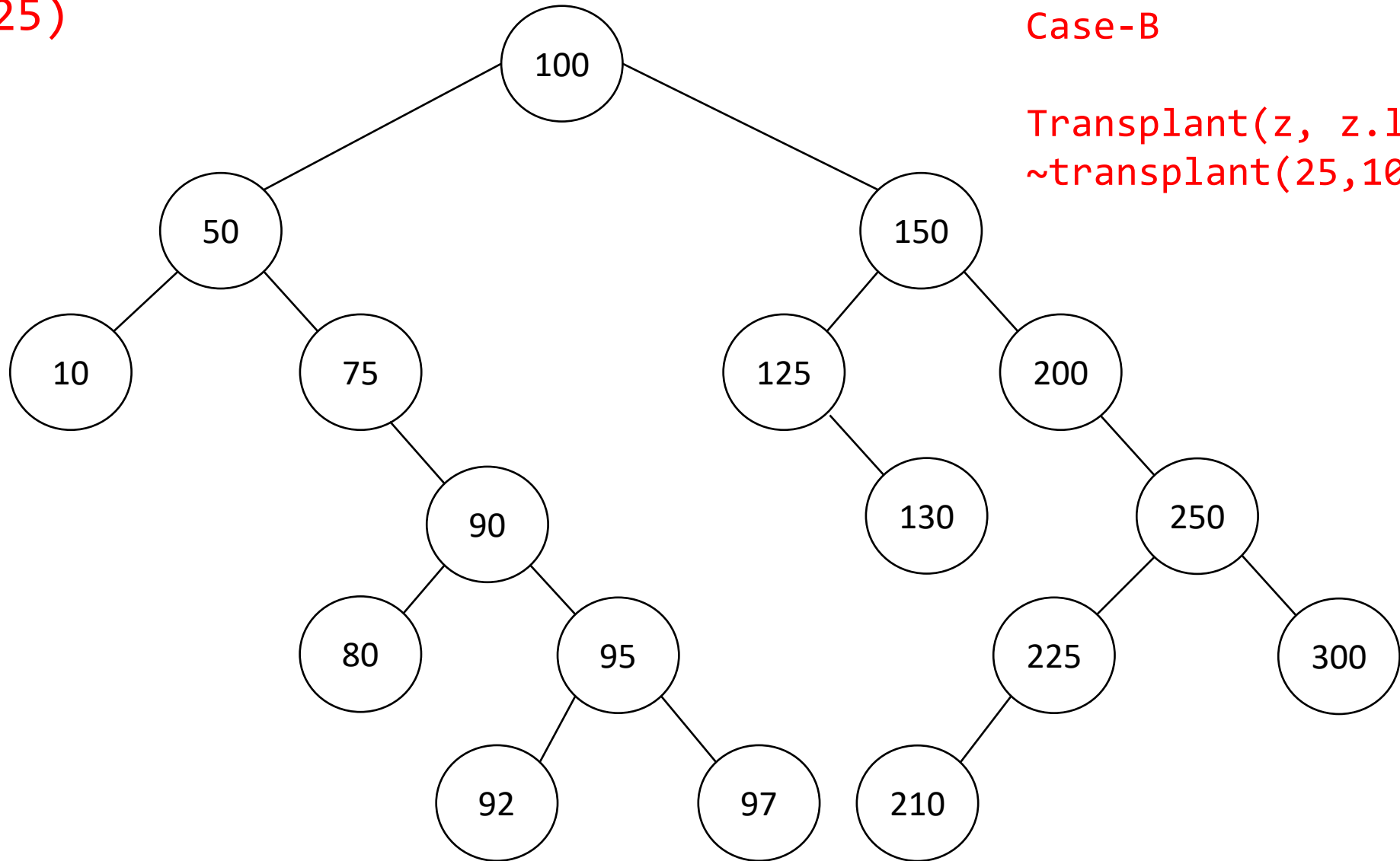
$\text{Transplant}(z, z.\text{left})$   
 $\sim \text{transplant}(25, 10)$



Delete (25)

$z=25$ ,  $z.\text{right}=\text{Null}$   
Case-B

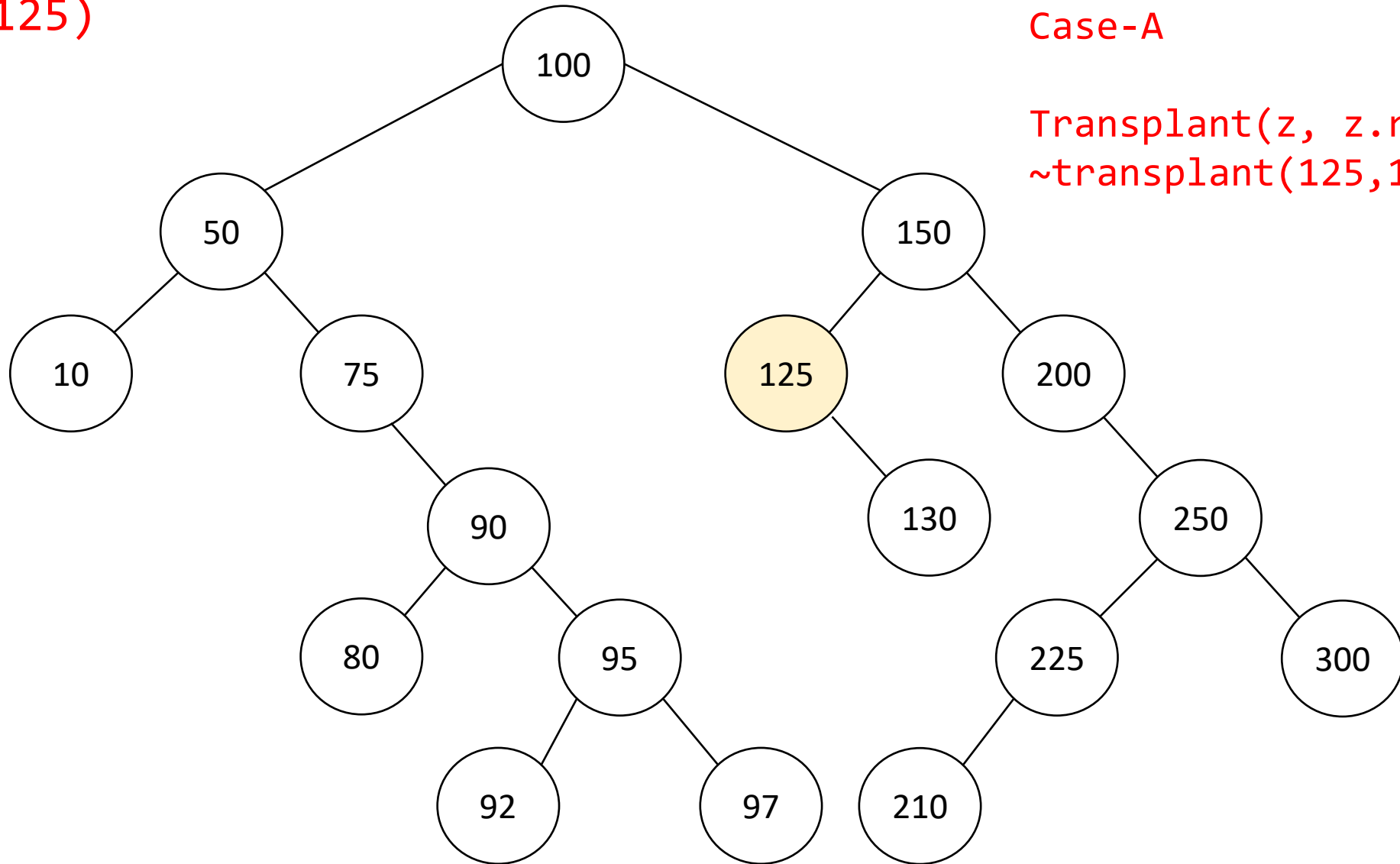
$\text{Transplant}(z, z.\text{left})$   
 $\sim \text{transplant}(25, 10)$



Delete (125)

$z=125$ ,  $z.\text{left}=\text{Null}$   
Case-A

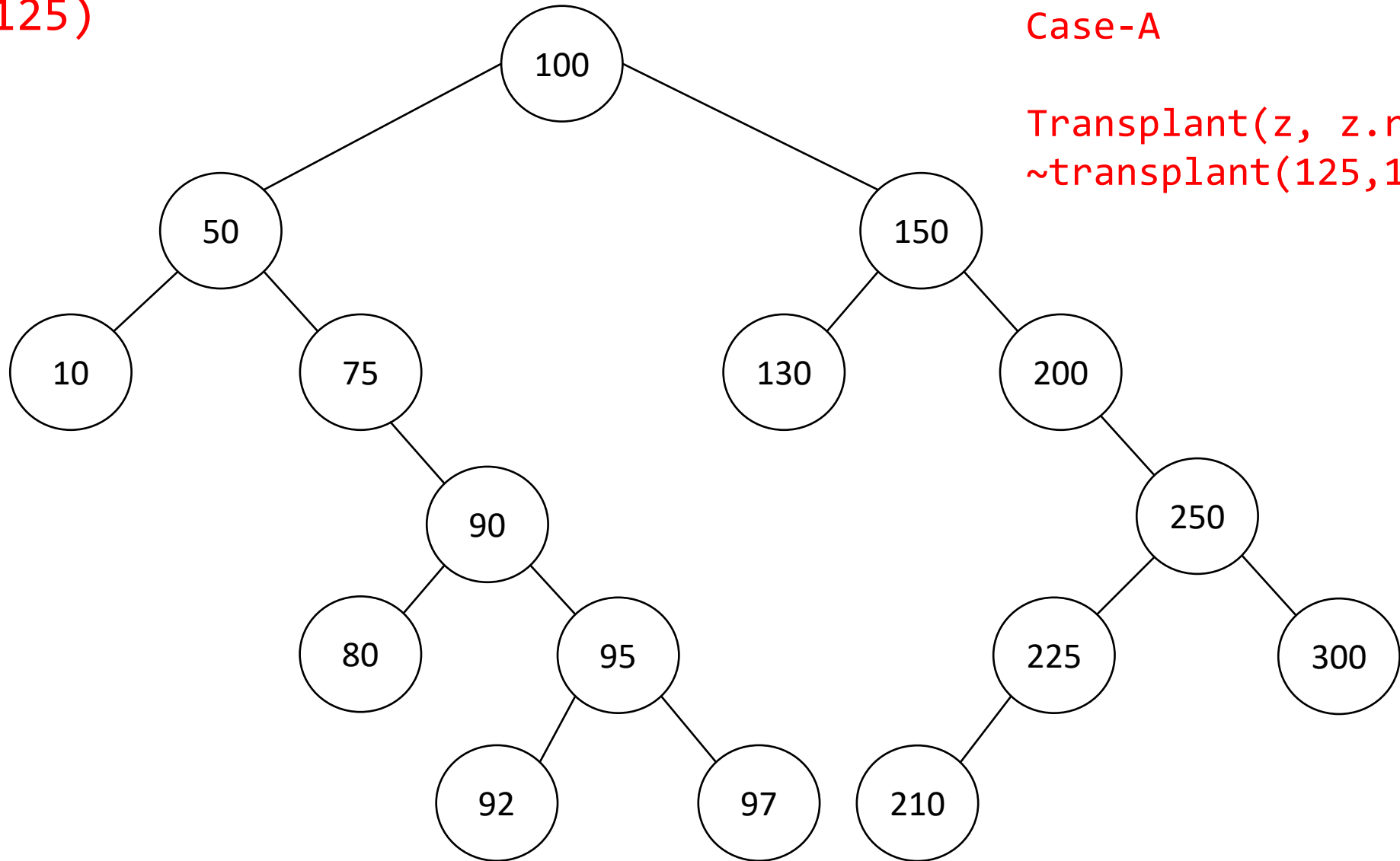
$\text{Transplant}(z, z.\text{right})$   
 $\sim \text{transplant}(125, 130)$



Delete (125)

$z=125$ ,  $z.\text{left}=\text{Null}$   
Case-A

$\text{Transplant}(z, z.\text{right})$   
 $\sim \text{transplant}(125, 130)$

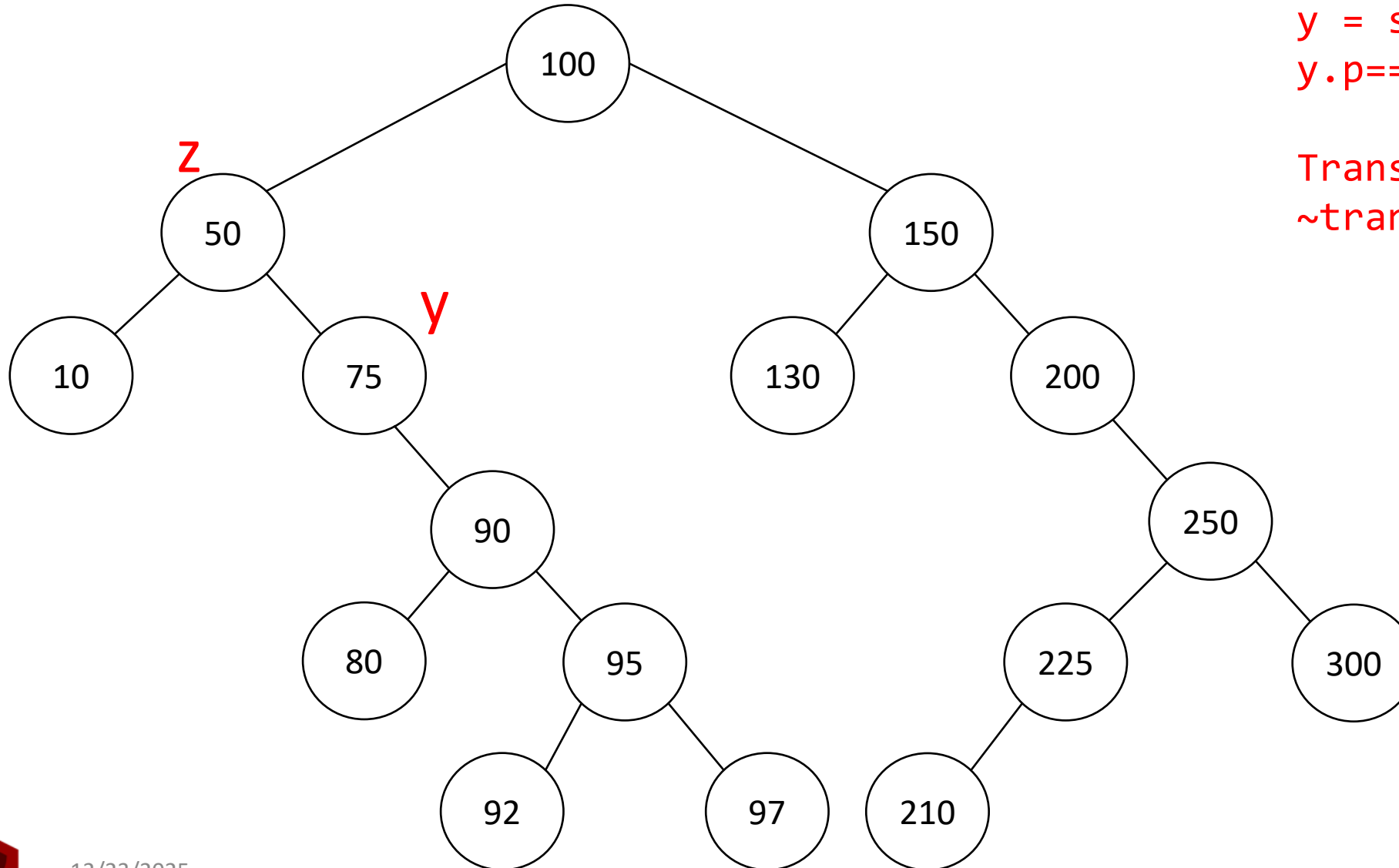


Delete (50)

$z=50$ ,  $z.left \neq \text{Null}$ ,  
 $z.right \neq \text{Null}$

$y = \text{successor}(50) = 75$   
 $y.p == z$ , case-C

$\text{Transplant}(z, y)$   
 $\sim \text{transplant}(50, 75)$



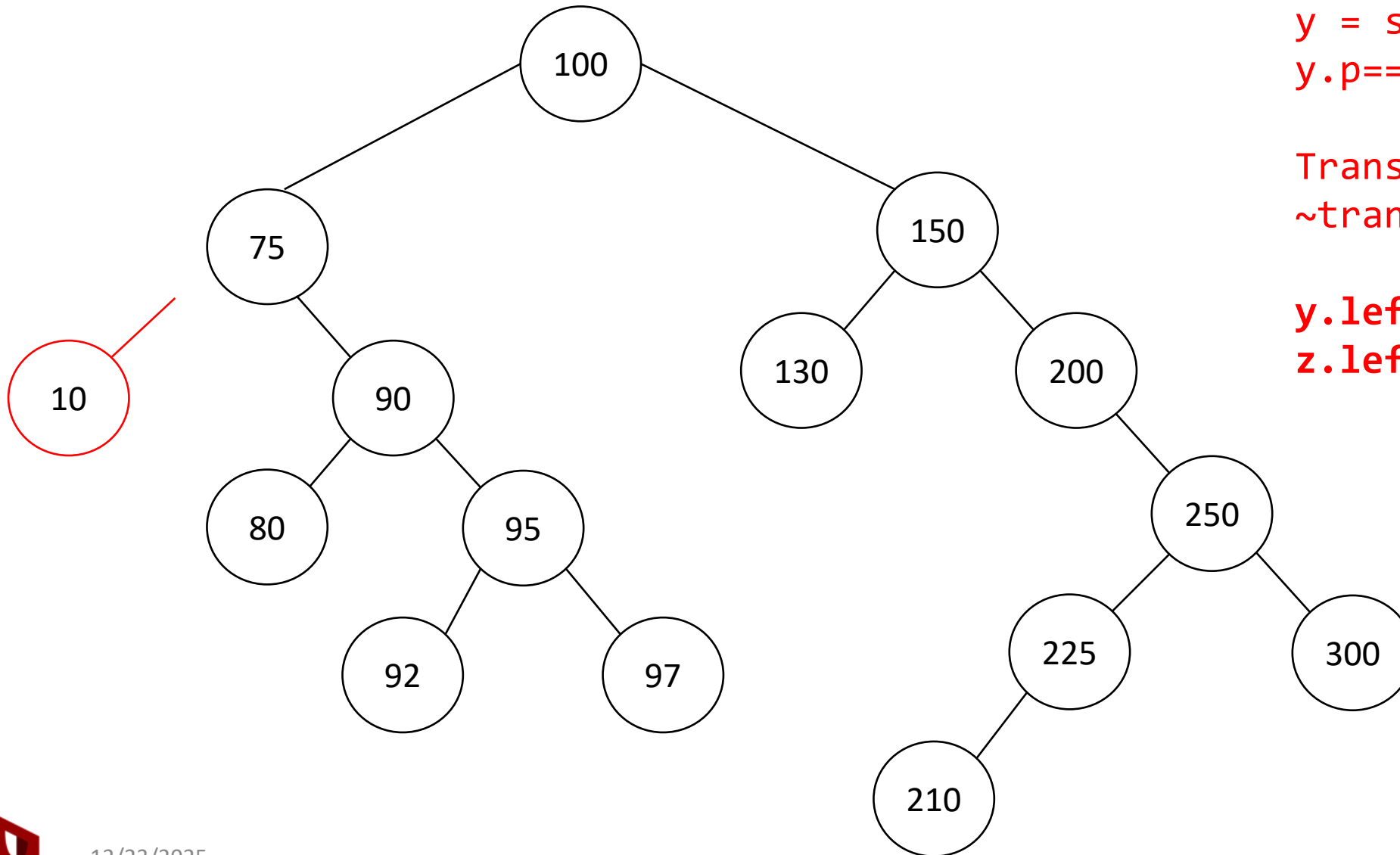
Delete (50)

$z=50$ ,  $z.left \neq \text{Null}$ ,  
 $z.right \neq \text{Null}$

$y = \text{successor}(50) = 75$   
 $y.p == z$ , case-C

$\text{Transplant}(z, y)$   
 $\sim \text{transplant}(50, 75)$

$y.left = z.left$   
 $z.left.p = y$



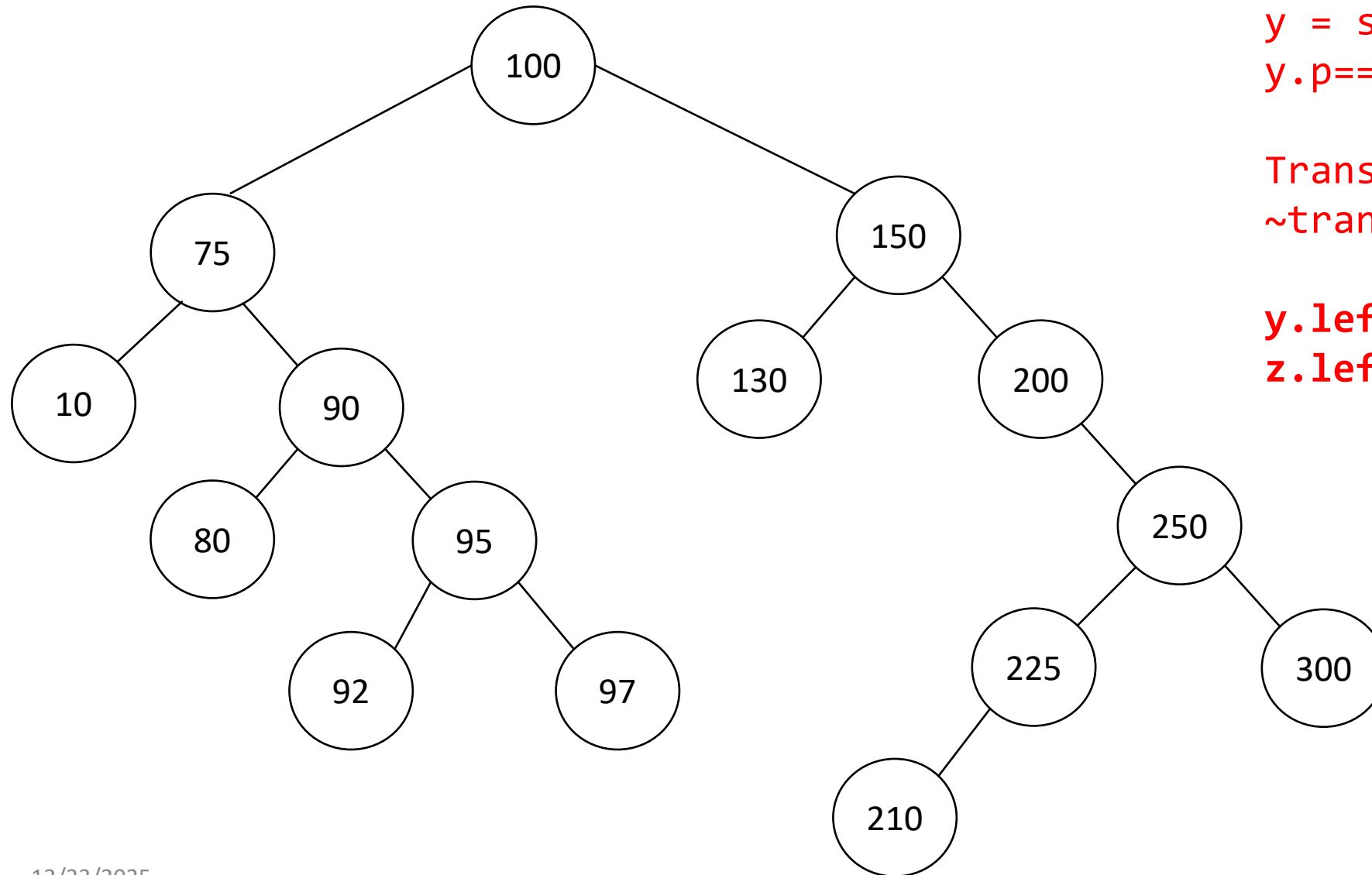
Delete (50)

$z=50$ ,  $z.left \neq \text{Null}$ ,  
 $z.right \neq \text{Null}$

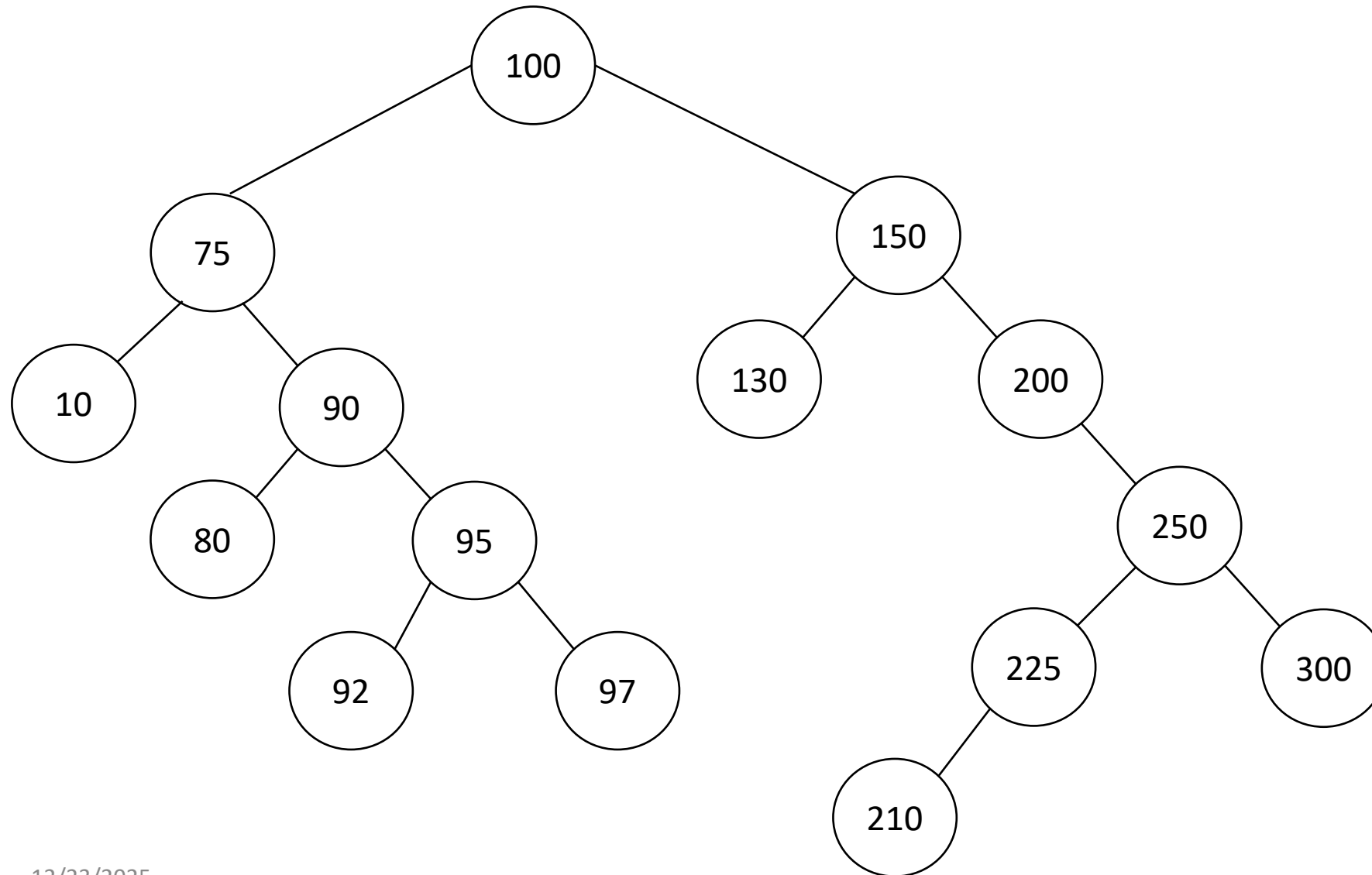
$y = \text{successor}(50) = 75$   
 $y.p == z$ , case-C

$\text{Transplant}(z, y)$   
 $\sim \text{transplant}(50, 75)$

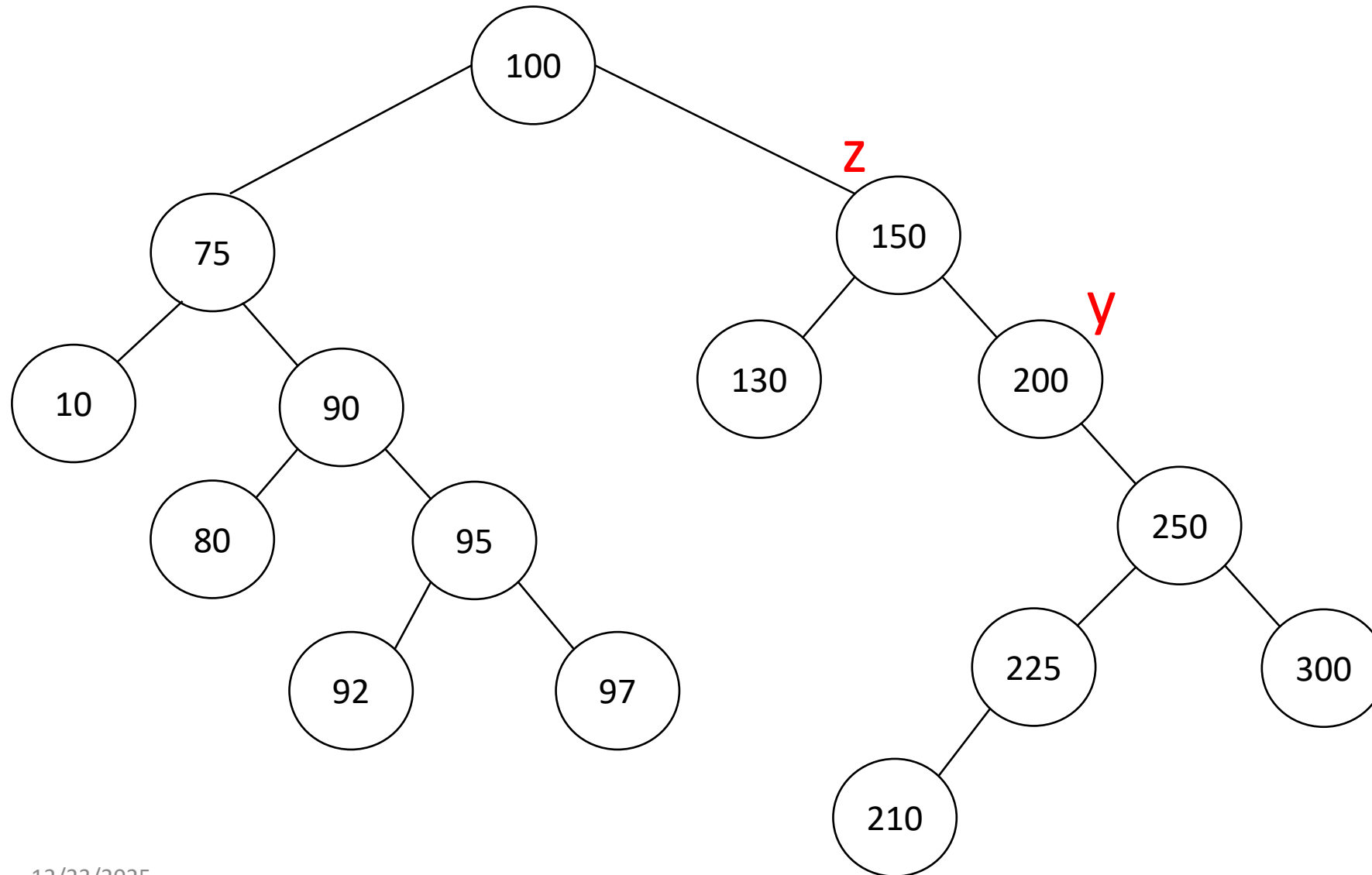
$y.left = z.left$   
 $z.left.p = y$



Delete (150)



Delete (150)



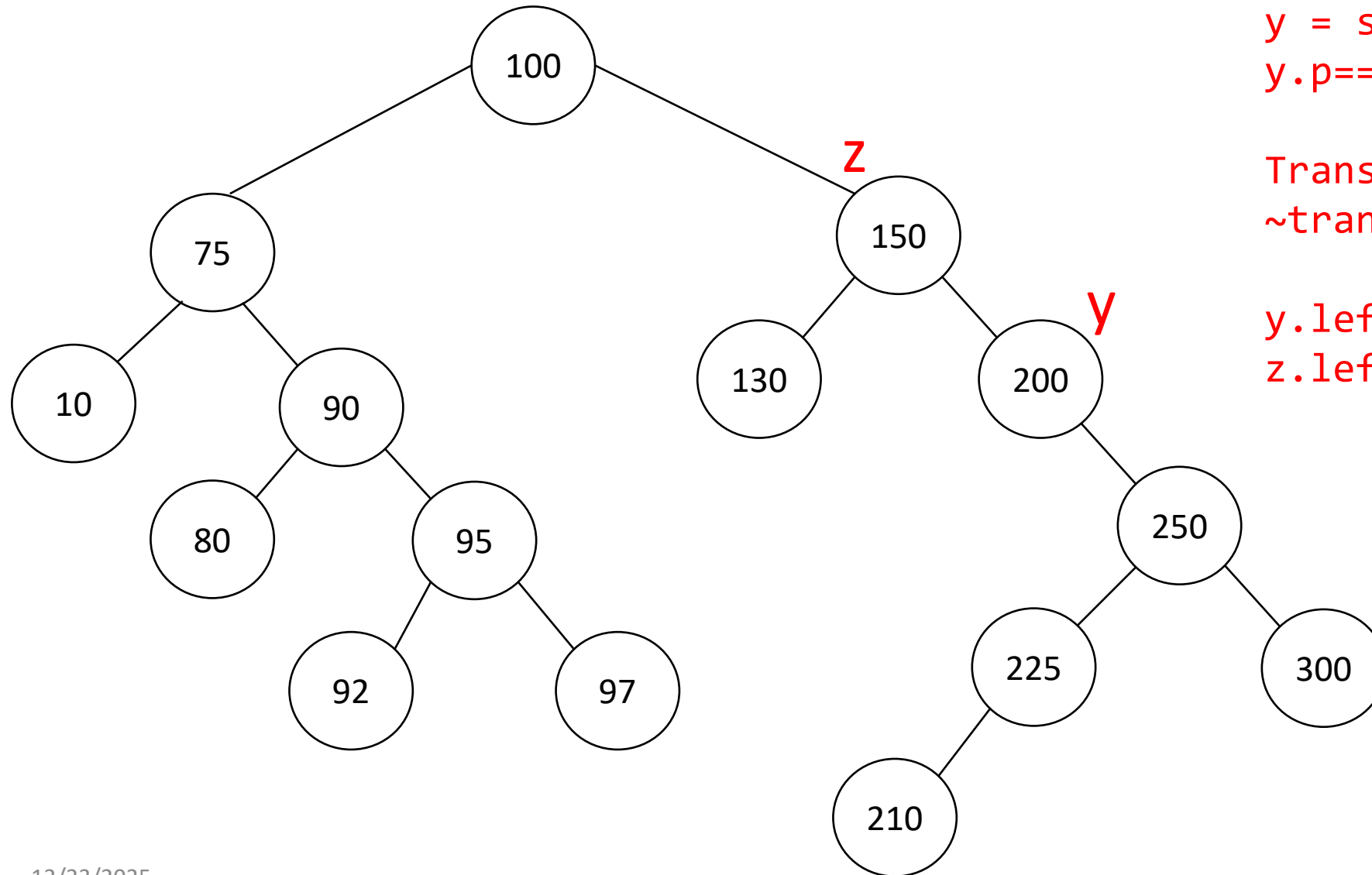
## Delete (150)

$z=150$ ,  $z.left \neq \text{Null}$ ,  
 $z.right \neq \text{Null}$

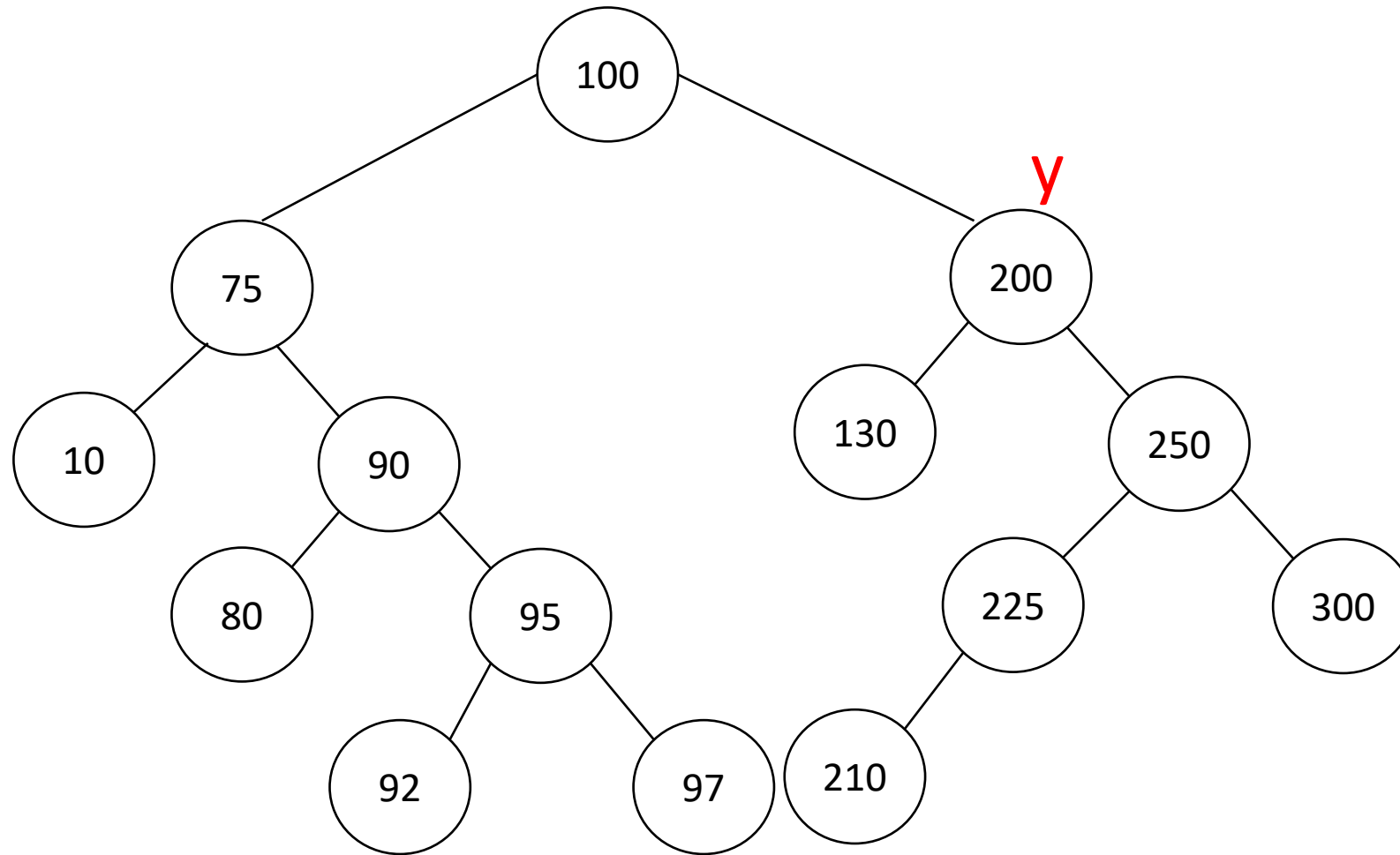
$y = \text{successor}(150)=200$   
 $y.p == z$ , case-C

$\text{Transplant}(z, y)$   
 $\sim \text{transplant}(150, 200)$

$y.left = z.left$   
 $z.left.p = y$



Delete (150)



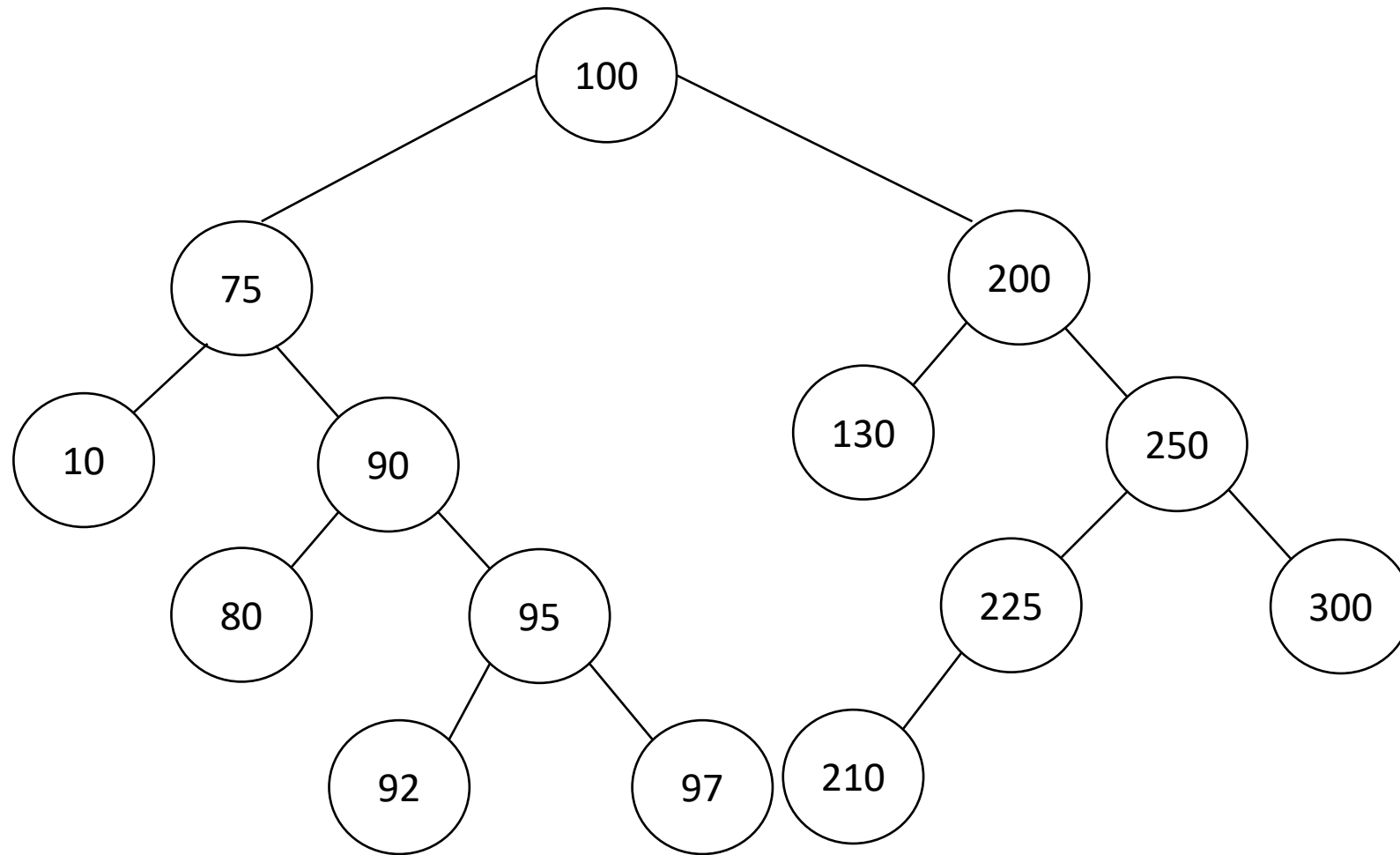
$z=150$ ,  $z.left \neq \text{Null}$ ,  
 $z.right \neq \text{Null}$

$y = \text{successor}(150)=200$   
 $y.p == z$ , case-C

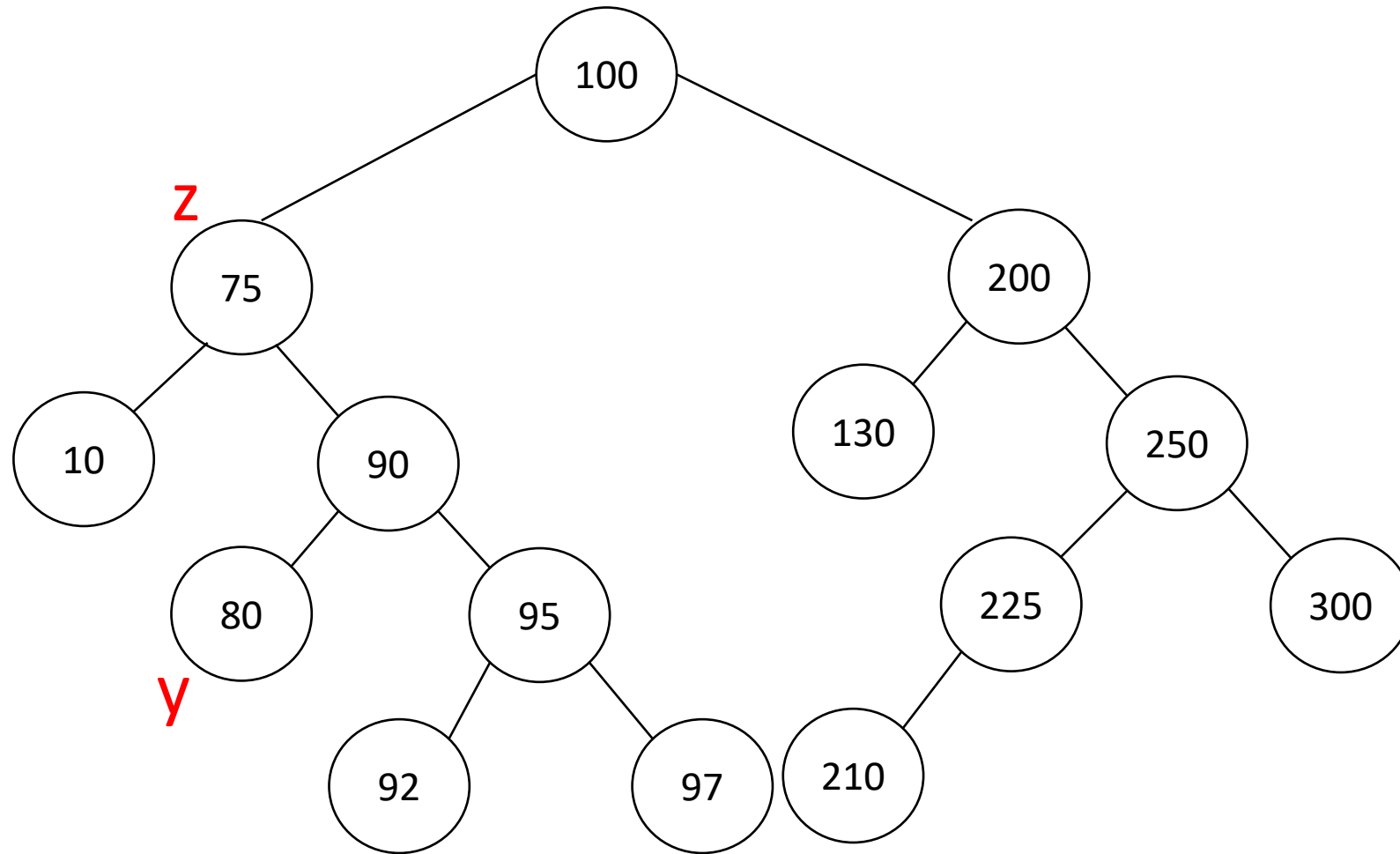
$\text{Transplant}(z, y)$   
 $\sim \text{transplant}(150, 200)$

$y.left = z.left$   
 $z.left.p = y$

Delete(75)



Delete(75)

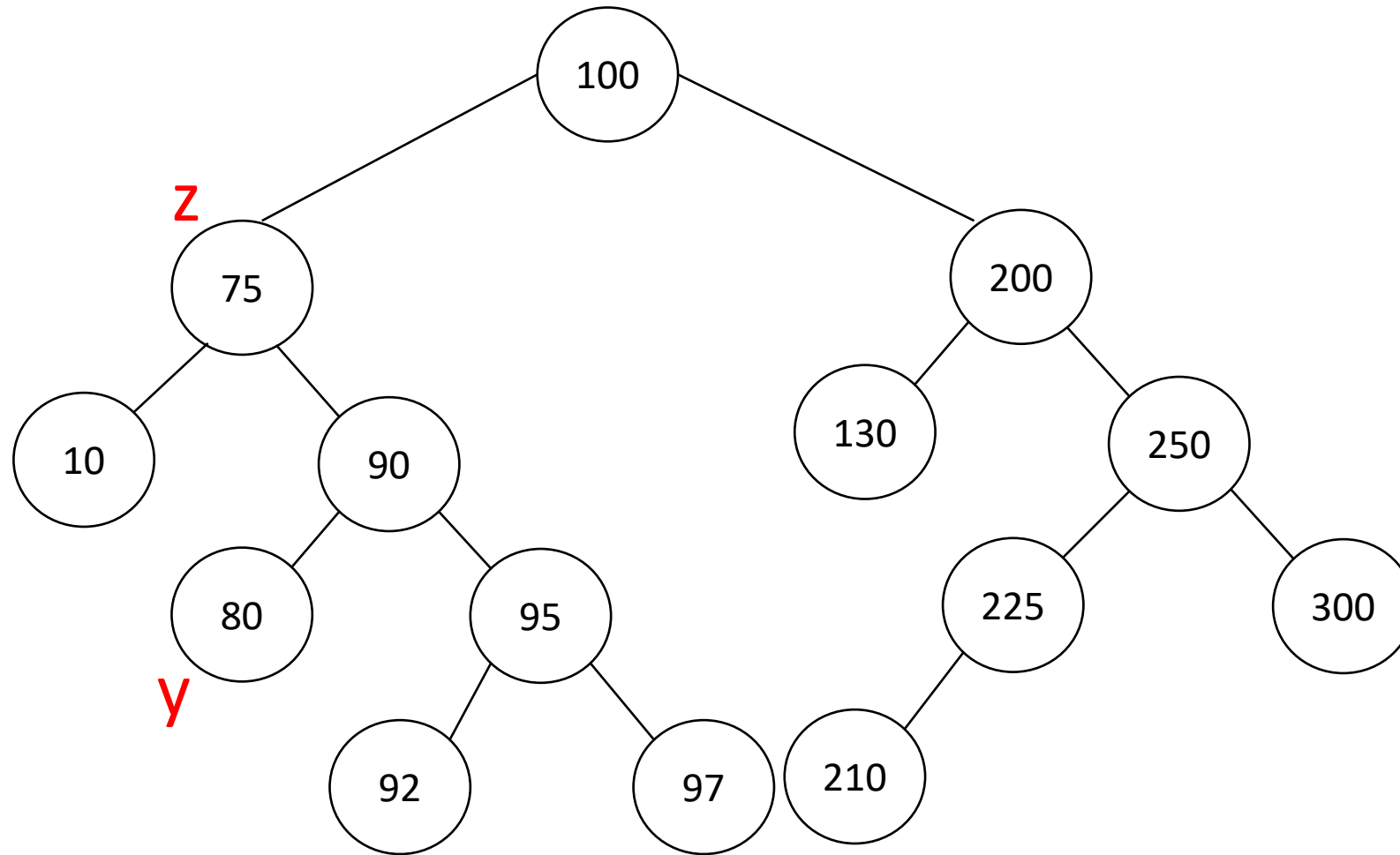


Delete(75)

$z=75$ ,  $z.left \neq \text{Null}$ ,  
 $z.right \neq \text{Null}$

$y = \text{successor}(75)=80$   
 $y.p \neq z$ , case-D

$\text{Transplant}(y, y.right)$   
 $\sim \text{Transplant}(80, \text{Null})$

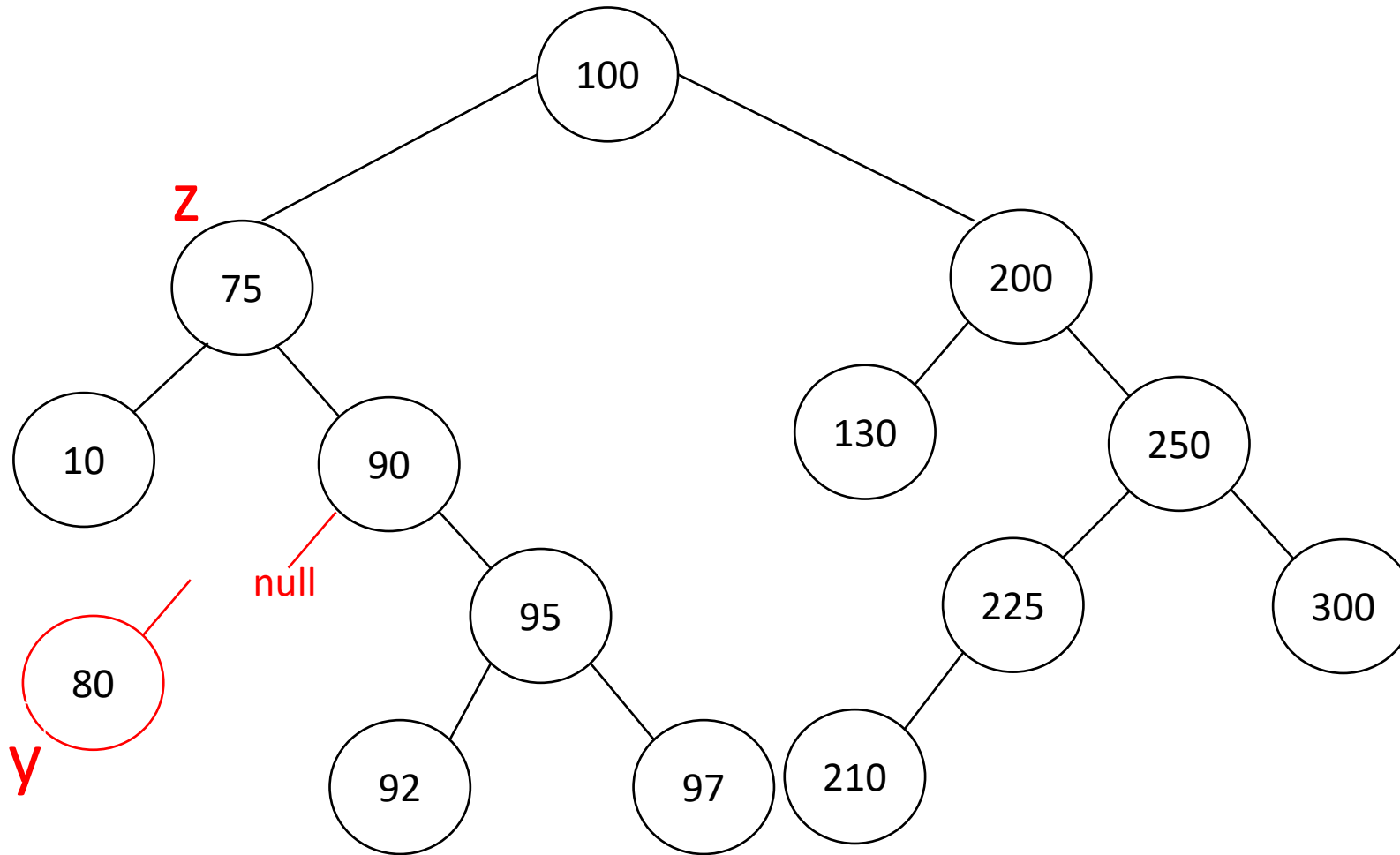


Delete(75)

$z=75$ ,  $z.left \neq \text{Null}$ ,  
 $z.right \neq \text{Null}$

$y = \text{successor}(75)=80$   
 $y.p \neq z$ , case-D

$\text{Transplant}(y, y.right)$   
 $\sim \text{Transplant}(80, \text{Null})$



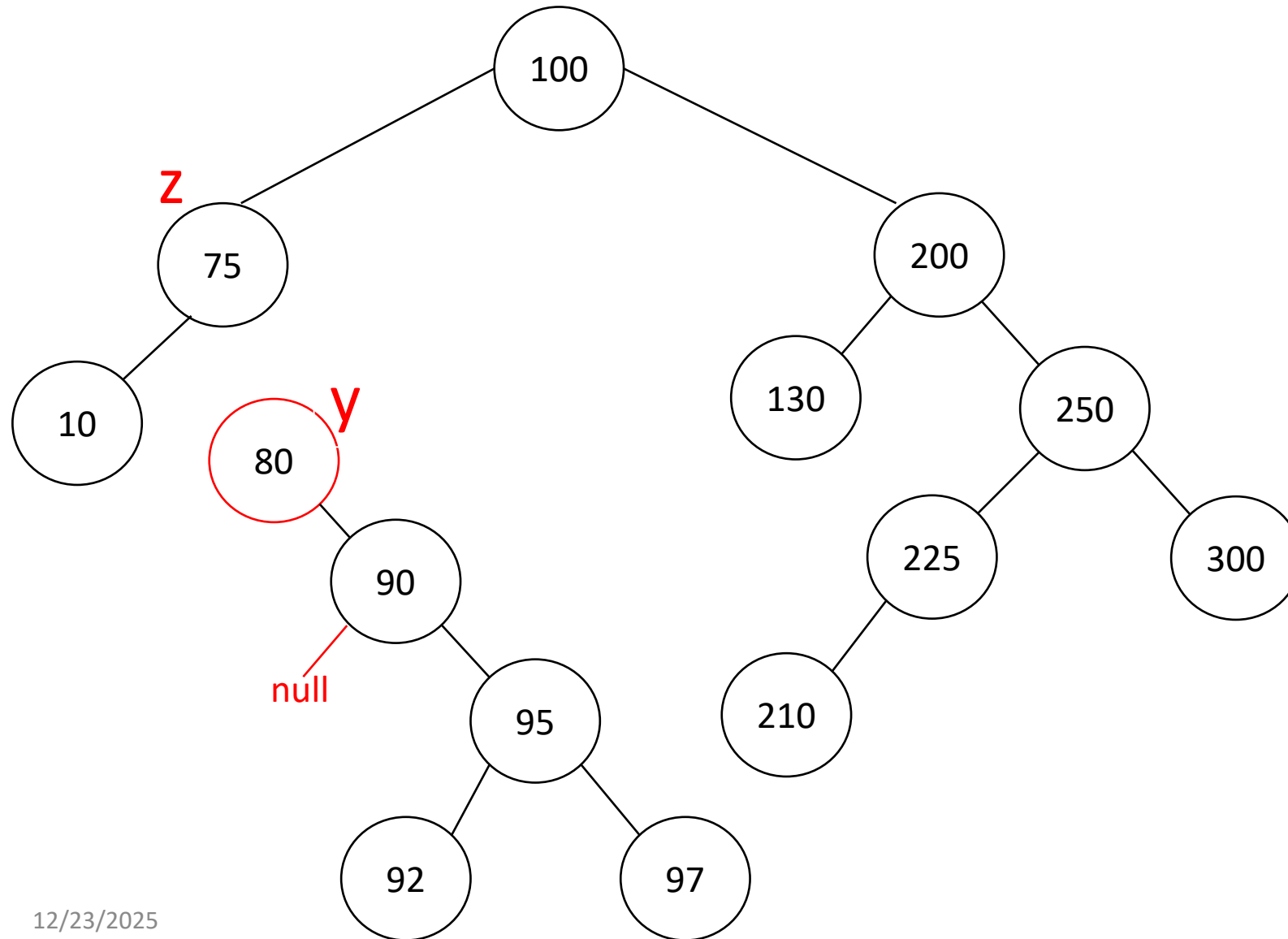
Delete(75)

$z=75$ ,  $z.left \neq \text{Null}$ ,  
 $z.right \neq \text{Null}$

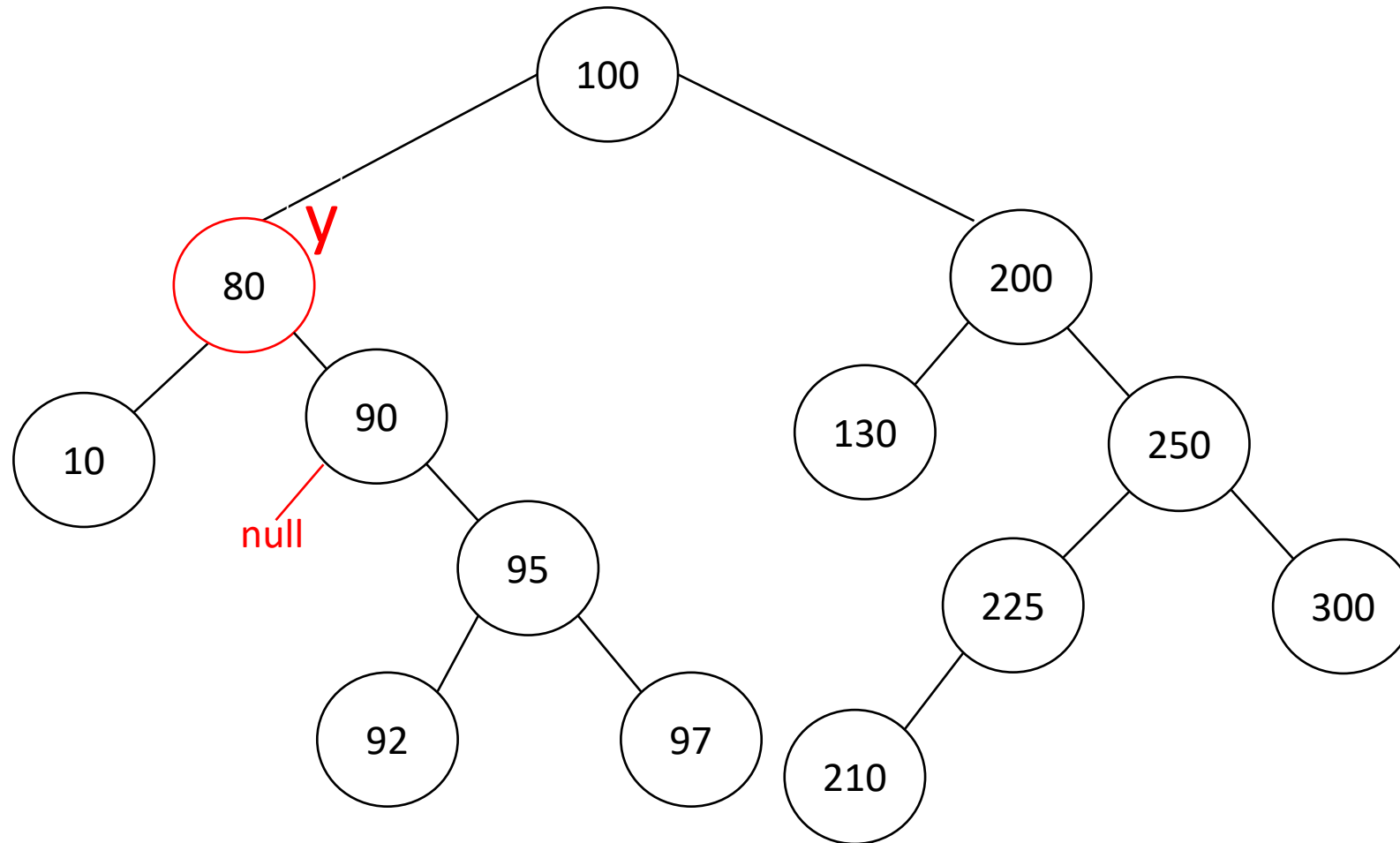
$y = \text{successor}(75)=80$   
 $y.p \neq z$ , case-D

$\text{Transplant}(y, y.right)$   
 $\sim \text{Transplant}(80, \text{Null})$

$y.right = z.right$   
 $z.right.p = y$



Delete(75)



$z=75$ ,  $z.left \neq \text{Null}$ ,  
 $z.right \neq \text{Null}$

$y = \text{successor}(75)=80$   
 $y.p \neq z$ , case-D

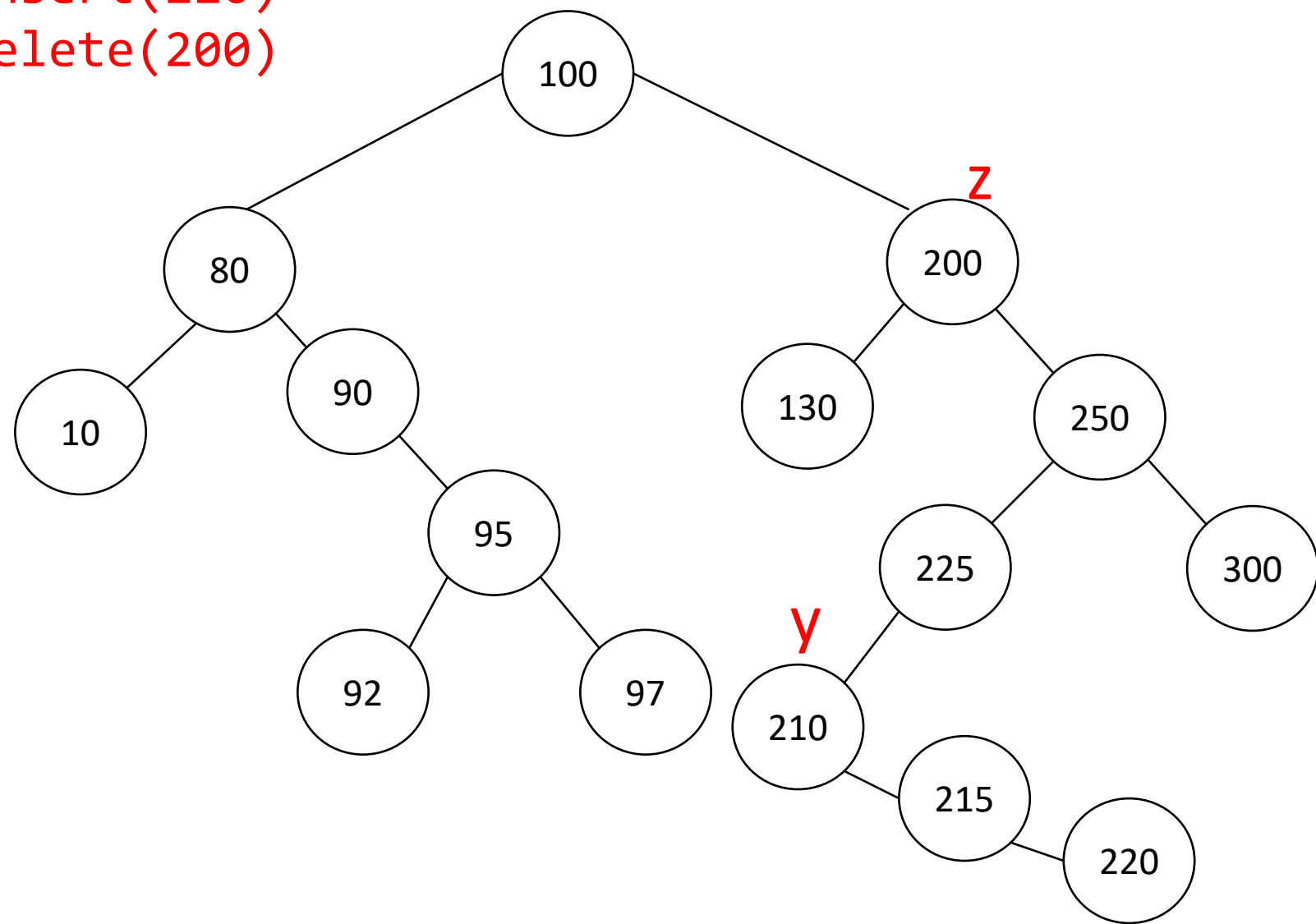
$\text{Transplant}(y, y.right)$   
 $\sim \text{Transplant}(80, \text{Null})$

$y.right = z.right$   
 $z.right.p = y$

$\text{Transplant}(z, y)$   
 $\sim \text{transplant}(75, 80)$

$y.left = z.left$   
 $z.left.p = y$

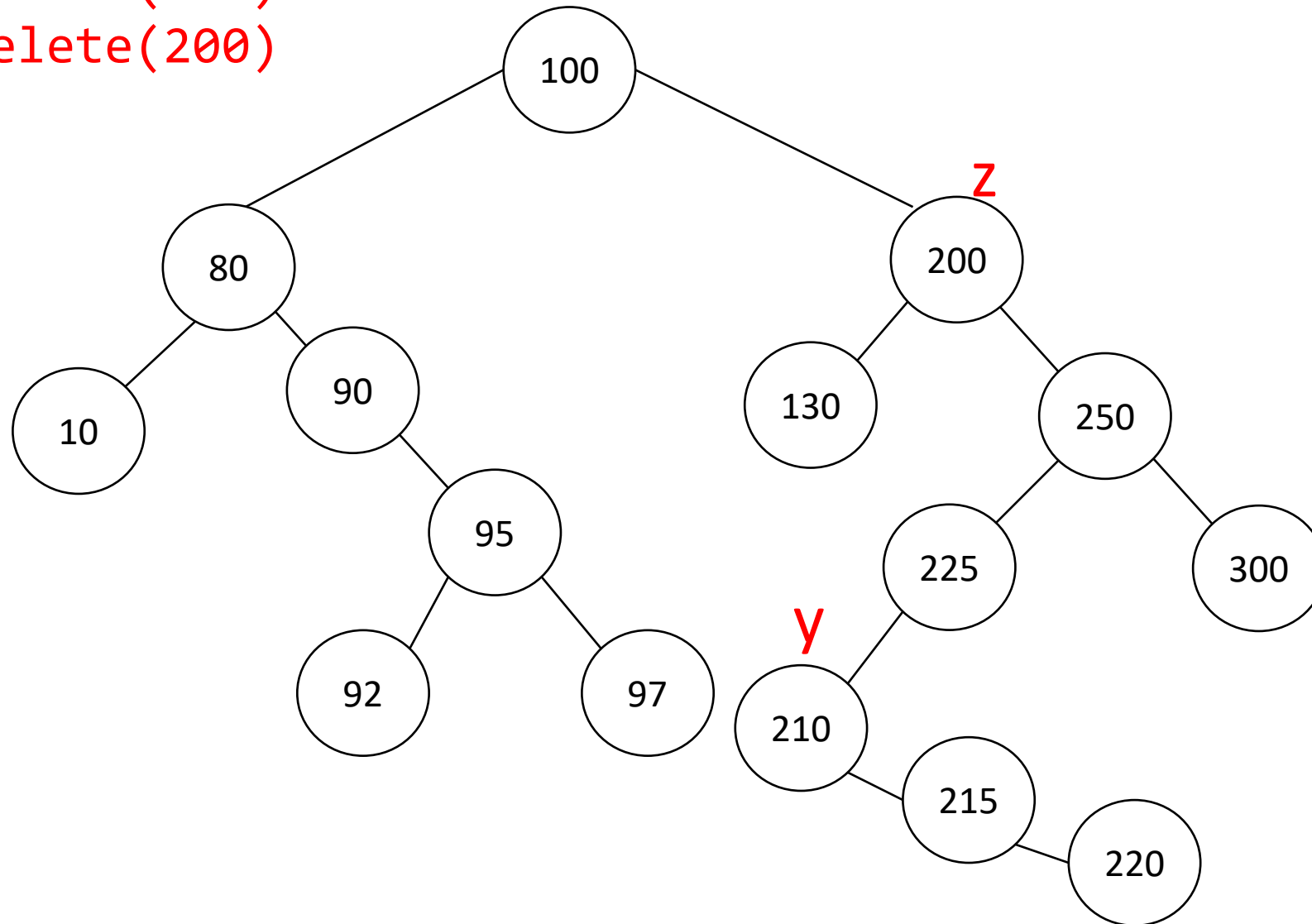
Insert(215)  
Insert(220)  
Delete(200)



Insert(215)  
Insert(220)  
Delete(200)

$z=200$ ,  $z.left \neq \text{Null}$ ,  
 $z.right \neq \text{Null}$

$y = \text{successor}(200)=210$   
 $y.p \neq z$ , case-D

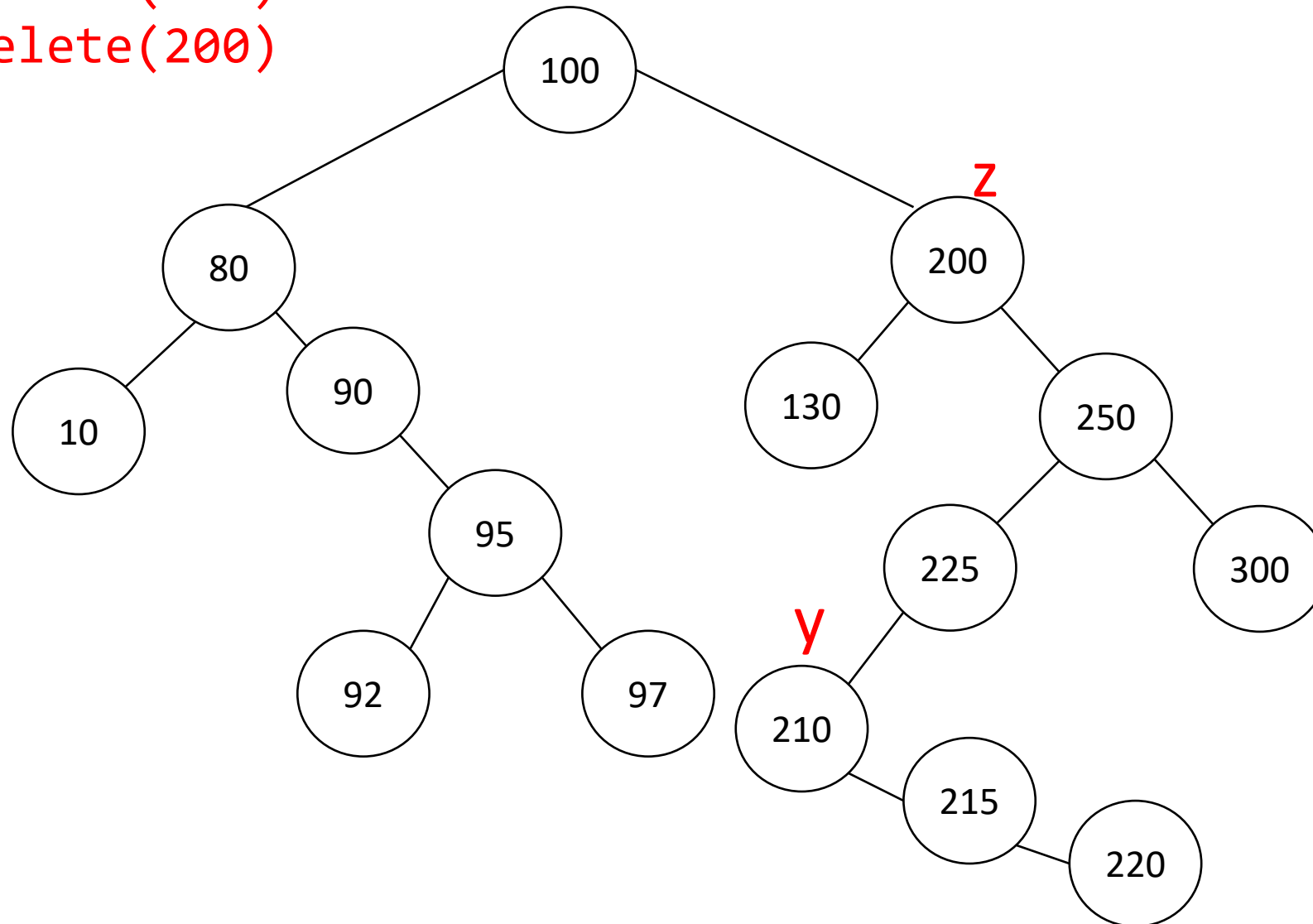


Insert(215)  
Insert(220)  
Delete(200)

$z = 200$ ,  $z.\text{left} \neq \text{Null}$ ,  
 $z.\text{right} \neq \text{Null}$

$y = \text{successor}(200) = 210$   
 $y.p \neq z$ , case-D

Transplant( $y$ ,  $y.\text{right}$ )  
~Transplant(210, 215)

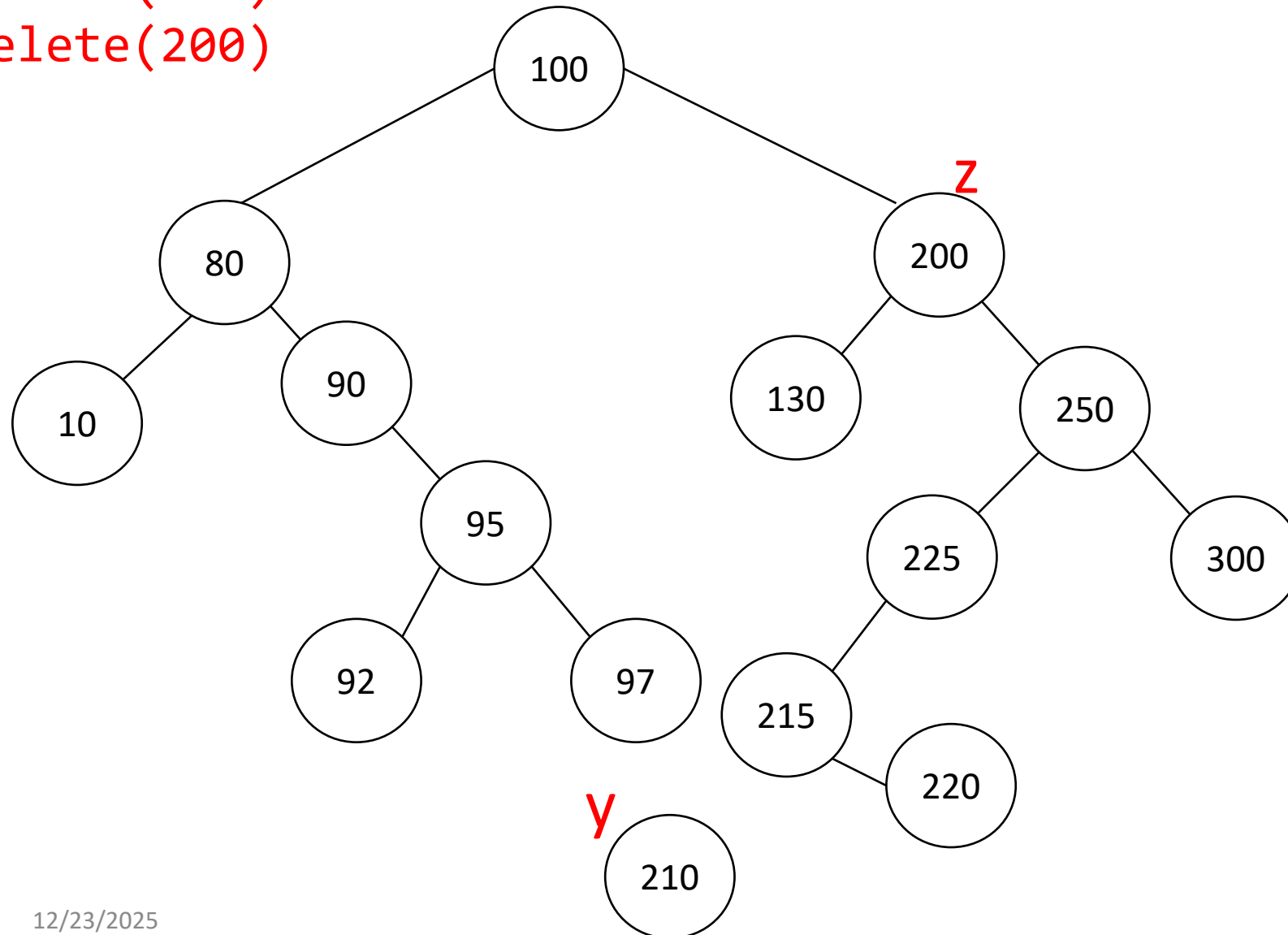


Insert(215)  
Insert(220)  
Delete(200)

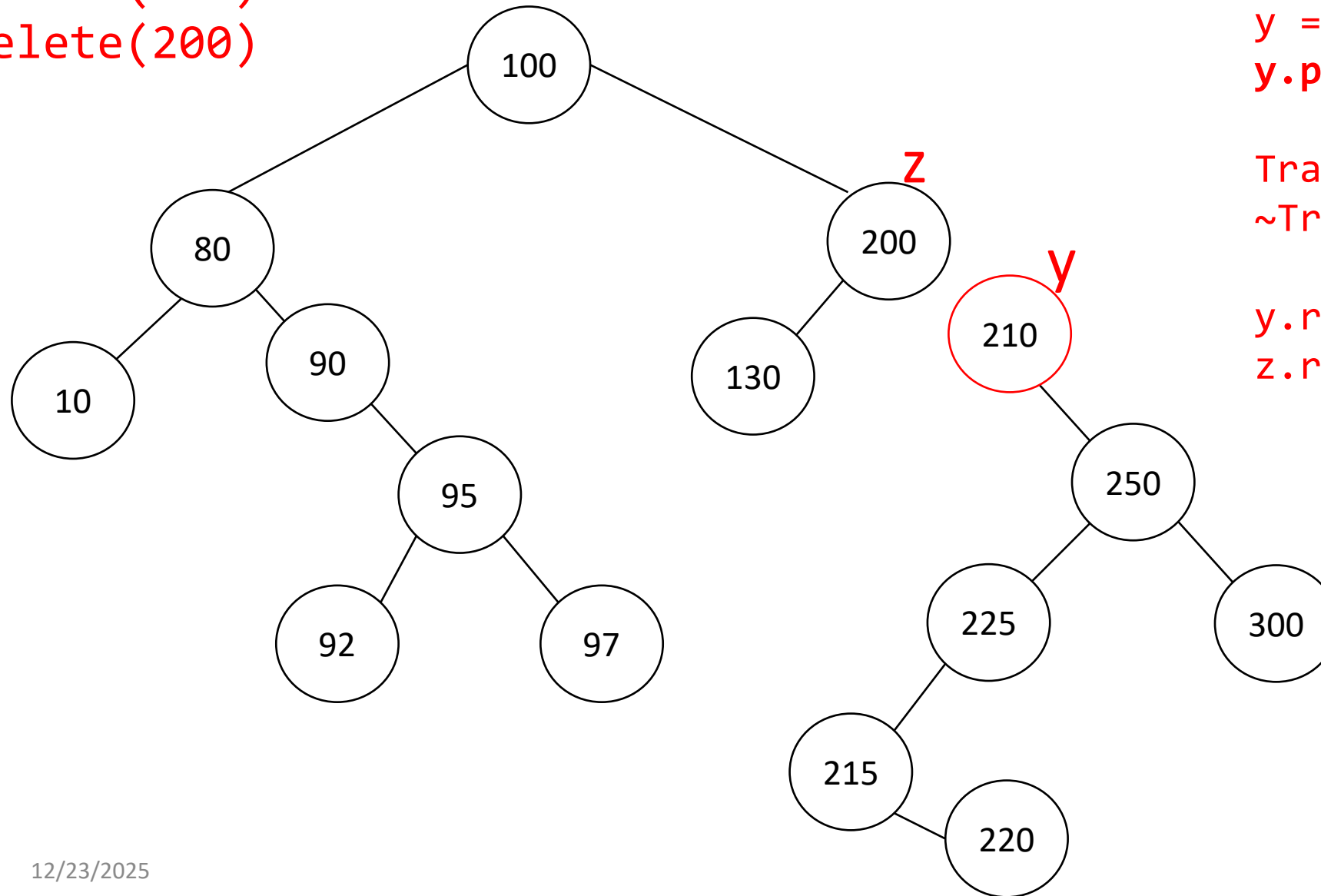
$z = 200$ ,  $z.\text{left} \neq \text{Null}$ ,  
 $z.\text{right} \neq \text{Null}$

$y = \text{successor}(200) = 210$   
 $y.p \neq z$ , case-D

Transplant( $y$ ,  $y.\text{right}$ )  
~Transplant(210, 215)



Insert(215)  
Insert(220)  
Delete(200)



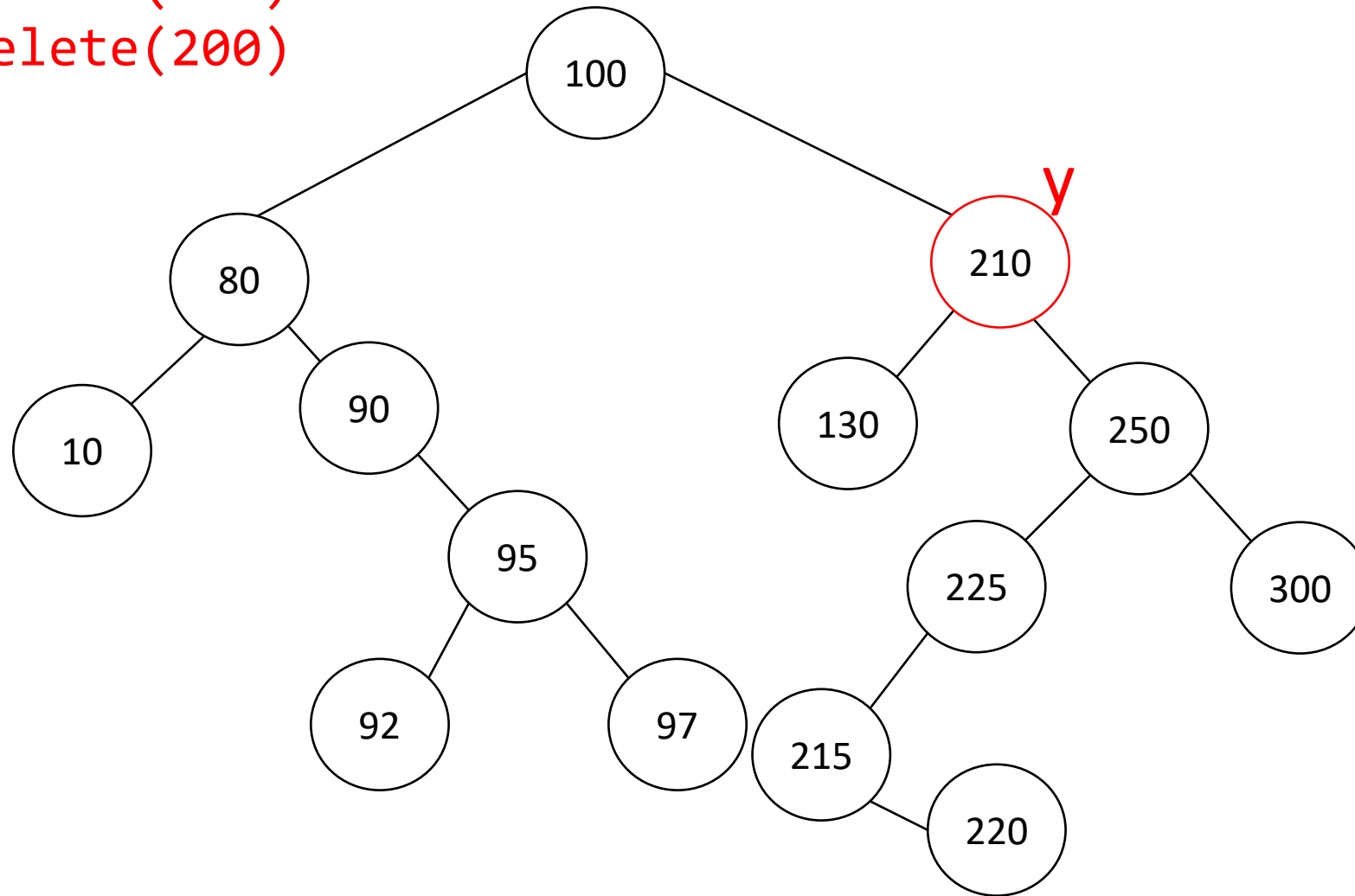
$z=200$ ,  $z.left \neq \text{Null}$ ,  
 $z.right \neq \text{Null}$

$y = \text{successor}(200)=210$   
 $y.p \neq z$ , case-D

$\text{Transplant}(y, y.right)$   
 $\sim \text{Transplant}(210, 215)$

$y.right = z.right$   
 $z.right.p = y$

Insert(215)  
Insert(220)  
Delete(200)



$z=200$ ,  $z.left \neq \text{Null}$ ,  
 $z.right \neq \text{Null}$

$y = \text{successor}(200)=210$   
 $y.p \neq z$ , **case-D**

$\text{Transplant}(y, y.right)$   
 $\sim \text{Transplant}(210, 215)$

$y.right = z.right$   
 $z.right.p = y$

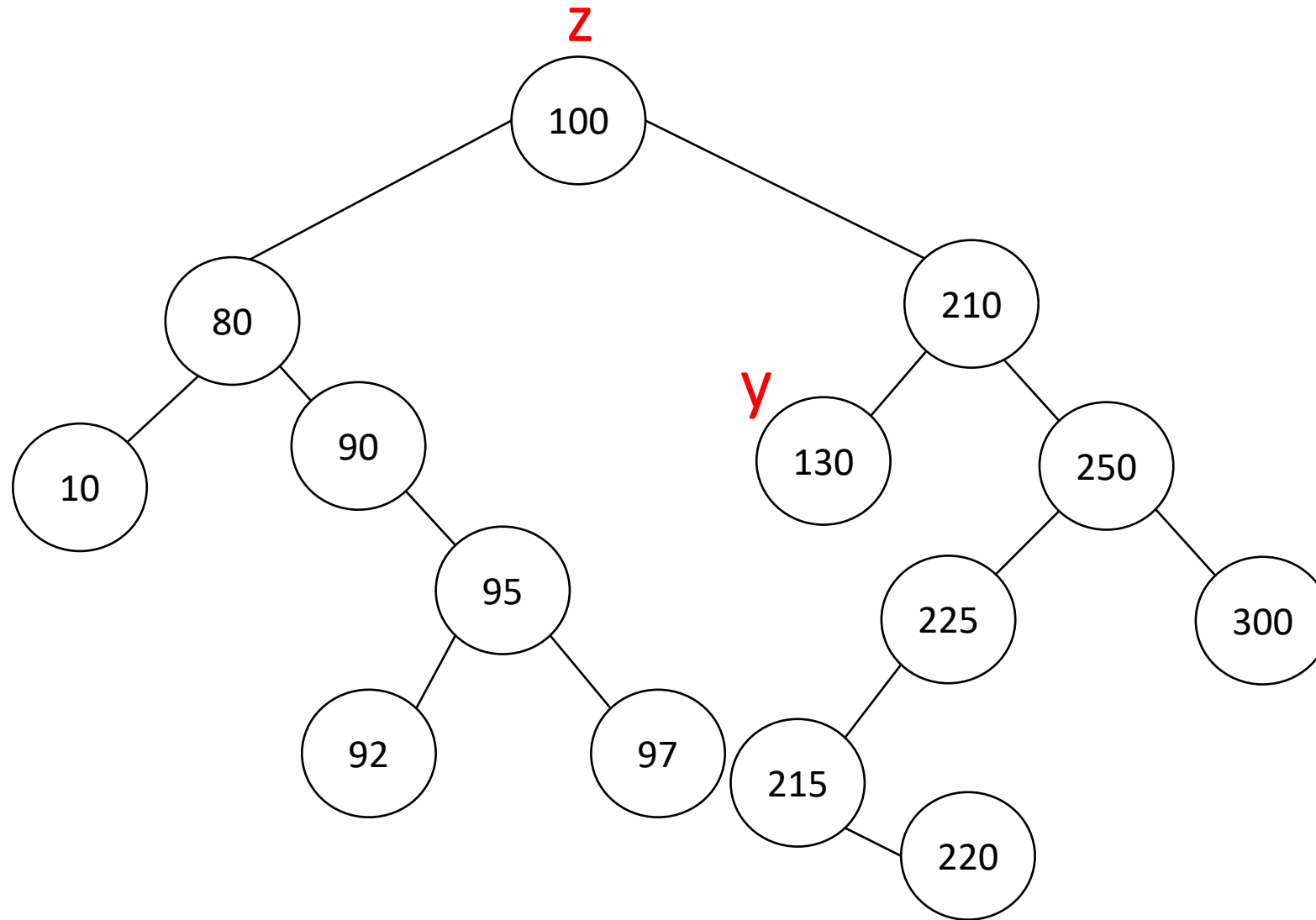
$\text{Transplant}(z, y)$   
 $\sim \text{transplant}(200, 210)$

$y.left = z.left$   
 $z.left.p = y$

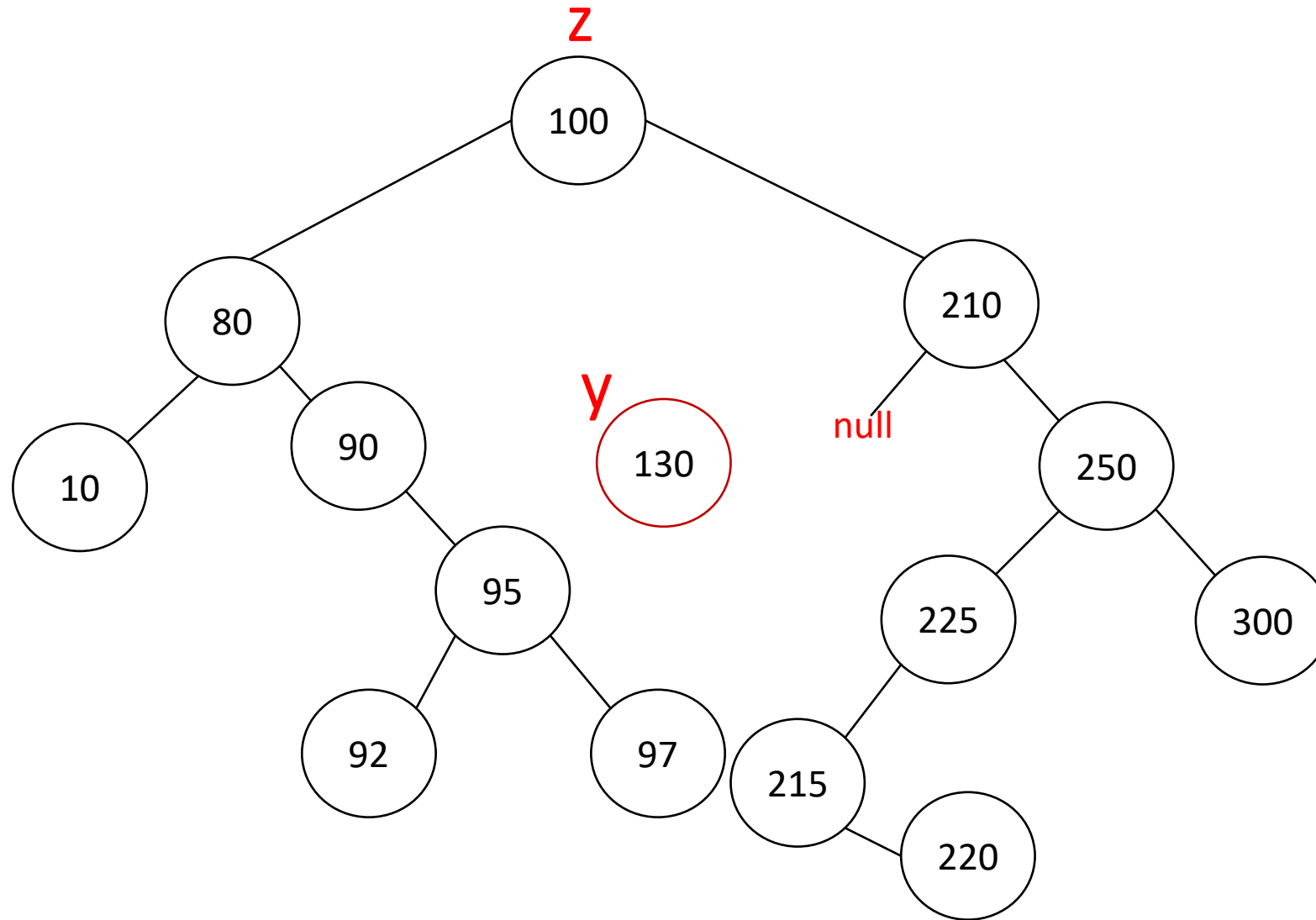
Delete(200)

$z=100$ ,  $z.left \neq \text{Null}$ ,  
 $z.right \neq \text{Null}$

$y = \text{successor}(100)=130$   
 $y.p \neq z$ , case-D



Delete(200)

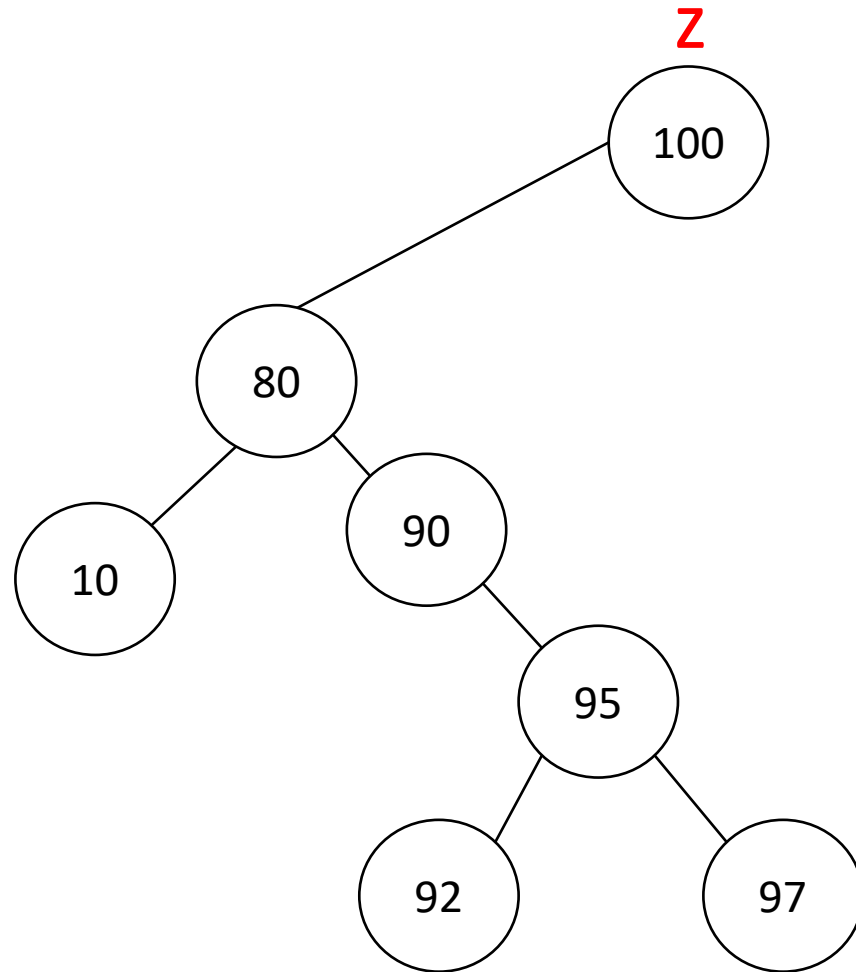


$z=100$ ,  $z.left \neq \text{Null}$ ,  
 $z.right \neq \text{Null}$

$y = \text{successor}(100)=130$   
 $y.p \neq z$ , case-D

$\text{Transplant}(y, y.right)$   
 $\sim \text{Transplant}(130, \text{Null})$

Delete(200)

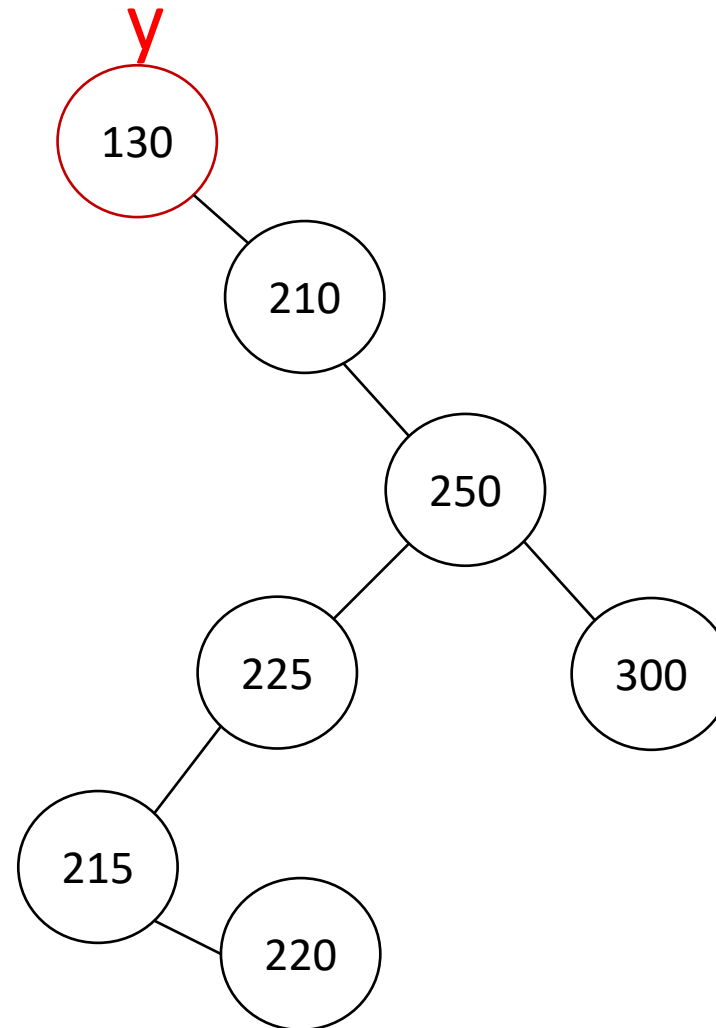


$z=100$ ,  $z.left \neq \text{Null}$ ,  
 $z.right \neq \text{Null}$

$y = \text{successor}(100)=130$   
 $y.p \neq z$ , case-D

$\text{Transplant}(y, y.right)$   
 $\sim \text{Transplant}(130, \text{Null})$

$y.right = z.right$   
 $z.right.p = y$



Delete(200)

$z=100$ ,  $z.left \neq \text{Null}$ ,  
 $z.right \neq \text{Null}$

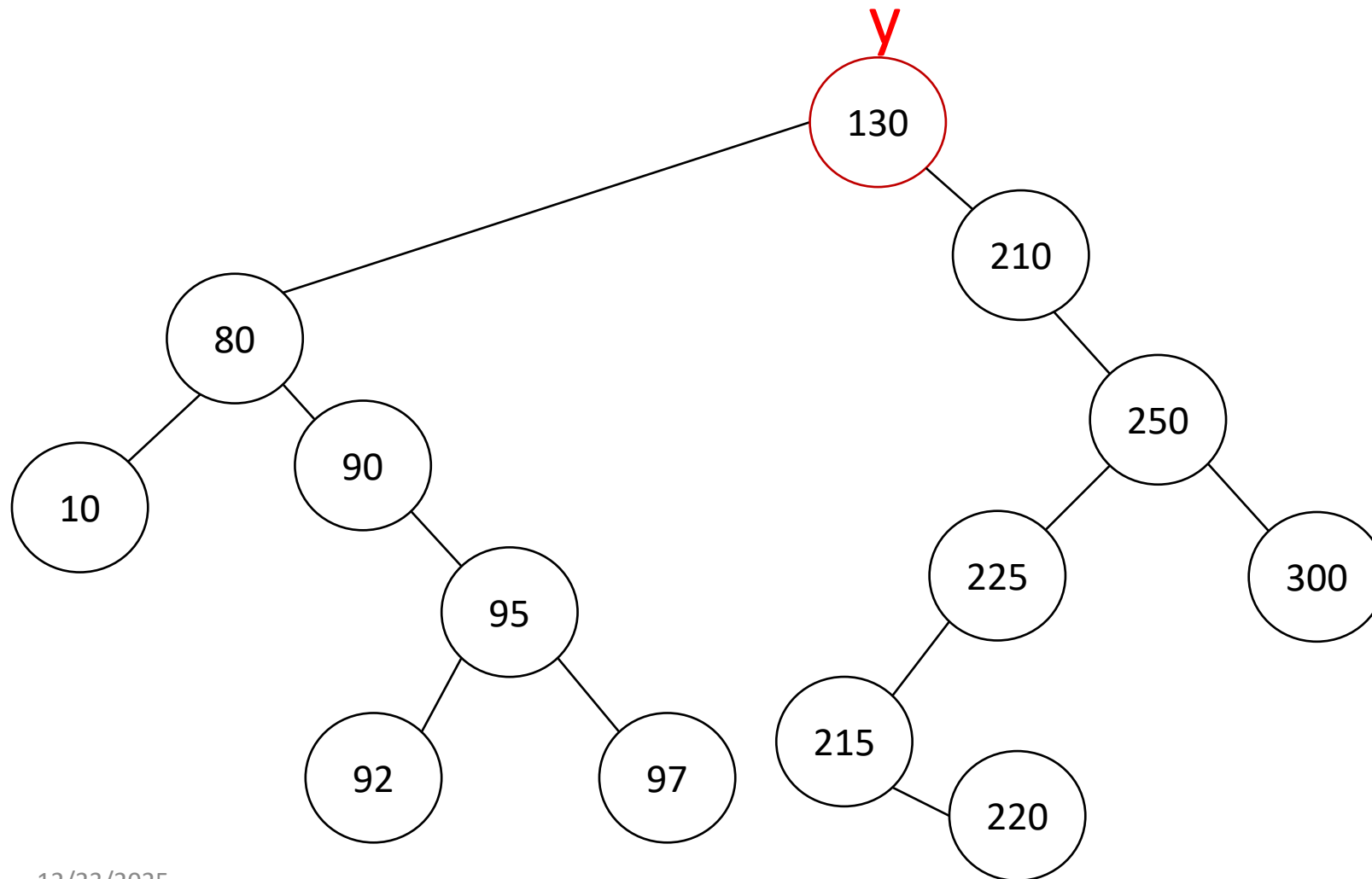
$y = \text{successor}(100)=130$   
 $y.p \neq z$ , case-D

$\text{Transplant}(y, y.right)$   
 $\sim \text{Transplant}(130, \text{Null})$

$y.right = z.right$   
 $z.right.p = y$

$\text{Transplant}(z, y)$   
 $\sim \text{transplant}(200, 210)$

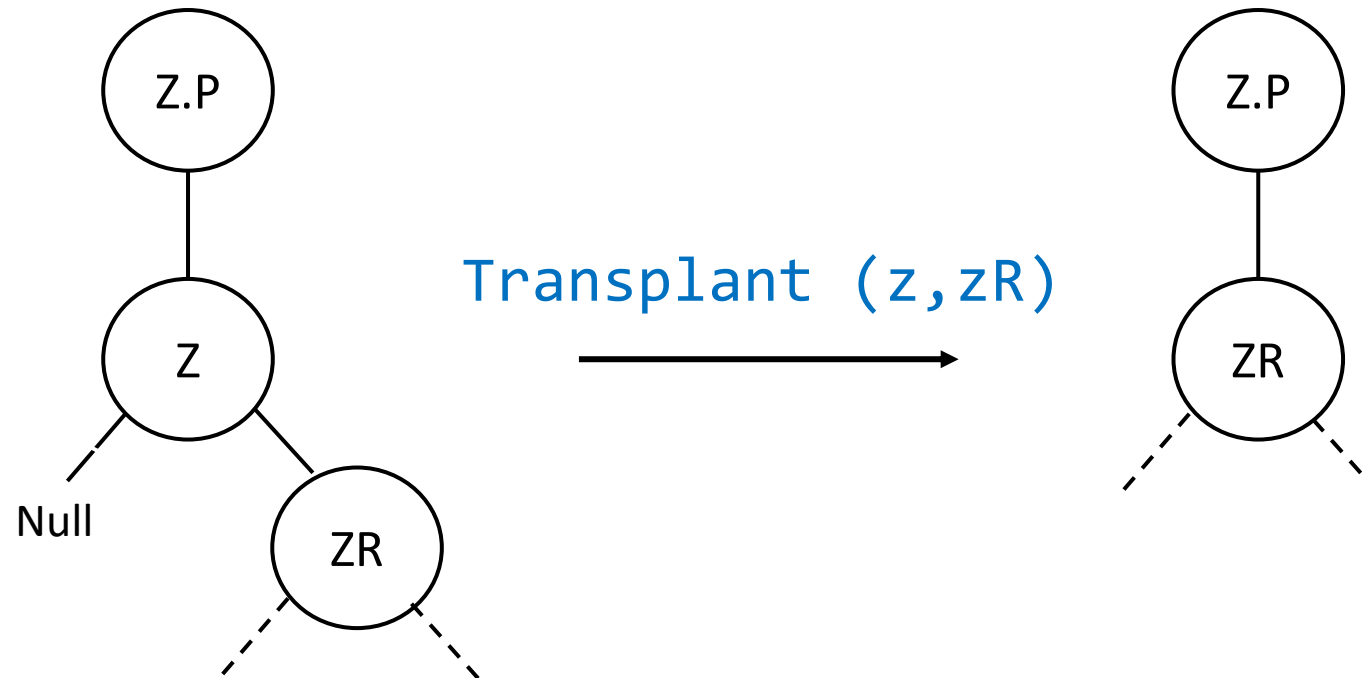
$y.left = z.left$   
 $z.left.p = y$



Tree\_delete(int key):

```
Tree_delete(int key):  
    Node* z = search(key)
```

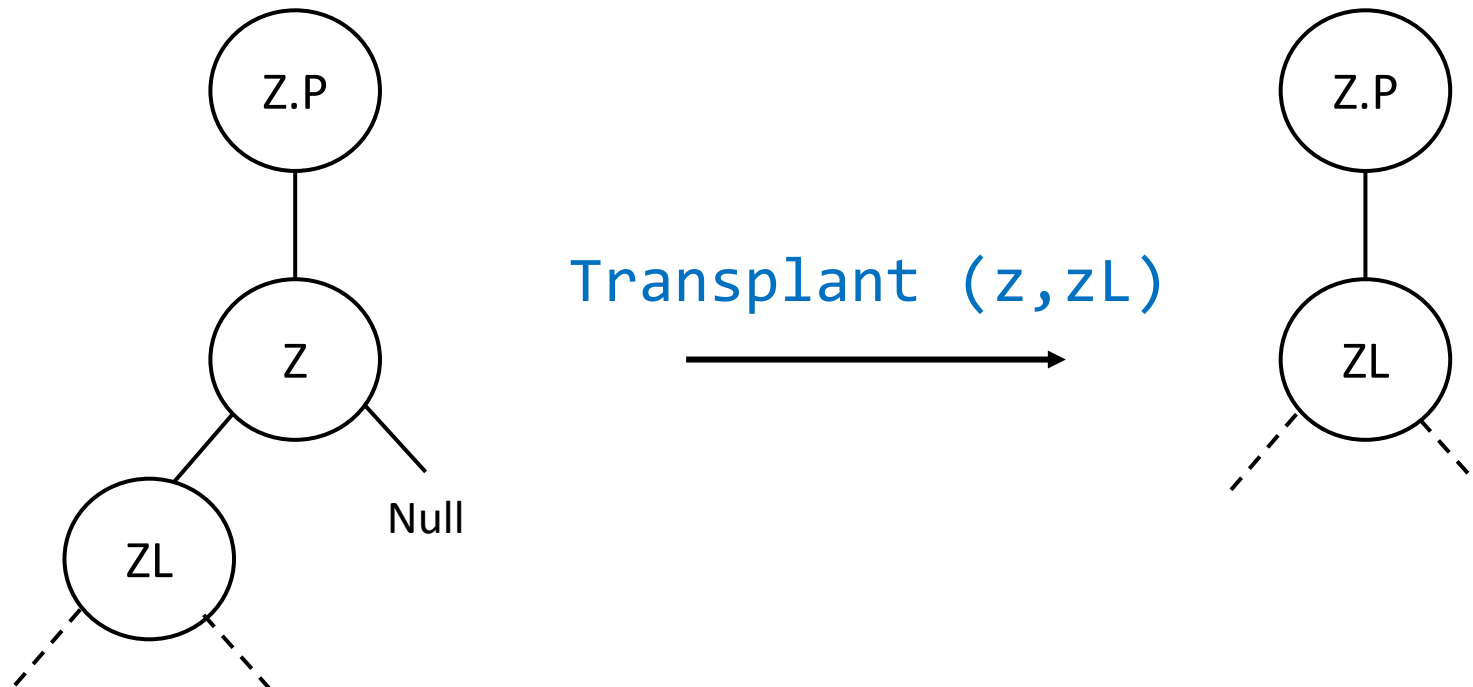
```
Tree_delete(int key):  
    Node* z = search(key)  
    if z.left == Null:  
        transplant(z, z.right)
```



```

Tree_delete(int key):
    Node* z = search(key)
    if z.left == Null:
        transplant(z, z.right)
    else if z.right == Null:
        transplant(z, z.left)

```



```

Tree_delete(int key):
    Node* z = search(key)
    if z.left == Null:
        transplant(z, z.right)
    else if z.right == Null:
        transplant(z, z.left)
    else
        y == minimum(z.right)

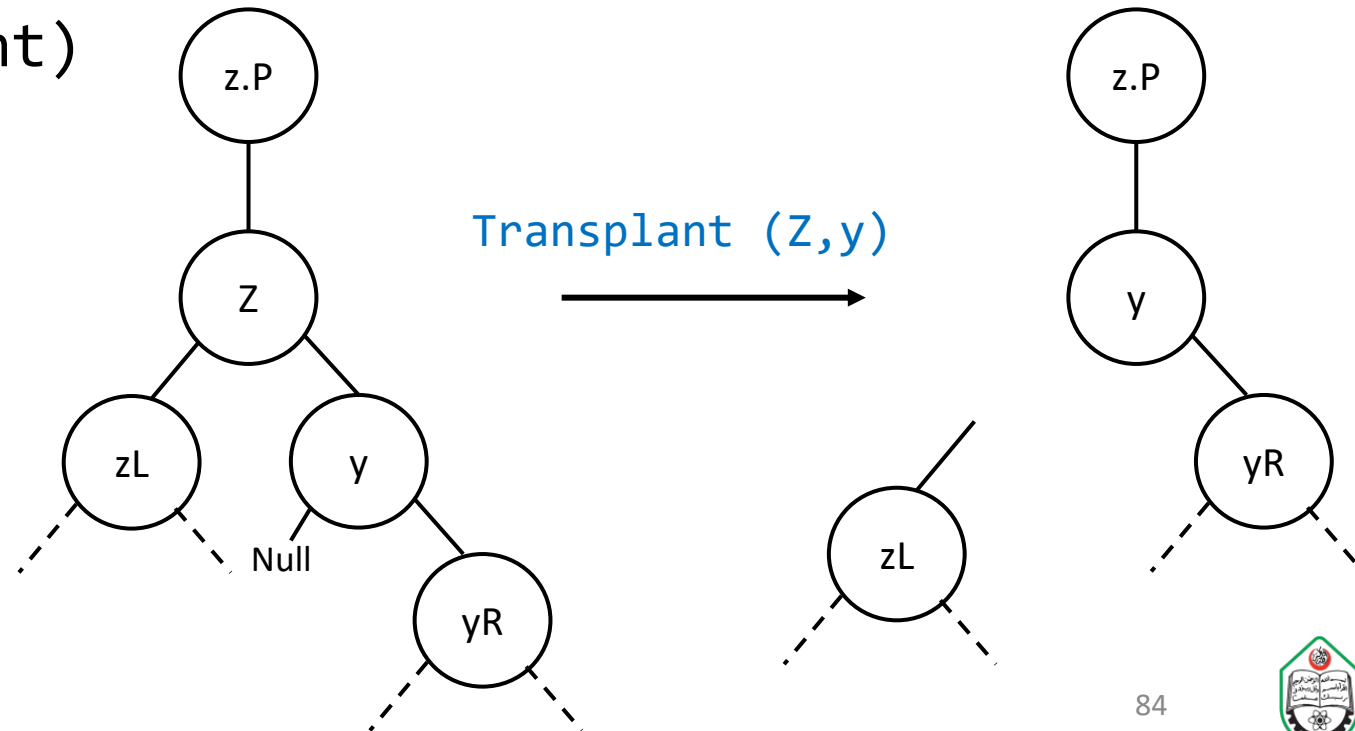
```

*(Assuming  $y.p == z$ )*

```

transplant(z,y)
y.left = z.left
z.left.p = y

```



```

Tree_delete(int key):
    Node* z = search(key)
    if z.left == Null:
        transplant(z, z.right)
    else if z.right == Null:
        transplant(z, z.left)
    else
        y == minimum(z.right)
        if y.p != z:
            transplant(y, y.right)
            y.right = z.right
            z.right.p = y
        transplant(z,y)
        y.left = z.left
        z.left.p = y

```

# References

1. Chapter 12, Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. 2009. Introduction to Algorithms, Third Edition (3rd. ed.). The MIT Press.

# Thank you